

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh City, October 2026 –

Table of Contents

I. Record of Changes 15
II. Software Design Document 16
1. System Design 16
1.1 System Architecture 16
1.2 Package Diagram 18
1.2.1 Backend Package Diagram 18
1.2.2 Auth Service Package Diagram 19
1.2.3 User Service Package Diagram 21
1.2.4 Order Service Package Diagram 22
1.2.5 Locker Service Package Diagram 24
1.2.6 Payment Service Package Diagram 25
1.2.7 Notification Service Package Diagram 27
1.2.8 IoT Service Package Diagram 28
1.2.9 Store Service Package Diagram 30
1.2.10 Loyalty Service Package Diagram 31
1.2.11 Assistant Service Package Diagram 33
1.2.12 Admin Web Application Package Diagram 34
1.2.13 Mobile Application Package Diagram 35
1.2.14 Mobile Feature Module Package Diagram 36
1.2.15 IoT Locker Controller Package Diagram 37
2. Database Design 38
2.1 auth_db (auth-service) 38
2.2 user_db (user-service) 39
2.3 store_db (store-service) 40
2.4 locker_db (locker-service) 41
2.5 order_db (order-service) 43
2.6 payment_db (payment-service) 46
2.7 notification_db (notification-service) 47
2.8 iot_db (iot-service) 48
2.9 loyalty_db (loyalty-service) 50
2.10 assistant_db (assistant-service) 51
3. Detailed Design 51

3.1 Authentication 52
3.1.1 Class Diagram 52
3.1.2 Sign in with Password 53
3.1.3 Sign in with Phone Number (Firebase OTP) 54
3.1.4 Administrator Sign in with 2FA 55
3.1.5 Refresh Token and Sign out 56
3.1.6 Class Specification 56
3.2 Kiosk Login by QR 57
3.2.1 Class Diagram 58
3.2.2 Kiosk Login by QR 59
3.2.3 Class Specification 59
3.3 Profile and User Management 60
3.3.1 Class Diagram 60
3.3.2 Update Profile Avatar 61
3.3.3 Administrator Creates and Deactivates a User 62
3.3.4 Class Specification 62
3.4 Store Management and Nearby Stores 63
3.4.1 Class Diagram 64
3.4.2 Find Nearby Stores 65
3.4.3 Class Specification 65
3.5 Locker and Box Management 66
3.5.1 Class Diagram 67
3.5.2 Create Locker and Box Grid 68
3.5.3 View Locker Layout 68
3.5.4 Class Specification 69
3.6 Send Parcel 69
3.6.1 Class Diagram 70
3.6.2 Create Send Order 71
3.6.3 Class Specification 71
3.7 VietQR Payment (SePay) 72
3.7.1 Class Diagram 73
3.7.2 Checkout with VietQR 74
3.7.3 SePay Webhook Completes the Payment 75

3.7.4 Class Specification 75
3.8 Drop and Pick Up Parcel at the Kiosk 76
3.8.1 Class Diagram 77
3.8.2 Open Box with PIN at the Kiosk 77
3.8.3 Confirm Parcel Drop 78
3.8.4 Class Specification 78
3.9 Rent Locker, Extend and End Rental 79
3.9.1 Class Diagram 79
3.9.2 Create Rental Order 80
3.9.3 Extend Rental 80
3.9.4 End Rental at the Kiosk 81
3.9.5 Class Specification 81
3.10 Delegate Pickup and Reset PIN 82
3.10.1 Class Diagram 82
3.10.2 Delegate Pickup 83
3.10.3 Reset PIN 83
3.10.4 Class Specification 84
3.11 Rate Order, Complaint and Feedback Handling 84
3.11.1 Class Diagram 85
3.11.2 Rate an Order 85
3.11.3 Administrator Replies to Feedback 86
3.11.4 Class Specification 86
3.12 Wallet Top-up and Withdrawal 87
3.12.1 Class Diagram 88
3.12.2 Top Up Wallet 88
3.12.3 Withdraw to Bank and Admin Approval 89
3.12.4 Class Specification 89
3.13 Refund Requests 90
3.13.1 Class Diagram 91
3.13.2 Request and Approve a Refund 92
3.13.3 Class Specification 92
3.14 Promotions, Vouchers and Loyalty Points 93
3.14.1 Class Diagram 93

3.14.2 Exchange Points for a Voucher 94
3.14.3 Earn Points when an Order Completes 94
3.14.4 Class Specification 95
3.15 Drone Delivery Booking 96
3.15.1 Class Diagram 96
3.15.2 Book a Drone Delivery 97
3.15.3 Class Specification 97
3.16 Drone Home Pickup 98
3.16.1 Class Diagram 99
3.16.2 Offer, Accept and Confirm a Home Pickup 100
3.16.3 Class Specification 100
3.17 Drone Dispatch, Loading and Launch 101
3.17.1 Class Diagram 102
3.17.2 Accept Drone Order and Confirm Loading 103
3.17.3 Launch the Drone 103
3.17.4 Class Specification 104
3.18 Drone Flight Tracking and Stage Progress 104
3.18.1 Class Diagram 105
3.18.2 Advance Flight Stage and Live Position 105
3.18.3 Class Specification 105
3.19 Drone Parcel Drop Incident 106
3.19.1 Class Diagram 107
3.19.2 Report a Dropped Parcel 108
3.19.3 Recovery, Proposal and Customer Decision 108
3.19.4 Class Specification 109
3.20 Locker Incident Ticket 109
3.20.1 Class Diagram 110
3.20.2 Customer Reports a Box Fault 110
3.20.3 Technician Claims and Resolves a Ticket 111
3.20.4 Class Specification 111
3.21 Periodic Inspection 112
3.21.1 Class Diagram 113
3.21.2 Plan and Perform a Periodic Inspection 114

3.21.3 Class Specification 114
3.22 Locker Controller Provisioning and Cabinet Control 115
3.22.1 Class Diagram 115
3.22.2 Assign a Controller to a Locker 116
3.22.3 Remote Open Box from the Admin Web 116
3.22.4 Class Specification 116
3.23 Bluetooth Open Test 117
3.23.1 Class Diagram 118
3.23.2 Open a Box by Bluetooth 118
3.23.3 Class Specification 119
3.24 Notifications and Realtime Updates 119
3.24.1 Class Diagram 120
3.24.2 Deliver a Domain Event to Users 120
3.24.3 Class Specification 121
3.25 AI Assistant (RAG) 121
3.25.1 Class Diagram 122
3.25.2 Ask the AI Assistant 122
3.25.3 Upload and Index a Knowledge Document 123
3.25.4 Class Specification 123
3.26 Business Settings 124
3.26.1 Class Diagram 125
3.26.2 Administrator Changes a Business Rule 125
3.26.3 Class Specification 126
3.27 Admin Dashboard and Revenue Reports 126
3.27.1 Class Diagram 127
3.27.2 View Revenue Summary 127
3.27.3 Class Specification 128
3.28 System Scheduled Jobs 128
3.28.1 Class Diagram 129
3.28.2 Expire Overdue Orders 130
3.28.3 Class Specification 130

List of Figures

Figure 1.1: System architecture of Lock.R 16
Figure 1.2.1: Backend package diagram (modules) 18
Figure 1.2.2: Auth Service package diagram 19
Figure 1.2.3: User Service package diagram 21
Figure 1.2.4: Order Service package diagram 22
Figure 1.2.5: Locker Service package diagram 24
Figure 1.2.6: Payment Service package diagram 25
Figure 1.2.7: Notification Service package diagram 27
Figure 1.2.8: IoT Service package diagram 28
Figure 1.2.9: Store Service package diagram 30
Figure 1.2.10: Loyalty Service package diagram 31
Figure 1.2.11: Assistant Service package diagram 33
Figure 1.2.12: Admin web application package diagram 34
Figure 1.2.13: Mobile application package diagram 35
Figure 1.2.14: Package structure inside one mobile feature module 36
Figure 1.2.15: IoT locker controller package diagram 37
Figure 2.1: Database diagram of auth_db 38
Figure 2.2: Database diagram of user_db 39
Figure 2.3: Database diagram of store_db 40
Figure 2.4: Database diagram of locker_db 41
Figure 2.5: Database diagram of order_db 43
Figure 2.6: Database diagram of payment_db 46
Figure 2.7: Database diagram of notification_db 47
Figure 2.8: Database diagram of iot_db 48
Figure 2.9: Database diagram of loyalty_db 50
Figure 2.10: Database diagram of assistant_db 51
Figure 3.1.1: Class diagram - Authentication 52
Figure 3.1.2: Sequence diagram - Sign in with Password 53
Figure 3.1.3: Sequence diagram - Sign in with Phone Number (Firebase OTP) 54
Figure 3.1.4: Sequence diagram - Administrator Sign in with 2FA 55
Figure 3.1.5: Sequence diagram - Refresh Token and Sign out 56
Figure 3.2.1: Class diagram - Kiosk Login by QR 58

Figure 3.2.2: Sequence diagram - Kiosk Login by QR 59
Figure 3.3.1: Class diagram - Profile and User Management 60
Figure 3.3.2: Sequence diagram - Update Profile Avatar 61
Figure 3.3.3: Sequence diagram - Administrator Creates and Deactivates a User 62
Figure 3.4.1: Class diagram - Store Management and Nearby Stores 64
Figure 3.4.2: Sequence diagram - Find Nearby Stores 65
Figure 3.5.1: Class diagram - Locker and Box Management 67
Figure 3.5.2: Sequence diagram - Create Locker and Box Grid 68
Figure 3.5.3: Sequence diagram - View Locker Layout 68
Figure 3.6.1: Class diagram - Send Parcel 70
Figure 3.6.2: Sequence diagram - Create Send Order 71
Figure 3.7.1: Class diagram - VietQR Payment (SePay) 73
Figure 3.7.2: Sequence diagram - Checkout with VietQR 74
Figure 3.7.3: Sequence diagram - SePay Webhook Completes the Payment 75
Figure 3.8.1: Class diagram - Drop and Pick Up Parcel at the Kiosk 77
Figure 3.8.2: Sequence diagram - Open Box with PIN at the Kiosk 77
Figure 3.8.3: Sequence diagram - Confirm Parcel Drop 78
Figure 3.9.1: Class diagram - Rent Locker, Extend and End Rental 79
Figure 3.9.2: Sequence diagram - Create Rental Order 80
Figure 3.9.3: Sequence diagram - Extend Rental 80
Figure 3.9.4: Sequence diagram - End Rental at the Kiosk 81
Figure 3.10.1: Class diagram - Delegate Pickup and Reset PIN 82
Figure 3.10.2: Sequence diagram - Delegate Pickup 83
Figure 3.10.3: Sequence diagram - Reset PIN 83
Figure 3.11.1: Class diagram - Rate Order, Complaint and Feedback Handling 85
Figure 3.11.2: Sequence diagram - Rate an Order 85
Figure 3.11.3: Sequence diagram - Administrator Replies to Feedback 86
Figure 3.12.1: Class diagram - Wallet Top-up and Withdrawal 88
Figure 3.12.2: Sequence diagram - Top Up Wallet 88
Figure 3.12.3: Sequence diagram - Withdraw to Bank and Admin Approval 89
Figure 3.13.1: Class diagram - Refund Requests 91
Figure 3.13.2: Sequence diagram - Request and Approve a Refund 92
Figure 3.14.1: Class diagram - Promotions, Vouchers and Loyalty Points 93

Figure 3.14.2: Sequence diagram - Exchange Points for a Voucher 94
Figure 3.14.3: Sequence diagram - Earn Points when an Order Completes 94
Figure 3.15.1: Class diagram - Drone Delivery Booking 96
Figure 3.15.2: Sequence diagram - Book a Drone Delivery 97
Figure 3.16.1: Class diagram - Drone Home Pickup 99
Figure 3.16.2: Sequence diagram - Offer, Accept and Confirm a Home Pickup 100
Figure 3.17.1: Class diagram - Drone Dispatch, Loading and Launch 102
Figure 3.17.2: Sequence diagram - Accept Drone Order and Confirm Loading 103
Figure 3.17.3: Sequence diagram - Launch the Drone 103
Figure 3.18.1: Class diagram - Drone Flight Tracking and Stage Progress 105
Figure 3.18.2: Sequence diagram - Advance Flight Stage and Live Position 105
Figure 3.19.1: Class diagram - Drone Parcel Drop Incident 107
Figure 3.19.2: Sequence diagram - Report a Dropped Parcel 108
Figure 3.19.3: Sequence diagram - Recovery, Proposal and Customer Decision 108
Figure 3.20.1: Class diagram - Locker Incident Ticket 110
Figure 3.20.2: Sequence diagram - Customer Reports a Box Fault 110
Figure 3.20.3: Sequence diagram - Technician Claims and Resolves a Ticket 111
Figure 3.21.1: Class diagram - Periodic Inspection 113
Figure 3.21.2: Sequence diagram - Plan and Perform a Periodic Inspection 114
Figure 3.22.1: Class diagram - Locker Controller Provisioning and Cabinet Control 115
Figure 3.22.2: Sequence diagram - Assign a Controller to a Locker 116
Figure 3.22.3: Sequence diagram - Remote Open Box from the Admin Web 116
Figure 3.23.1: Class diagram - Bluetooth Open Test 118
Figure 3.23.2: Sequence diagram - Open a Box by Bluetooth 118
Figure 3.24.1: Class diagram - Notifications and Realtime Updates 120
Figure 3.24.2: Sequence diagram - Deliver a Domain Event to Users 120
Figure 3.25.1: Class diagram - AI Assistant (RAG) 122
Figure 3.25.2: Sequence diagram - Ask the AI Assistant 122
Figure 3.25.3: Sequence diagram - Upload and Index a Knowledge Document 123
Figure 3.26.1: Class diagram - Business Settings 125
Figure 3.26.2: Sequence diagram - Administrator Changes a Business Rule 125
Figure 3.27.1: Class diagram - Admin Dashboard and Revenue Reports 127
Figure 3.27.2: Sequence diagram - View Revenue Summary 127

Figure 3.28.1: Class diagram - System Scheduled Jobs 129
Figure 3.28.2: Sequence diagram - Expire Overdue Orders 130

List of Tables

Table 1.1: Components of the system architecture 16
Table 1.2.1: Backend modules and common-lib packages 18
Table 1.2.2: Packages of auth-service 20
Table 1.2.3: Packages of user-service 21
Table 1.2.4: Packages of order-service 22
Table 1.2.5: Packages of locker-service 24
Table 1.2.6: Packages of payment-service 26
Table 1.2.7: Packages of notification-service 27
Table 1.2.8: Packages of iot-service 29
Table 1.2.9: Packages of store-service 30
Table 1.2.10: Packages of loyalty-service 31
Table 1.2.11: Packages of assistant-service 33
Table 1.2.12: Packages of the admin web application 34
Table 1.2.13: Packages of the mobile application 35
Table 1.2.14: Layers of a mobile feature module 37
Table 1.2.15: Packages of the IoT locker controller 37
Table 2.1: Table descriptions of auth_db 39
Table 2.2: Table descriptions of user_db 40
Table 2.3: Table descriptions of store_db 40
Table 2.4: Table descriptions of locker_db 41
Table 2.5: Table descriptions of order_db 43
Table 2.6: Table descriptions of payment_db 46
Table 2.7: Table descriptions of notification_db 47
Table 2.8: Table descriptions of iot_db 48
Table 2.9: Table descriptions of loyalty_db 50
Table 2.10: Table descriptions of assistant_db 51
Table 3.1.6.a: AuthController description 56
Table 3.1.6.b: AuthService description 57
Table 3.1.6.c: JwtService description 57
Table 3.1.6.d: EmailOtpService description 57
Table 3.2.3.a: KioskQrLoginController description 59
Table 3.2.3.b: KioskQrLoginService description 60

Table 3.3.4.a: UserController description 62
Table 3.3.4.b: MediaController description 63
Table 3.3.4.c: UserProfileService description 63
Table 3.4.3.a: StoreController description 65
Table 3.4.3.b: StoreService description 66
Table 3.5.4.a: LockerController description 69
Table 3.5.4.b: LockerService description 69
Table 3.6.3.a: OrderController description 71
Table 3.6.3.b: OrderService description 71
Table 3.6.3.c: OrderBenefitService description 72
Table 3.6.3.d: PaidOrderEffects description 72
Table 3.7.4.a: PaymentController description 75
Table 3.7.4.b: PaymentService description 75
Table 3.7.4.c: SepayService description 76
Table 3.7.4.d: OrderPaymentEventListener description 76
Table 3.7.4.e: PaidOrderEffects description 76
Table 3.8.4.a: IotController description 78
Table 3.8.4.b: IotService description 78
Table 3.8.4.c: LockerMqttService description 79
Table 3.9.5.a: OrderController description 81
Table 3.9.5.b: OrderService description 81
Table 3.10.4.a: OrderController description 84
Table 3.10.4.b: OrderService description 84
Table 3.10.4.c: QrTokenService description 84
Table 3.11.4.a: OrderController description 86
Table 3.11.4.b: AdminFeedbackController description 86
Table 3.11.4.c: OrderService description 87
Table 3.11.4.d: AdminFeedbackService description 87
Table 3.12.4.a: WalletController description 89
Table 3.12.4.b: PaymentController description 90
Table 3.12.4.c: WalletService description 90
Table 3.13.3.a: PaymentController description 92
Table 3.13.3.b: AdminPaymentController description 92

Table 3.13.3.c: PaymentService description 93
Table 3.14.4.a: OrderController description 95
Table 3.14.4.b: VoucherExchangeService description 95
Table 3.14.4.c: LoyaltyController description 95
Table 3.14.4.d: LoyaltyService description 95
Table 3.14.4.e: OrderCompletedPointsListener description 96
Table 3.15.3.a: OrderController description 97
Table 3.15.3.b: DroneOrderBookingService description 98
Table 3.15.3.c: DroneSourceLockerSelector description 98
Table 3.15.3.d: DroneWeatherService description 98
Table 3.16.3.a: OrderController description 100
Table 3.16.3.b: DronePickupService description 101
Table 3.17.4.a: OrderController description 104
Table 3.17.4.b: DroneOrderMaintenanceService description 104
Table 3.18.3.a: OrderController description 105
Table 3.18.3.b: DroneMissionProgressService description 106
Table 3.18.3.c: DroneTelemetryMqttListener description 106
Table 3.18.3.d: DroneTelemetryService description 106
Table 3.19.4.a: DroneParcelIncidentController description 109
Table 3.19.4.b: DroneParcelIncidentService description 109
Table 3.20.4.a: OrderController description 111
Table 3.20.4.b: LockerController description 111
Table 3.20.4.c: OrderService description 112
Table 3.20.4.d: LockerService description 112
Table 3.21.3.a: LockerController description 114
Table 3.21.3.b: LockerService description 114
Table 3.21.3.c: LockerScheduler description 115
Table 3.22.4.a: GatewayController description 116
Table 3.22.4.b: CabinetControlController description 117
Table 3.22.4.c: GatewayProvisioningService description 117
Table 3.22.4.d: CabinetControlService description 117
Table 3.23.3.a: BleTestController description 119
Table 3.23.3.b: BleTestService description 119

Table 3.24.3.a: NotificationController description 121
Table 3.24.3.b: NotificationEventListener description 121
Table 3.24.3.c: NotificationService description 121
Table 3.25.4.a: AssistantController description 123
Table 3.25.4.b: KnowledgeAdminController description 123
Table 3.25.4.c: AssistantService description 124
Table 3.25.4.d: KnowledgeService description 124
Table 3.25.4.e: KnowledgeIndexer description 124
Table 3.26.3.a: BusinessSettingsController description 126
Table 3.26.3.b: BusinessSettings description 126
Table 3.27.3.a: AdminRevenueController description 128
Table 3.27.3.b: RevenueReportService description 128
Table 3.28.3.a: OrderScheduler description 130
Table 3.28.3.b: OrderController description 130

I. Record of Changes

Date	A* M, D	In charge	Change Description
28/10/2026	A		Initial Software Design Document: system architecture, package diagrams of the backend services, admin web, mobile app and IoT controller, database design of the 10 service databases, class diagrams, sequence diagrams and class specifications of 28 features.

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

Lock.R is built as a set of Spring Boot microservices behind one API gateway. Every client (mobile app, admin web and the kiosk on each locker) talks to the backend over HTTPS REST and receives realtime updates over STOMP / WebSocket. Each service owns its PostgreSQL database; services call each other synchronously with OpenFeign (internal endpoints) and asynchronously through the RabbitMQ topic exchange laundry.events. Physical lockers are driven by a Raspberry Pi that exchanges commands and events with iot-service over MQTT. The whole backend runs with Docker Compose on one Azure VM behind Nginx; the web front-ends are deployed on Cloudflare Workers.

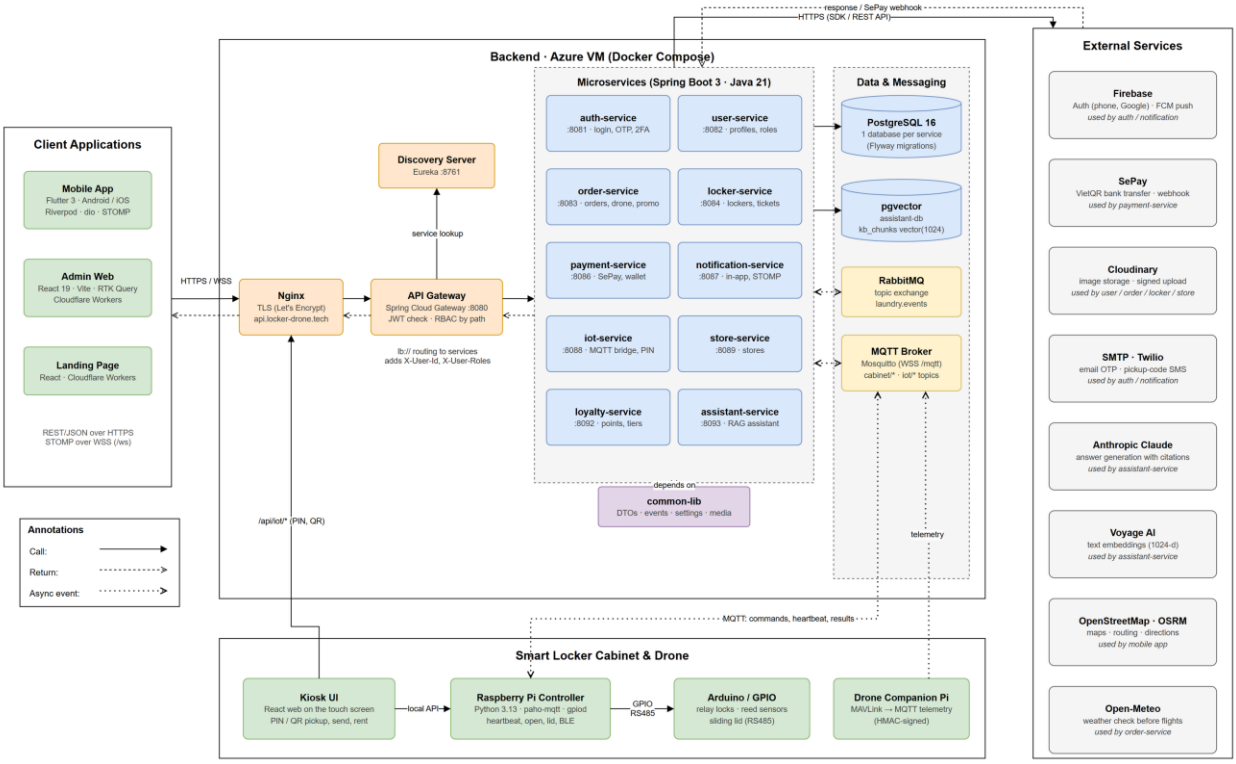


Figure 1.1: System architecture of Lock.R

The table below explains each component of the architecture diagram.

Table 1.1: Components of the system architecture

No	Component	Description
01	Mobile App	Flutter application for customers, receivers, locker technicians and drone technicians (Android and iOS). Uses dio for REST, STOMP for realtime events, Firebase Auth for phone / Google sign-in and FCM for push.
02	Admin Web	React 19 + Vite + RTK Query single-page app for administrators: dashboard, users, stores, lockers, cabinet control, maintenance, drones, orders, payments, refunds, promotions, loyalty, settings and knowledge base.

No	Component	Description
03	Kiosk UI	React web app shown on the touch screen of each locker. Customers pick up with PIN / QR, sign in by QR, send and rent; it calls /api/iot/* and the local API of the Pi.
04	Nginx	TLS termination (Let's Encrypt) for api.locker-drone.tech; forwards HTTP and WebSocket traffic to the API gateway and MQTT over WSS to the broker.
05	API Gateway	Spring Cloud Gateway. Verifies the JWT, applies role-based access by path prefix, blocks /internal/**, adds X-User-Id / X-Account-Id / X-User-Roles headers and routes to services through Eureka (lb://).
06	Discovery Server	Eureka registry where every service registers and the gateway / Feign clients resolve service instances.
07	auth-service	Credentials, password / OTP / Firebase / kiosk-QR sign-in, admin 2FA, JWT and refresh tokens.
08	user-service	User profiles, roles and status; admin user management; signed Cloudinary upload parameters.
09	order-service	SEND / RENTAL / DRONE_DELIVERY orders, pricing, vouchers and points, ratings and complaints, drone missions, pickups, telemetry, incidents, revenue reports and scheduled jobs.
10	locker-service	Lockers, boxes and layouts, tickets and repair logs, periodic inspections, technician presence, drones and drone devices.
11	payment-service	Payments (SePay VietQR, wallet), SePay webhook, refunds, wallet ledger and withdrawals.
12	notification-service	In-app notifications, FCM push, STOMP /ws broker, SMS / email to receivers without an account.
13	iot-service	MQTT bridge to lockers: open / lid / setup commands, heartbeats and door states, PIN / QR verification, controller provisioning, BLE open tokens.
14	store-service	Stores (locations hosting lockers) and nearby search.
15	loyalty-service	Loyalty accounts, points, tiers, redemption and voucher exchange.
16	assistant-service	RAG assistant: knowledge base indexing, vector search on pgvector, answers from Claude with citations.
17	common-lib	Shared library: ApiResponse / PageResponse DTOs, domain events, exceptions, business settings framework, Cloudinary media, security and utilities.
18	PostgreSQL / pgvector	One PostgreSQL 16 database per service (Flyway migrations, ddl validate); a separate pgvector database for the assistant.
19	RabbitMQ	Topic exchange laundry.events with queues order.payment.events, loyalty.order.events and notification.events.
20	MQTT Broker	Mosquitto broker for cabinet/{lockerId}/command/*, heartbeat, status and iot/{mac}/* topics, and drone telemetry.
21	Raspberry Pi Controller	Python controller in each locker: drives relays, door sensors and the sliding lid through GPIO / Arduino RS485, sends heartbeats and results, verifies BLE tokens offline.
22	External services	Firebase (Auth, FCM), SePay (VietQR + webhook), Cloudinary (images), SMTP and Twilio (OTP, pickup codes), Anthropic Claude and Voyage AI (assistant), OpenStreetMap / OSRM (maps), Open-Meteo (flight weather).

1.2 Package Diagram

The package diagrams show how the source code of each sub-system is organised. The backend is a Maven multi-module project; every business service follows the same layered structure (API, application, domain, infrastructure) and depends on common-lib.

1.2.1 Backend Package Diagram

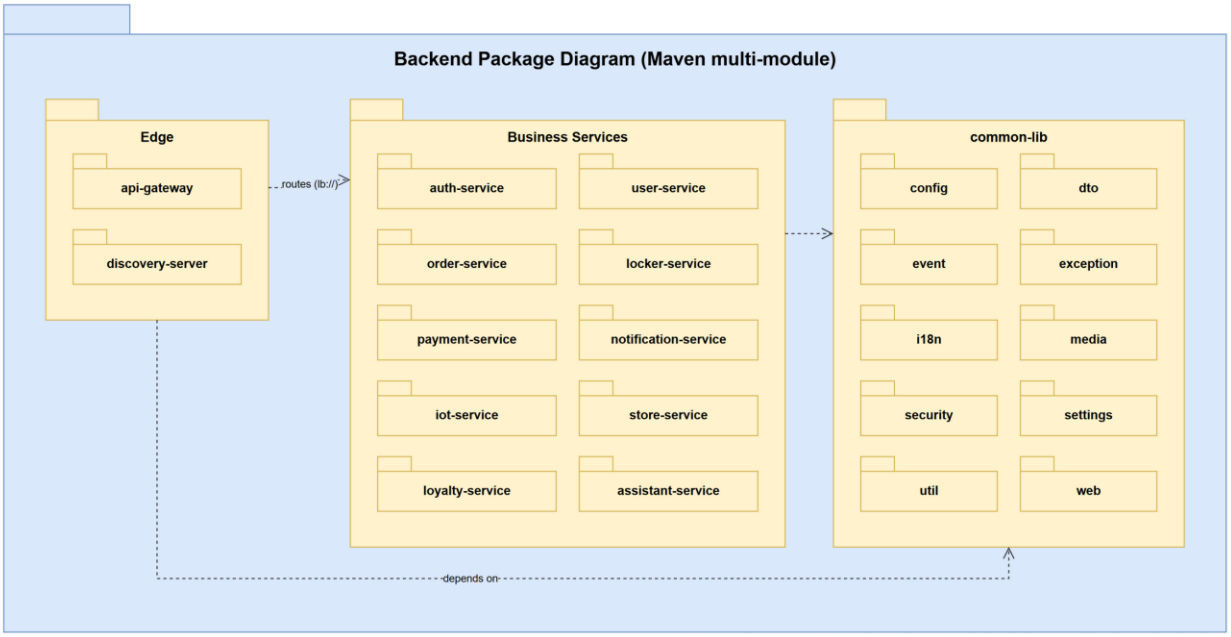


Figure 1.2.1: Backend package diagram (modules)

Table 1.2.1: Backend modules and common-lib packages

No	Package	Description
01	api-gateway	Spring Cloud Gateway: JWT validation, RBAC by path, routing, correlation id.
02	discovery-server	Eureka service registry.
03	auth-service	Authentication and token issuing.
04	user-service	User profiles, roles and media signatures.
05	order-service	Orders, promotions, drone delivery and reports.
06	locker-service	Lockers, boxes, tickets, inspections and drones.
07	payment-service	Payments, refunds, wallet and withdrawals.
08	notification-service	Notifications, push and realtime messaging.
09	iot-service	MQTT bridge and access control of the lockers.
10	store-service	Stores and nearby search.
11	loyalty-service	Points, tiers and voucher exchange.
12	assistant-service	AI assistant with retrieval-augmented generation.
13	common-lib/config	Flyway repair configuration shared by all services.

No	Package	Description
14	common-lib/dto	ApiResponse, ApiError, PageResponse, IdResponse and summaries (UserSummary, OrderSummary, LockerBoxSummary).
15	common-lib/event	DomainEvent, DomainEventNames (exchange laundry.events and routing keys) and change-event helpers.
16	common-lib/exception	BusinessException, NotFoundException and GlobalExceptionHandler (uniform error body).
17	common-lib/i18n	MessageService for Vietnamese / English messages.
18	common-lib/media	CloudinaryMediaStorage: signed upload parameters and verification of uploaded images.
19	common-lib/security	UserRoles constants and SecuritySecrets checks.
20	common-lib/settings	BusinessSettings framework: catalog, JDBC store with audit, admin / public controller.
21	common-lib/util	AfterCommit, BusinessTime (Asia/Ho_Chi_Minh) and CodeGenerator.
22	common-lib/web	CorrelationIdFilter for request tracing.

1.2.2 Auth Service Package Diagram

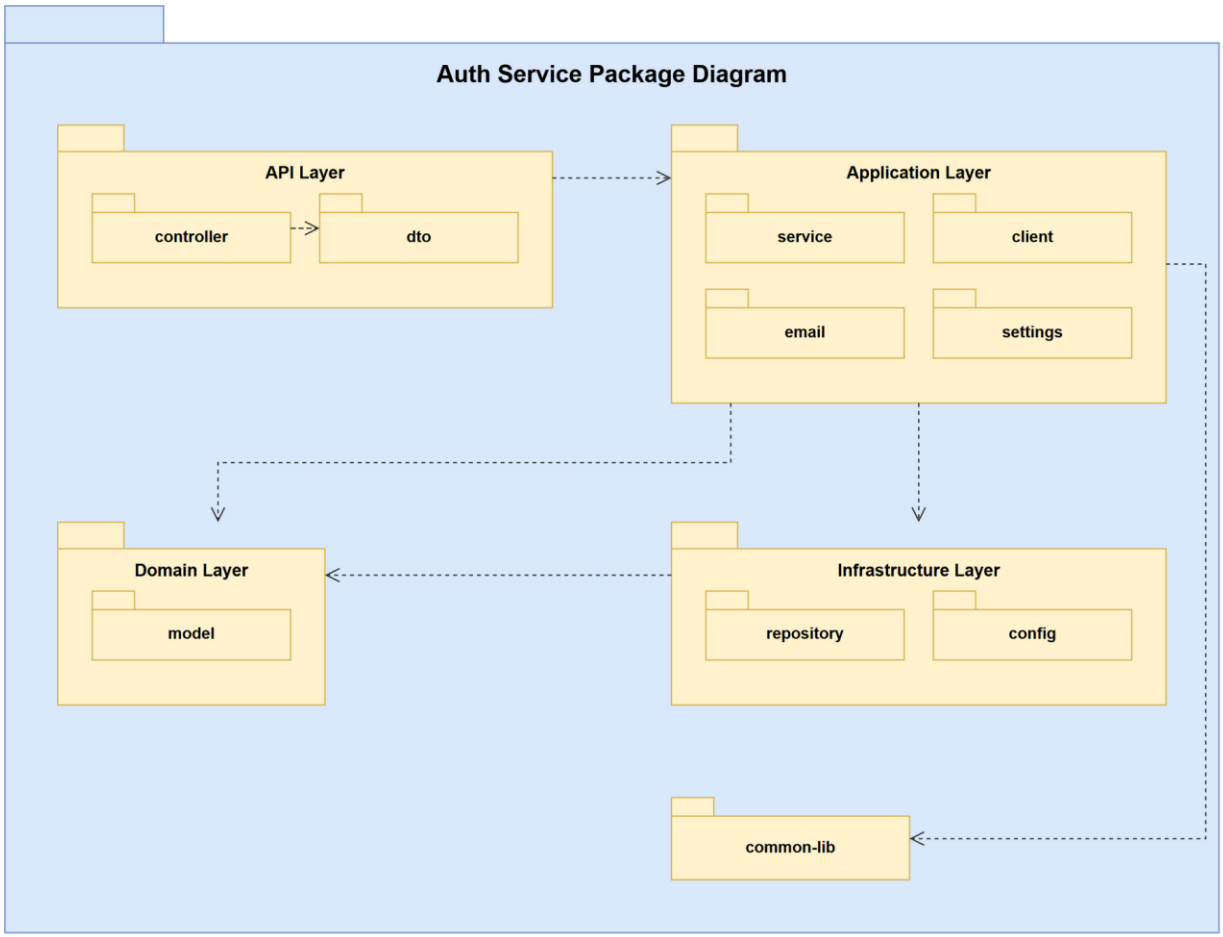


Figure 1.2.2: Auth Service package diagram

Table 1.2.2: Packages of auth-service

No	Package	Description
01	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: UserClient.
02	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: AdminBootstrap, FirebaseConfig, SecurityConfig.
03	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: AuthController, KioskQrLoginController.
04	dto	Request and response records exchanged with clients and other services. Main classes: AccountIdentifiersRequest, AccountStatusRequest, AuthResponse, CreateAccountRequest, FirebaseLoginRequest, KioskQrDtos, LoginRequest, LogoutRequest … (11 classes).
05	email	SMTP e-mail sender used for OTP codes. Main classes: EmailService, SmtpEmailService.
06	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AuthAccount, EmailOtp, RefreshToken, SocialIdentity.
07	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: AuthAccountRepository, EmailOtpRepository, RefreshTokenRepository, SocialIdentityRepository.
08	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: AuthService, EmailOtpService, JwtService, KioskQrLoginService.
09	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: AuthRules, AuthSettingsCatalog.

1.2.3 User Service Package Diagram

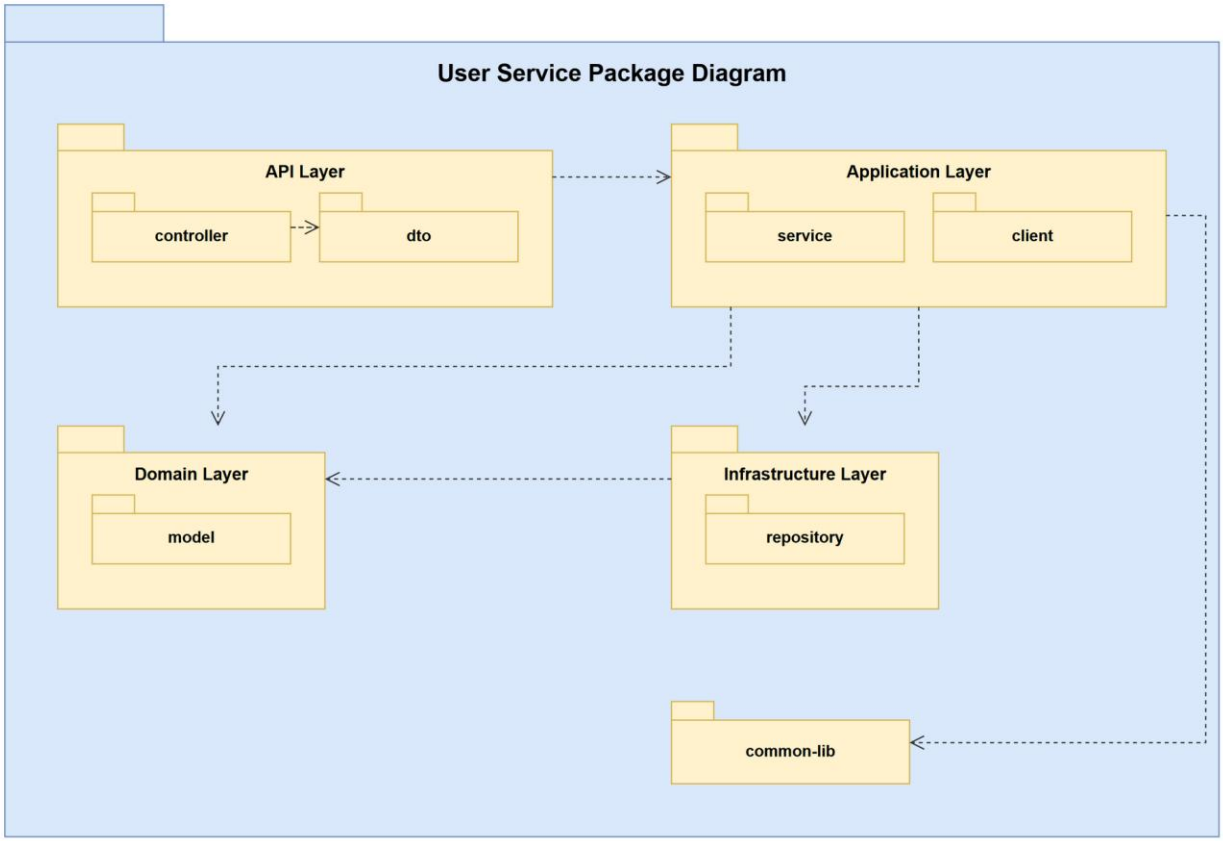


Figure 1.2.3: User Service package diagram

Table 1.2.3: Packages of user-service

No	Package	Description
01	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: AuthClient, NotificationClient.
02	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: InternalUserLookupController, MediaController, UserController.
03	dto	Request and response records exchanged with clients and other services. Main classes: AdminCreateUserRequest, AdminUserView, UserGrowthPoint, UserProfileRequest, UserProfileView.
04	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AdminDataChangeListener, UserProfile.
05	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: UserProfileRepository.
06	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: UserProfileService.

1.2.4 Order Service Package Diagram

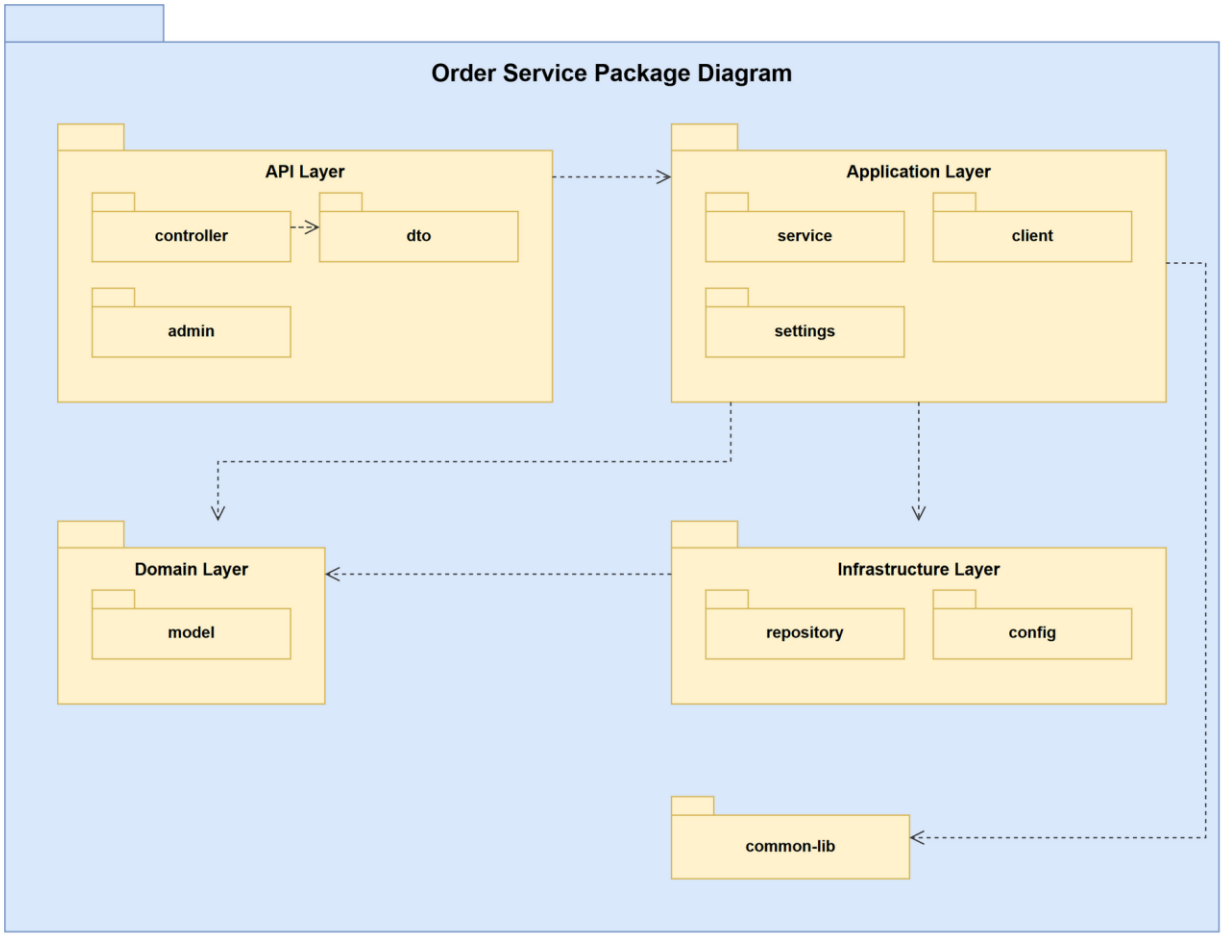


Figure 1.2.4: Order Service package diagram

Table 1.2.4: Packages of order-service

No	Package	Description
01	admin	Response DTOs used by the admin web lists and reports. Main classes: AdminOrderResponse, BoxInfo, CollectedPayments, FeedbackDtos, LockerInfo, OrderBriefResponse, OrderPaymentSummary, RevenueDtos … (9 classes).
02	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: DronePositionClient, LockerCellClient, LockerClient, LockerDroneClient, LockerLookupClient, LoyaltyClient, NotificationClient, PaymentRefundClient … (12 classes).
03	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: RabbitConfig.

No	Package	Description
04	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: AdminFeedbackController, AdminOrderController, AdminRevenueController, DroneLockerCommLogController, DroneParcelIncidentController, OrderController.
05	dto	Request and response records exchanged with clients and other services. Main classes: AcceptDroneOrderRequest, ActiveBoxOrderResponse, AdvanceDroneOrderRequest, AssignDronePickupRequest, AssignDroneRecoveryRequest, BoxCellDto, BoxIncidentNotice, CancelDroneOrderRequest … (81 classes).
06	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AdminDataChangeListener, DroneDeliverySession, DroneFailureReasons, DroneIncidentEvidence, DroneIncidentResolutionProposal, DroneIncidentStates, DroneIncidentTimeline, DroneLockerCommLog … (22 classes).
07	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: DroneDeliverySessionRepository, DroneIncidentEvidenceRepository, DroneIncidentResolutionProposalRepository, DroneIncidentTimelineRepository, DroneLockerCommLogRepository, DroneMissionRepository, DroneParcelIncidentRepository, DronePickupOfferRepository … (19 classes).
08	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: AdminFeedbackService, AdminOrderQueryService, AdminReferenceResolver, DashboardInsightService, DroneDeliveryBlockService, DroneDeliveryFailureService, DroneDeliveryMailer, DroneDeliveryQueryService … (55 classes).
09	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: OrderRules, OrderSettingsCatalog.

1.2.5 Locker Service Package Diagram

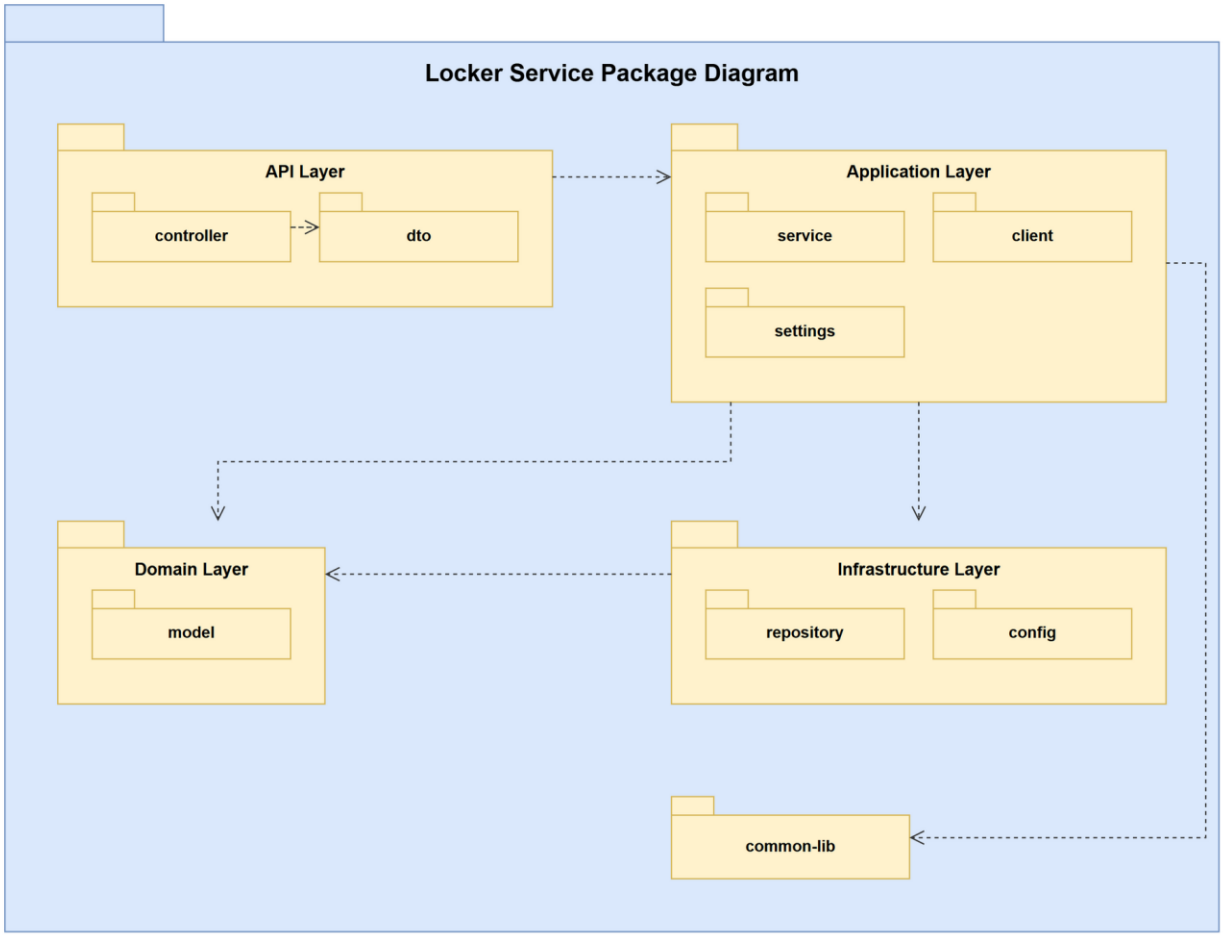


Figure 1.2.5: Locker Service package diagram

Table 1.2.5: Packages of locker-service

No	Package	Description
01	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: IotClient, OrderClient, UserClient.
02	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: RabbitConfig.
03	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: DroneDeliveryController, InternalLockerLookupController, LockerController.
04	dto	Request and response records exchanged with clients and other services. Main classes: ActiveBoxOrderDto, AssignDroneDeviceRequest, AssignTechnicianScheduleRequest, BatchCreateBoxesRequest, BatchDeleteBoxesRequest, BatchDeleteLockersRequest, BatchDeleteReportsRequest, BoxAnomalyResponse … (61 classes).

No	Package	Description
05	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AttachmentStage, CellType, DroneDeliveryRequest, DroneDevice, DroneIotLog, DroneMaintenanceLog, DroneStatus, DroneUnit … (20 classes).
06	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: DroneDeliveryRequestRepository, DroneDeviceRepository, DroneIotLogRepository, DroneMaintenanceLogRepository, DroneUnitRepository, LockerBoxRepository, LockerReportRatingRepository, LockerReportRepository … (14 classes).
07	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: DroneDeliveryService, LockerLookupService, LockerScheduler, LockerService, ReportAttachmentService, TechnicianPresenceService.
08	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: LockerRules, LockerSettingsCatalog.

1.2.6 Payment Service Package Diagram

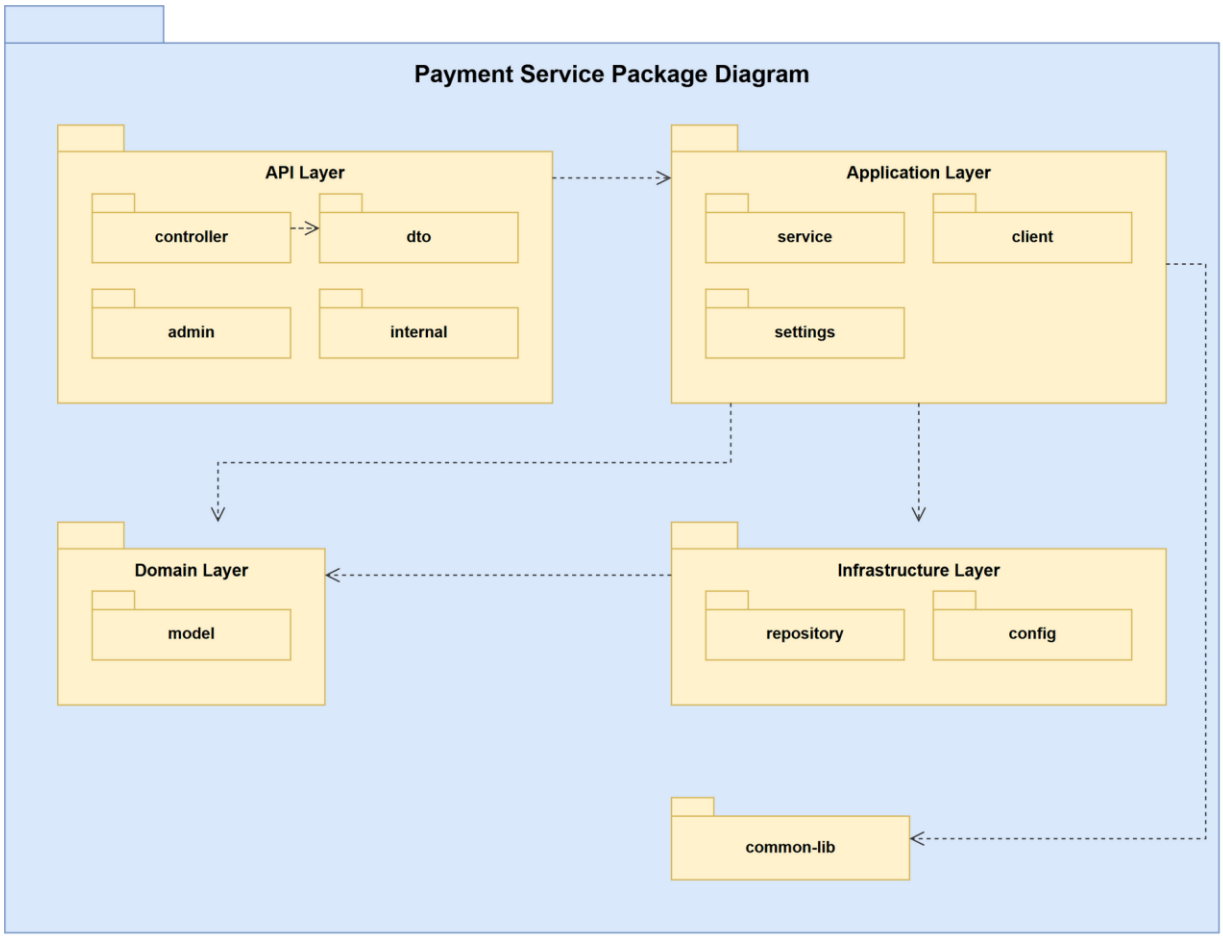


Figure 1.2.6: Payment Service package diagram

Table 1.2.6: Packages of payment-service

No	Package	Description
01	admin	Response DTOs used by the admin web lists and reports. Main classes: AdminPaymentDetailResponse, AdminPaymentResponse, AdminRefundResponse, AdminWalletTransactionResponse, OrderBrief, OrderRef, PaymentStatsResponse, PersonRef.
02	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: NotificationClient, OrderClient, OrderLookupClient, UserLookupClient.
03	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: RabbitConfig.
04	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: AdminPaymentController, InternalPaymentReportController, PaymentController, WalletController.
05	dto	Request and response records exchanged with clients and other services. Main classes: AdminWithdrawProcessRequest, CheckoutRequest, CreatePaymentRequest, CreateRefundRequest, CreateTopupRequest, PaymentResponse, ProcessRefundRequest, RefundRequest … (23 classes).
06	internal	DTOs of the /internal/** API that only other services call. Main classes: CollectedPaymentsResponse, CompensationRefundResult, OrderPaymentSummary, OrderRefundResult.
07	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AdminDataChangeListener, PaymentRecord, RefundRecord, UserBankAccount, Wallet, WalletTransaction, WithdrawalRecord.
08	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: PaymentRepository, PaymentSpecifications, RefundRepository, UserBankAccountRepository, WalletRepository, WalletTransactionRepository, WithdrawalRepository.
09	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: AdminPaymentQueryService, InternalPaymentReportService, PaymentReferenceResolver, PaymentReportRules, PaymentService, PaymentStatsService, SepayService, WalletEvents … (9 classes).
10	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: PaymentRules, PaymentSettingsCatalog.

1.2.7 Notification Service Package Diagram

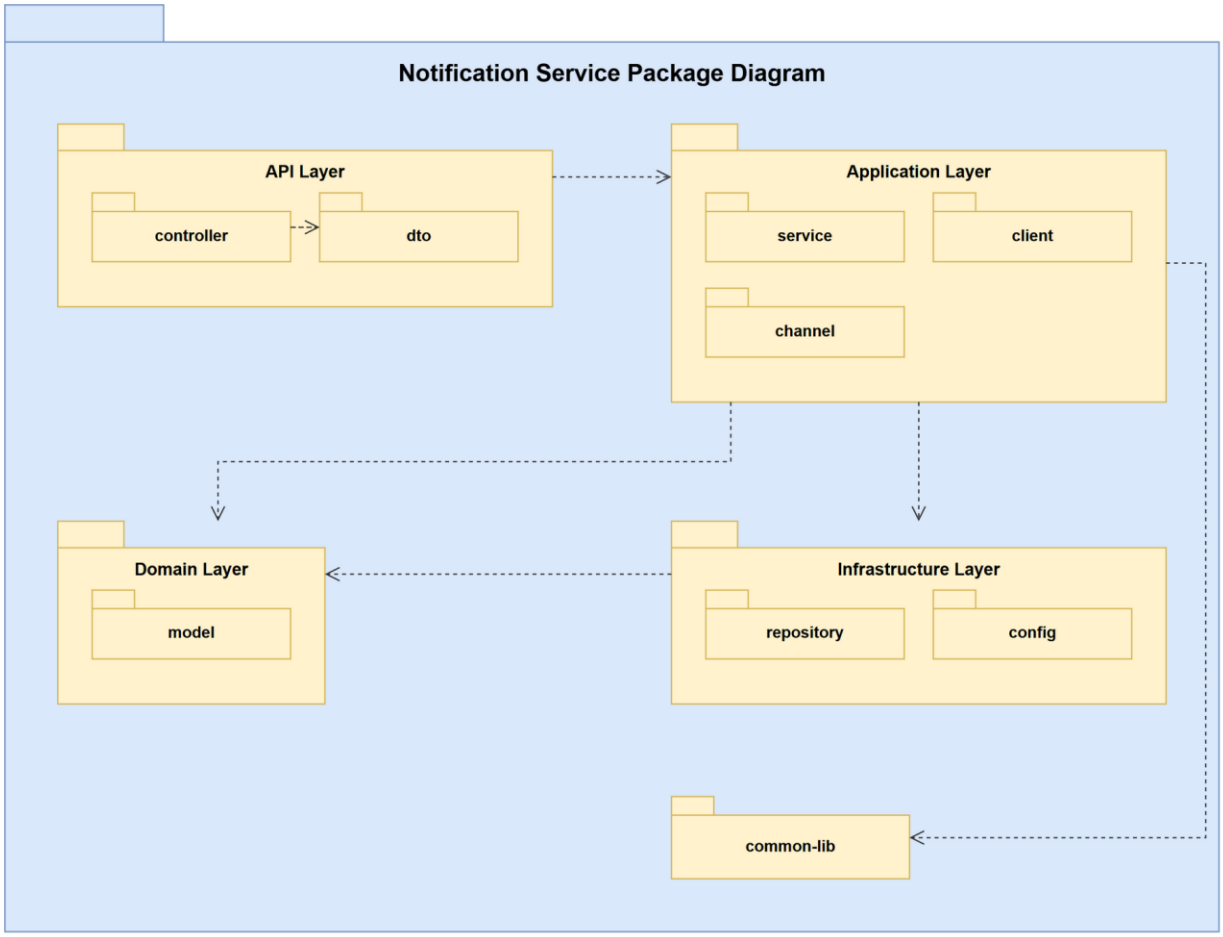


Figure 1.2.7: Notification Service package diagram

Table 1.2.7: Packages of notification-service

No	Package	Description
01	channel	SMS and e-mail channels (Twilio, eSMS, logging fallback) used for pickup codes of guests. Main classes: EsmsSmsSender, GuestEmailSender, LoggingSmsSender, PhoneNumbers, SmsChannelConfig, SmsSender, TwilioSmsSender.
02	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: OrderParticipantClient.
03	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: BrokerDestinationGuardInterceptor, DeliveryTopicSubscriptionInterceptor, FirebaseConfig, JwtHandshakePrincipalHandler, JwtStompPrincipalChannelInterceptor, RabbitConfig, StompPrincipal, WebSocketConfig.
04	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: NotificationController.

No	Package	Description
05	dto	Request and response records exchanged with clients and other services. Main classes: AdminBroadcastRequest, DeliveryStatusNotificationRequest, DeviceTokenRequest, DronePositionRequest, FcmTokenRequest, GuestNotificationRequest, GuestNotificationResponse, NotificationResponse … (9 classes).
06	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: FcmToken, NotificationMessage.
07	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: FcmTokenRepository, NotificationRepository.
08	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: DeliveryNotificationService, DronePositionService, FcmPushNotificationService, GuestNotificationService, NotificationEventListener, NotificationService, WebSocketNotificationService.

1.2.8 IoT Service Package Diagram

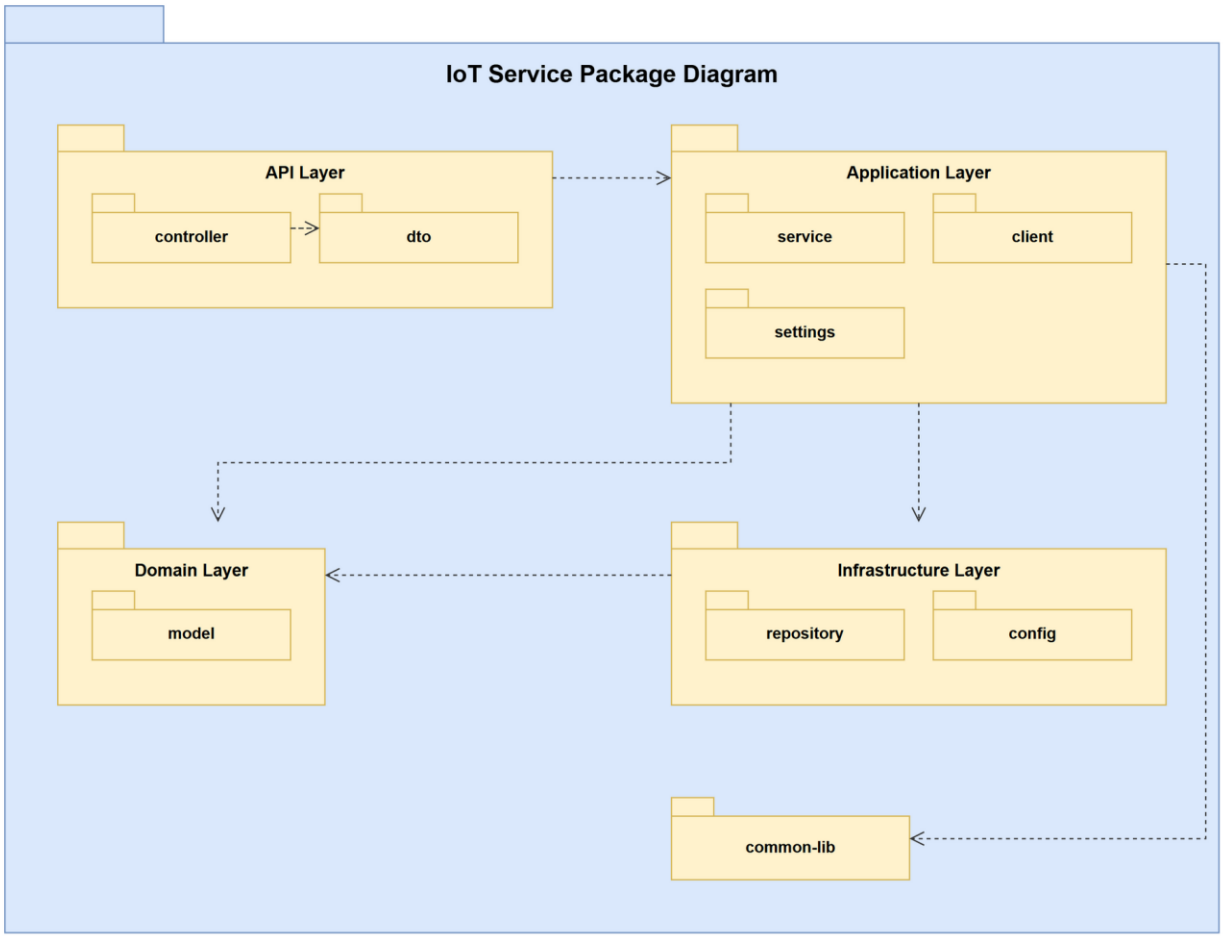


Figure 1.2.8: IoT Service package diagram

Table 1.2.8: Packages of iot-service

No	Package	Description
01	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: LockerClient, OrderClient.
02	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: RabbitConfig.
03	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: BleTestController, CabinetControlController, GatewayController, IotController, TechnicianController.
04	dto	Request and response records exchanged with clients and other services. Main classes: AssignGatewayRequest, BleOpenTokenResponse, BleTestGrantRequest, BleTestGrantResponse, BleTestLockerResponse, BoxAccessLogResponse, BoxHardwareStatusResponse, BoxStateSyncRequest … (25 classes).
05	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: AccessAttempt, BleSigningKey, BleTestGrant, BoxAccessLog, BoxHardwareStatus, CabinetControlLog, CabinetHardwareConfig, DeviceStatus … (9 classes).
06	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: AccessAttemptRepository, BleSigningKeyRepository, BleTestGrantRepository, BoxAccessLogRepository, BoxHardwareStatusRepository, CabinetControlLogRepository, CabinetHardwareConfigRepository, DeviceStatusRepository … (9 classes).
07	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: BleTestService, BleTokenSigner, CabinetControlLogService, CabinetControlService, CabinetHardwareConfigService, CabinetLayoutLookup, DeviceManagementService, GatewayProvisioningService … (10 classes).
08	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: IotRules, IotSettingsCatalog.

1.2.9 Store Service Package Diagram

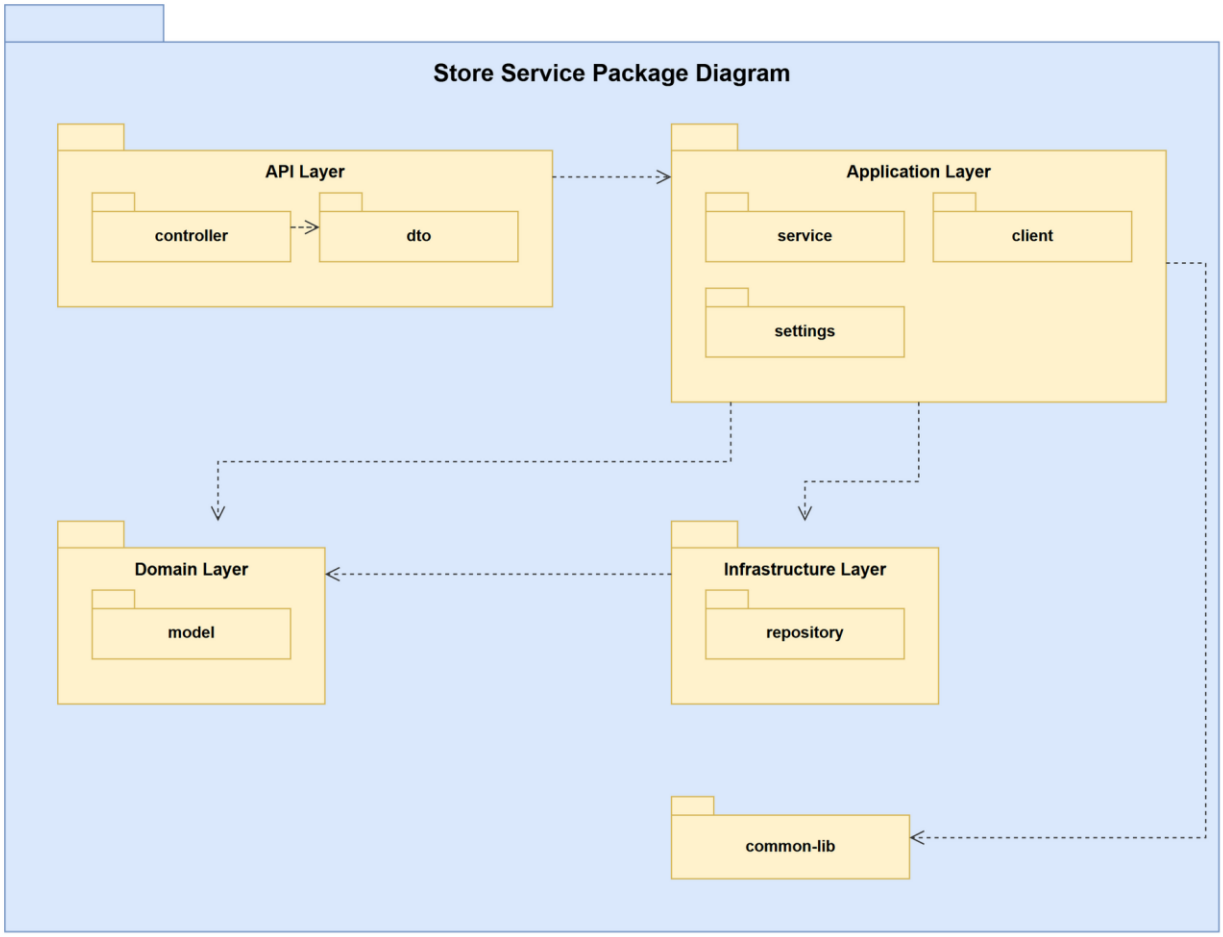


Figure 1.2.9: Store Service package diagram

Table 1.2.9: Packages of store-service

No	Package	Description
01	client	OpenFeign clients that call other services through Eureka on their /internal/** endpoints. Main classes: LockerClient, OrderClient.
02	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: InternalStoreLookupController, StoreController.
03	dto	Request and response records exchanged with clients and other services. Main classes: StoreRequest, StoreResponse.
04	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: StoreLocation.
05	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: StoreRepository.

No	Package	Description
06	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: StoreService.
07	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: StoreRules, StoreSettingsCatalog.

1.2.10 Loyalty Service Package Diagram

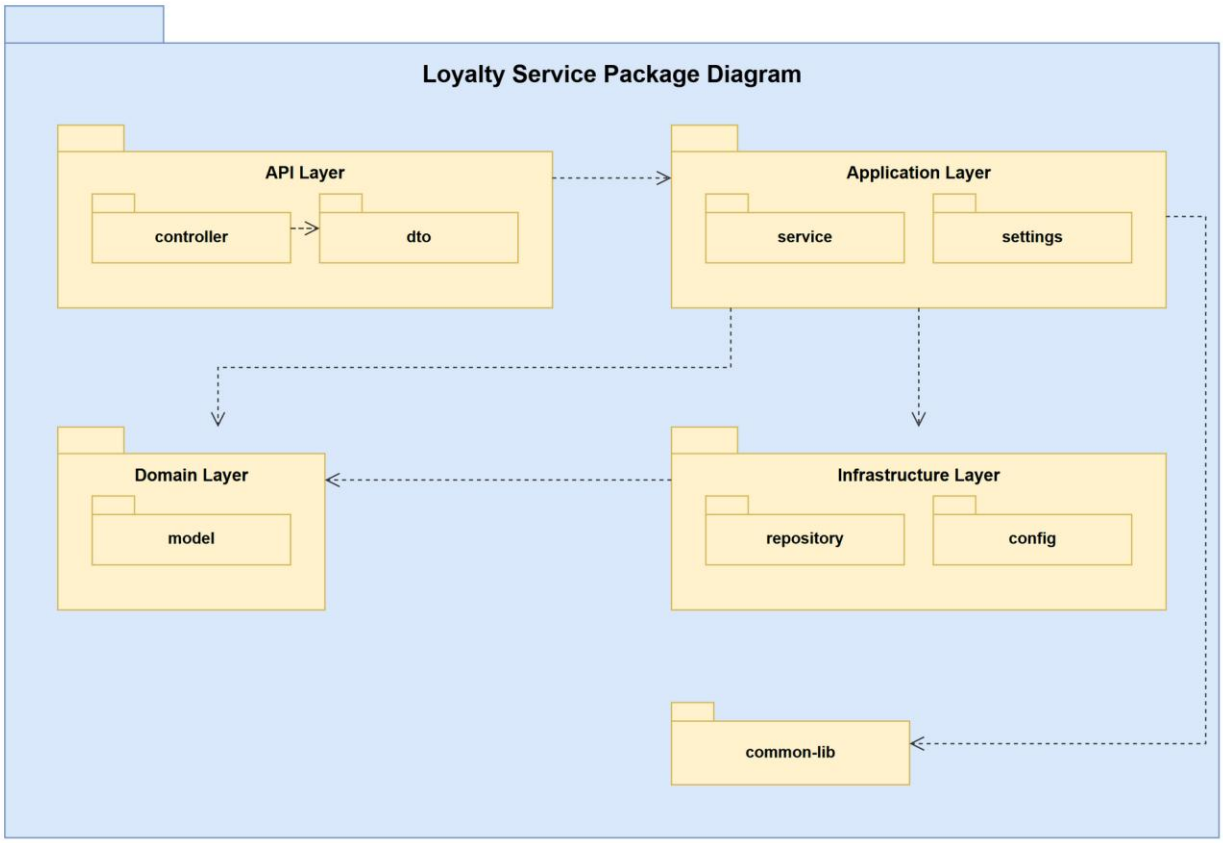


Figure 1.2.10: Loyalty Service package diagram

Table 1.2.10: Packages of loyalty-service

No	Package	Description
01	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: RabbitConfig.
02	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: LoyaltyController.

No	Package	Description
03	dto	Request and response records exchanged with clients and other services. Main classes: AdjustPointsRequest, LoyaltyAccountResponse, LoyaltyStatisticsResponse, OrderRedemptionRefundResponse, OrderRedemptionRequest, OrderRedemptionResponse, PointTransactionResponse, RedeemPointsRequest … (12 classes).
04	model	JPA entities mapped to the tables of the service database, and entity listeners that publish change events. Main classes: LoyaltyAccount, PointTransaction.
05	repository	Spring Data JPA repositories: derived queries, custom JPQL, pessimistic locks. Main classes: LoyaltyAccountRepository, PointTransactionRepository.
06	service	Business logic (@Service / @Component): use cases, business-rule checks, transactions, scheduled jobs and publishing of domain events. Main classes: LoyaltyEvents, LoyaltyService, OrderCompletedPointsListener.
07	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: LoyaltyRules, LoyaltySettingsCatalog.

1.2.11 Assistant Service Package Diagram

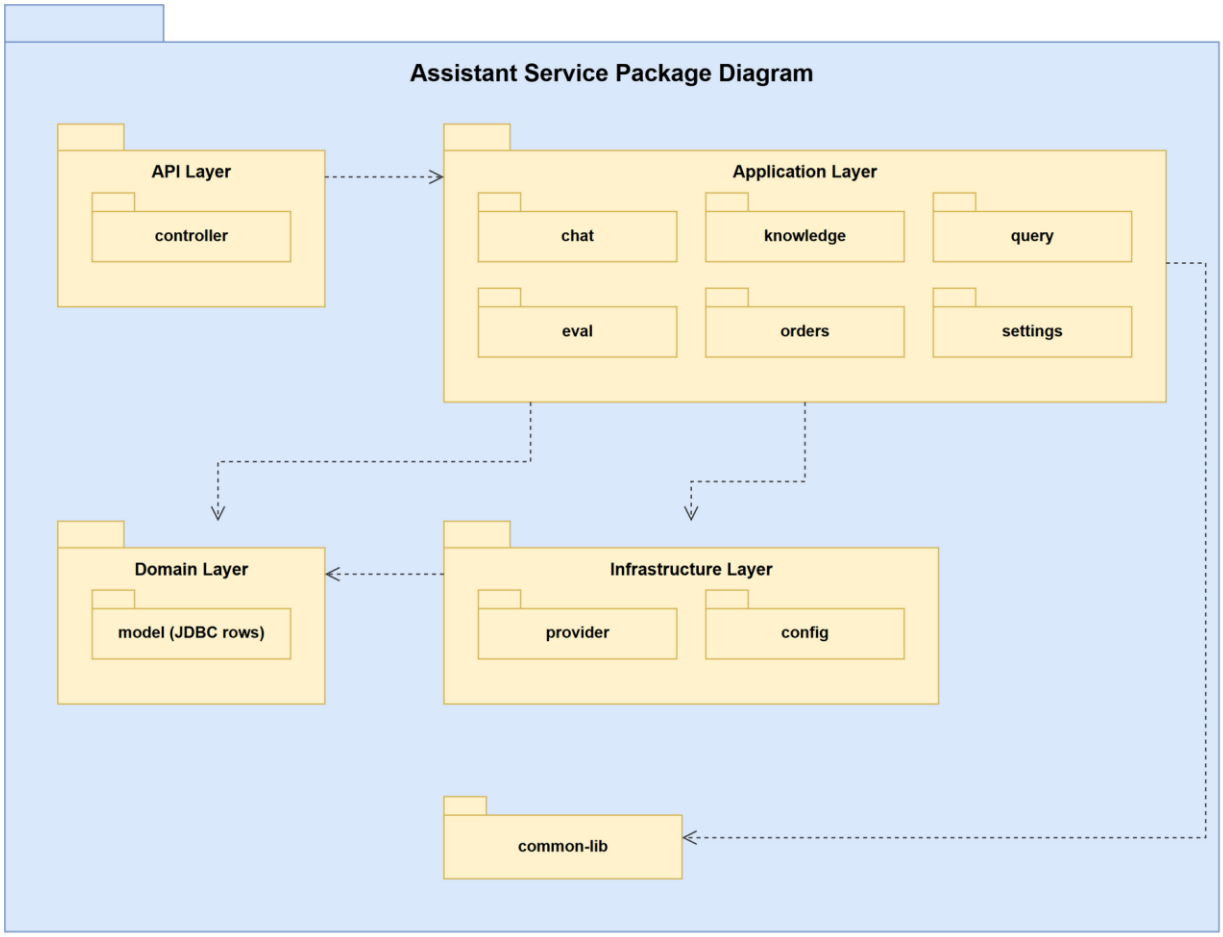


Figure 1.2.11: Assistant Service package diagram

Table 1.2.11: Packages of assistant-service

No	Package	Description
01	chat	Question answering: AssistantService and the conversation repository (JDBC). Main classes: AssistantService, ChatDtos, ConversationRepository.
02	config	Spring configuration: RabbitMQ exchange / queues / bindings, security, Firebase, WebSocket. Main classes: AssistantProperties.
03	controller	REST controllers (@RestController). Receive requests routed by the gateway, read the X-User-Id / X-User-Roles headers, validate the input and wrap results in ApiResponse. Main classes: AssistantController, AssistantExceptionHandler, KnowledgeAdminController.
04	eval	Evaluation cases used to measure retrieval quality. Main classes: EvalService.
05	knowledge	Knowledge base: documents, chunks, document parser, text chunker and background indexer. Main classes: ChunkRepository, DocumentParser, DocumentRepository, KnowledgeChunk, KnowledgeDocument, KnowledgeIndexer, KnowledgeService, RetrievedChunk … (10 classes).

No	Package	Description
06	orders	Feign clients and formatter that load the asker's own orders as answer context. Main classes: LockerLookupClient, OrderContextService, OrderDocumentFormatter, OrderDtos, OrderServiceClient.
07	provider	Adapters for external AI providers: Claude answer generator, Voyage embeddings, diacritics restorer. Main classes: AnswerGenerator, ClaudeAnswerGenerator, ClaudeClientProvider, ClaudeDiacriticsRestorer, DiacriticsRestorer, EmbeddingProvider, ProviderException, VoyageEmbeddingProvider.
08	query	Question preparation: Vietnamese normalisation and order intent detection. Main classes: OrderIntent, QuestionPreparer, VietnameseText.
09	settings	Rules and SettingsCatalog of the service: business settings editable on the admin web (common-lib BusinessSettings). Main classes: AssistantRules, AssistantSettingsCatalog.

1.2.12 Admin Web Application Package Diagram

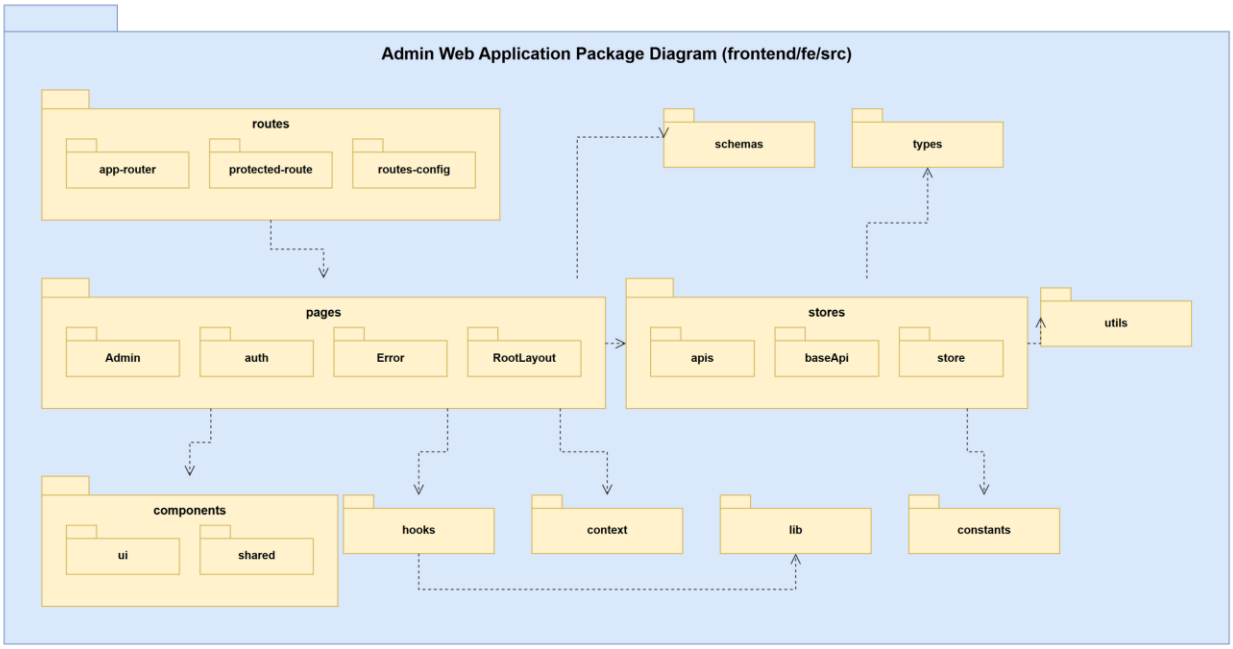


Figure 1.2.12: Admin web application package diagram

Table 1.2.12: Packages of the admin web application

No	Package	Description
01	routes	app-router, protected-route (role check) and routes-config of every admin page (lazy loaded).
02	pages	Screens: auth (2FA login) and Admin/* feature folders - dashboard, revenue, users, technicians, stores, lockers, cabinet-control, maintenance, drones, drone-orders, drone-incidents, drone-logs, orders, payments, refunds, promotions, loyalty, feedback, notifications, scheduler, services, settings, knowledge.

No	Package	Description
03	components	ui (shadcn-style primitives: button, dialog, table, form...) and shared (data-table, layout, page-header, media, status tabs, charts).
04	stores	Redux store and RTK Query: baseApi (auth header, refresh, error handling) and apis (adminApi, authApi, loyaltyApi, notificationApi, media).
05	hooks	useAdminRealtimeSync (STOMP -> cache invalidation), useWebSocket, useImageUpload, use-toast.
06	context	AuthContext (session and role), sidebar and theme contexts.
07	lib	Helpers: admin realtime tag map, Cloudinary upload, date/time, locker layout helper, report formatting, validation.
08	schemas	Zod schemas for admin forms.
09	types	TypeScript types of API resources.
10	constants	API paths, roles, sidebar menu, order setting keys.
11	utils	API helpers, auth session storage, i18n.

1.2.13 Mobile Application Package Diagram

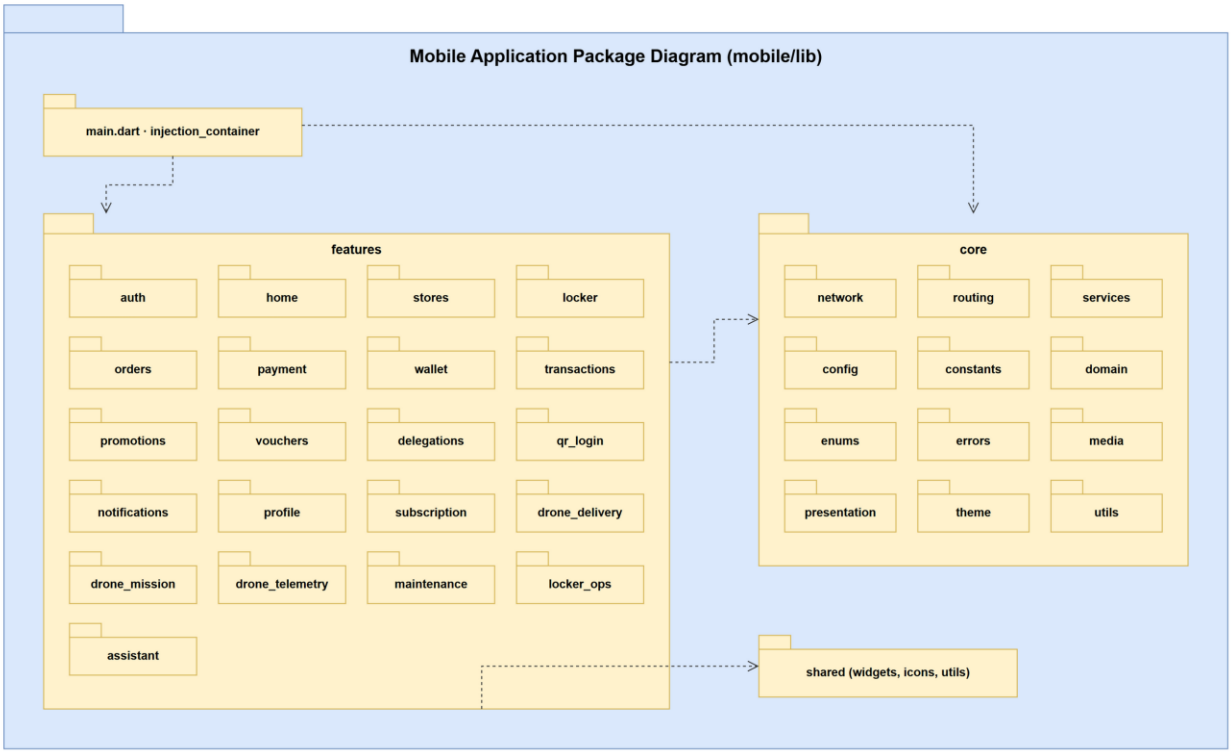


Figure 1.2.13: Mobile application package diagram

Table 1.2.13: Packages of the mobile application

No	Package	Description
01	main.dart / injection_container	Application entry, Firebase initialisation and provider factories that wire data sources, repositories and use cases (per-feature *_injection providers).

No	Package	Description
02	features	One folder per business feature: auth, home, stores, locker, orders, payment, wallet, transactions, promotions, vouchers, delegations, qr_login, notifications, profile, subscription, drone_delivery, drone_mission, drone_telemetry, maintenance (locker technician), locker_ops, assistant.
03	core/network	dio ApiClient, AuthInterceptor (token refresh), gateway client.
04	core/routing	go_router configuration, role-based routes, main navigation wrapper, notification deep links.
05	core/services	Session and token storage, Firebase auth / messaging, BLE open service, biometric, app event bus.
06	core/config, constants, theme	Environment config, business config loaded from /api/settings/*/public, colours, strings, themes.
07	core/domain, enums, errors, utils, media, presentation	Shared entities, enums, failures, formatters, media upload and common pages.
08	shared	Reusable widgets, icons and utilities.

1.2.14 Mobile Feature Module Package Diagram

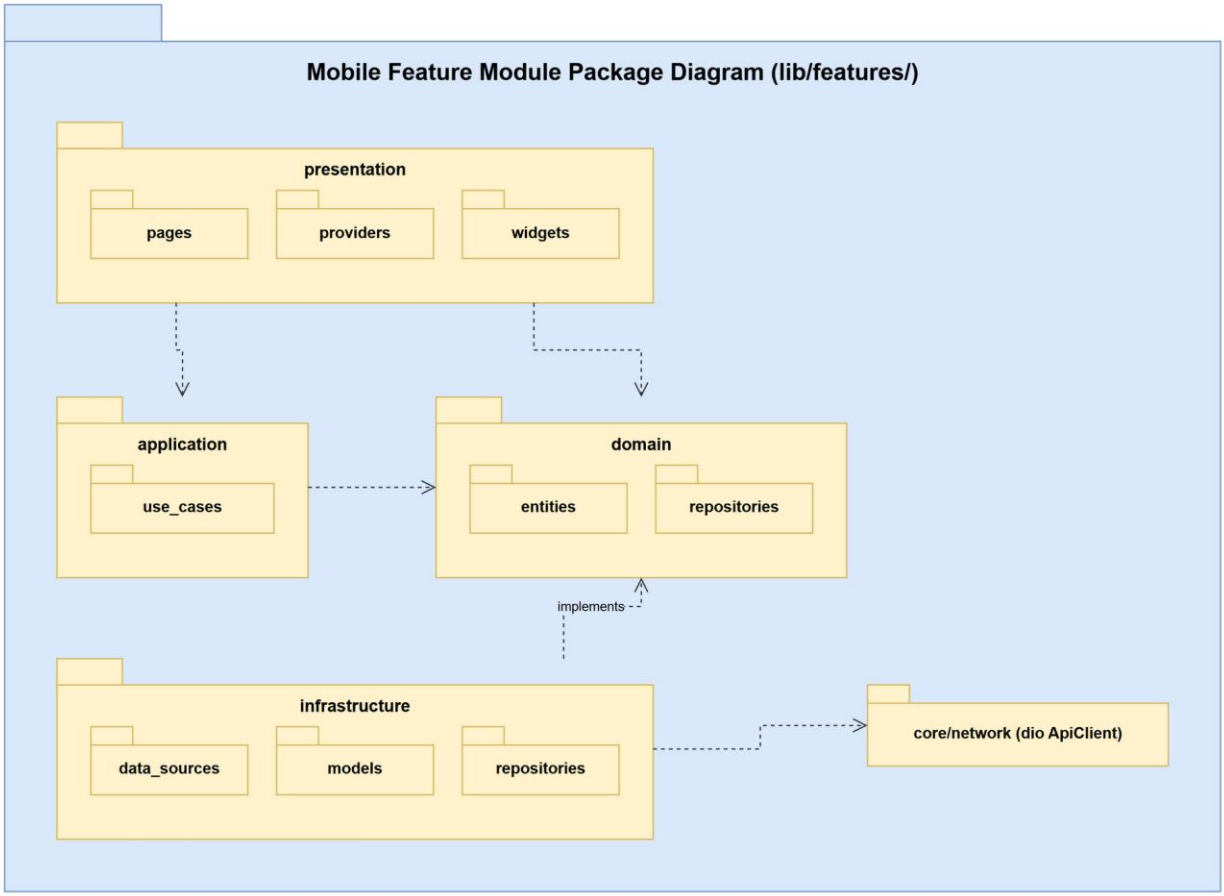


Figure 1.2.14: Package structure inside one mobile feature module

Table 1.2.14: Layers of a mobile feature module

No	Package	Description
01	presentation	pages (screens), widgets and providers (Riverpod / provider / bloc state).
02	application	use_cases: one class per user action (e.g. LoginUseCase, GetMyOrdersUseCase).
03	domain	entities and repository interfaces of the feature, independent of the API.
04	infrastructure	data_sources (remote calls through ApiClient), models (JSON serializable, *.g.dart) and repository implementations.

1.2.15 IoT Locker Controller Package Diagram

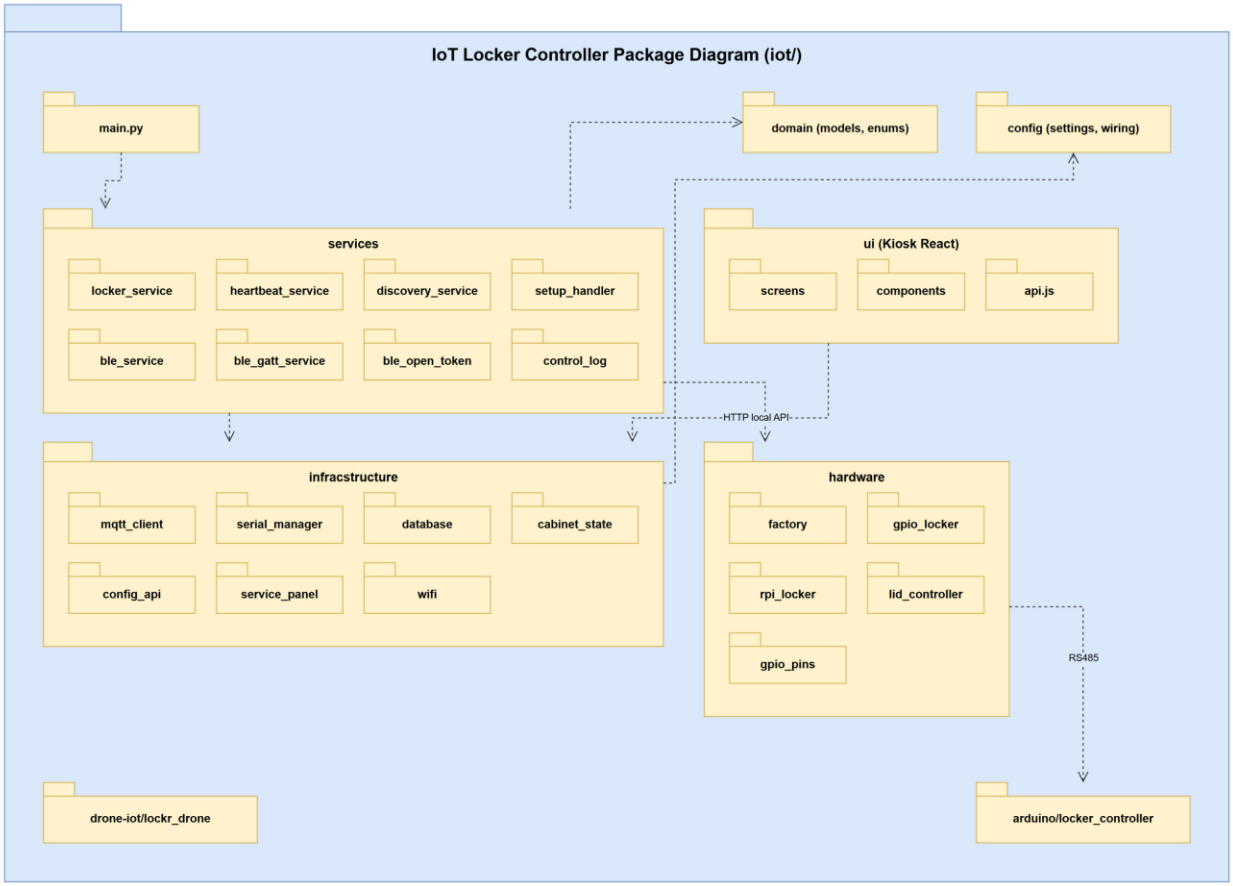


Figure 1.2.15: IoT locker controller package diagram

Table 1.2.15: Packages of the IoT locker controller

No	Package	Description
01	main.py	Starts the controller: loads config, connects MQTT, starts heartbeat, discovery / setup handlers, BLE service and local API.
02	services	locker_service (open / lid / sync commands), heartbeat_service, discovery_service and setup_handler (provisioning), ble_service / ble_gatt_service / ble_open_token (offline open), control_log.

No	Package	Description
03	infracstructure	mqtt_client (paho-mqtt, topics cabinet/* and iot/*), serial_manager (RS485 to Arduino), database (local PostgreSQL log store, psycopg2), cabinet_state, config_api and service_panel (local FastAPI HTTP API), wifi.
04	hardware	factory selecting the driver, gpio_locker / rpi_locker (relays and reed sensors through gpiod), lid_controller (sliding lid), gpio_pins.
05	domain	Models and enums of boxes, commands and results.
06	config	settings (environment), wiring_default.json and hardware_config.
07	ui (Kiosk React)	KioskScreen and AdminScreen, components (virtual keyboard, animations) and api.js calling the backend and the Pi.
08	arduino/locker_controller	Arduino sketch driving relays and reading sensors over RS485.
09	drone-iot/lockr_drone	Companion computer on the drone: reads MAVLink and publishes signed telemetry to MQTT.

2. Database Design

Lock.R follows the database-per-service pattern: each microservice owns one PostgreSQL 16 database whose schema is created by Flyway migrations and validated by Hibernate at start-up; the assistant uses a separate pgvector database. Foreign keys exist only between tables of the same database. A column that refers to a row of another service (for example orders.user_id -> user_profiles) is a logical reference: it is checked by the owning service through Feign calls and is shown as (FK) in the diagrams. The figures list the primary key, the foreign keys and the first columns of each table; very wide tables (orders, drone_parcel_incidents, drone_missions) show the number of remaining columns. Crow's-foot notation: | exactly one, o| zero or one, o< zero or many.

2.1 auth_db (auth-service)

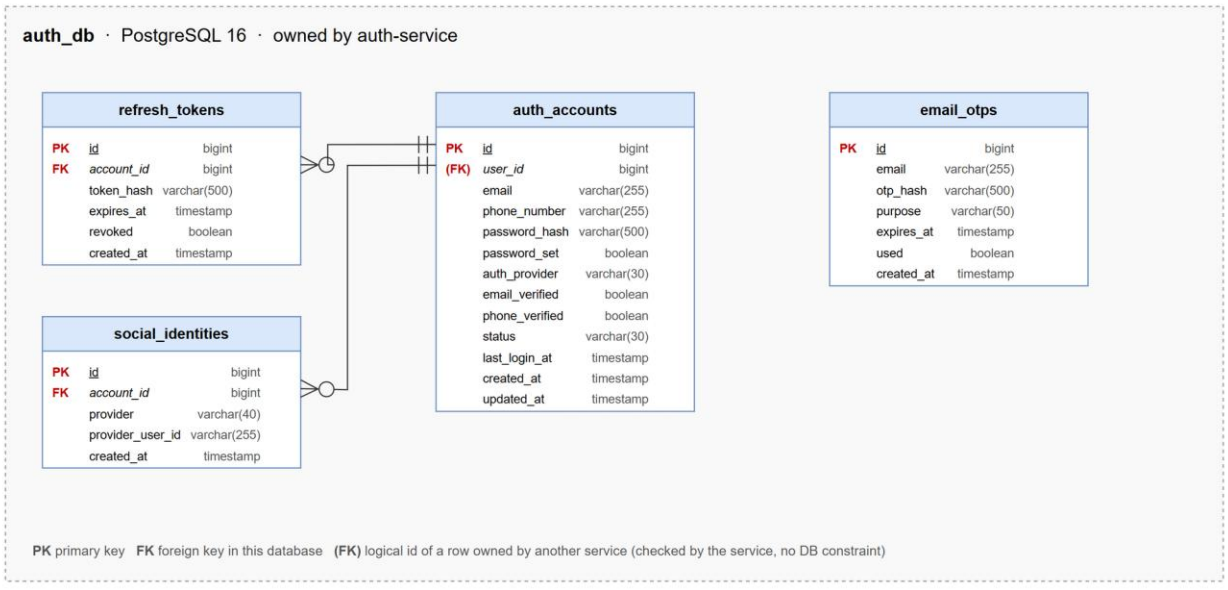


Figure 2.1: Database diagram of auth_db

Table 2.1: Table descriptions of auth_db

No	Table	Description
01	auth_accounts	Login credentials of a user: email, phone, password hash, provider (LOCAL / GOOGLE / PHONE), status and 2FA flag. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
02	email_otps	One-time passwords for login, password reset and admin 2FA, stored hashed with expiry and attempts. - Primary keys: id - Foreign keys: none
03	refresh_tokens	Refresh tokens issued at login, stored hashed and revoked on logout or when the account is locked. - Primary keys: id - Foreign keys: account_id -> auth_accounts
04	social_identities	Google / Firebase identities linked to an account. - Primary keys: id - Foreign keys: account_id -> auth_accounts

2.2 user_db (user-service)

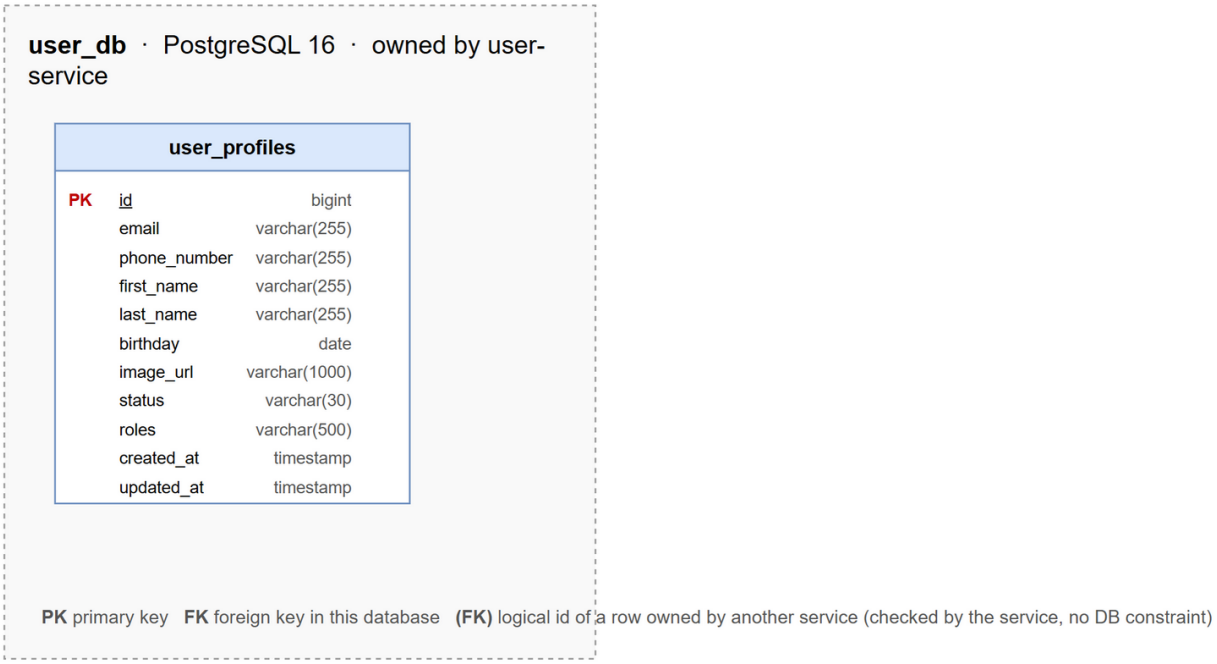


Figure 2.2: Database diagram of user_db

Table 2.2: Table descriptions of user_db

No	Table	Description
01	user_profiles	Profile of a user: name, email, phone, avatar, roles (CUSTOMER, LOCKER_TECHNICIAN, DRONE_TECHNICIAN, ADMIN) and status. - Primary keys: id - Foreign keys: none

2.3 store_db (store-service)

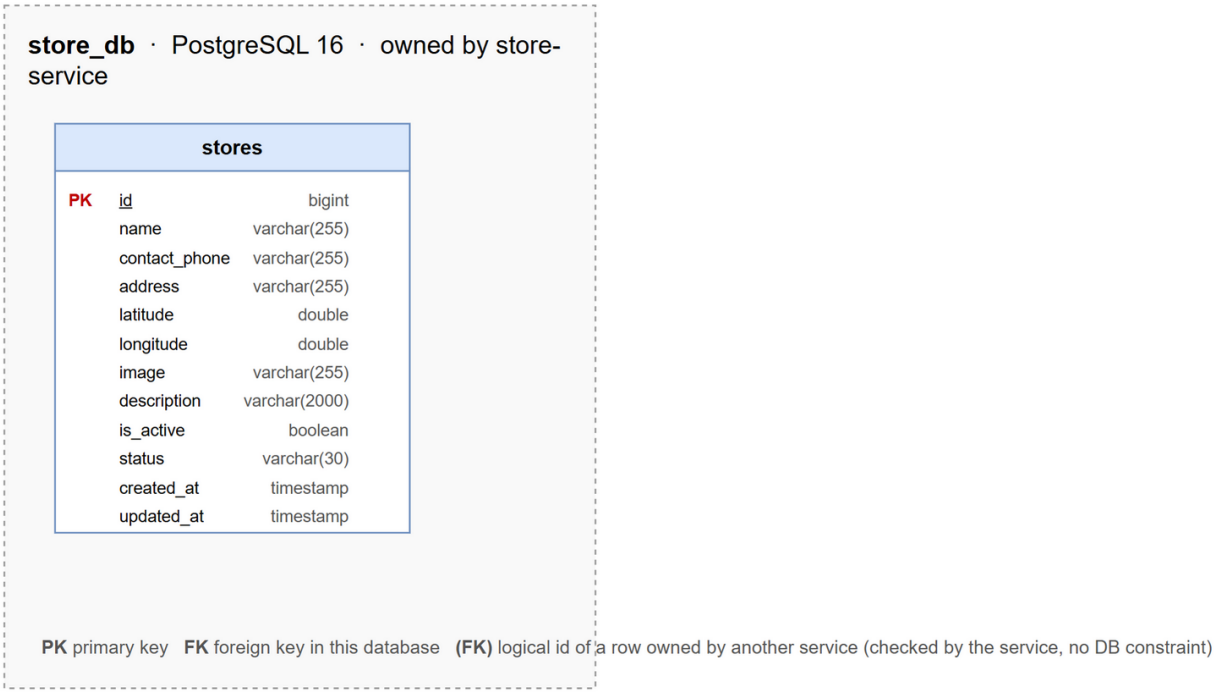


Figure 2.3: Database diagram of store_db

Table 2.3: Table descriptions of store_db

No	Table	Description
01	stores	Store (location) that hosts lockers: name, address, coordinates, status. - Primary keys: id - Foreign keys: none

2.4 locker_db (locker-service)

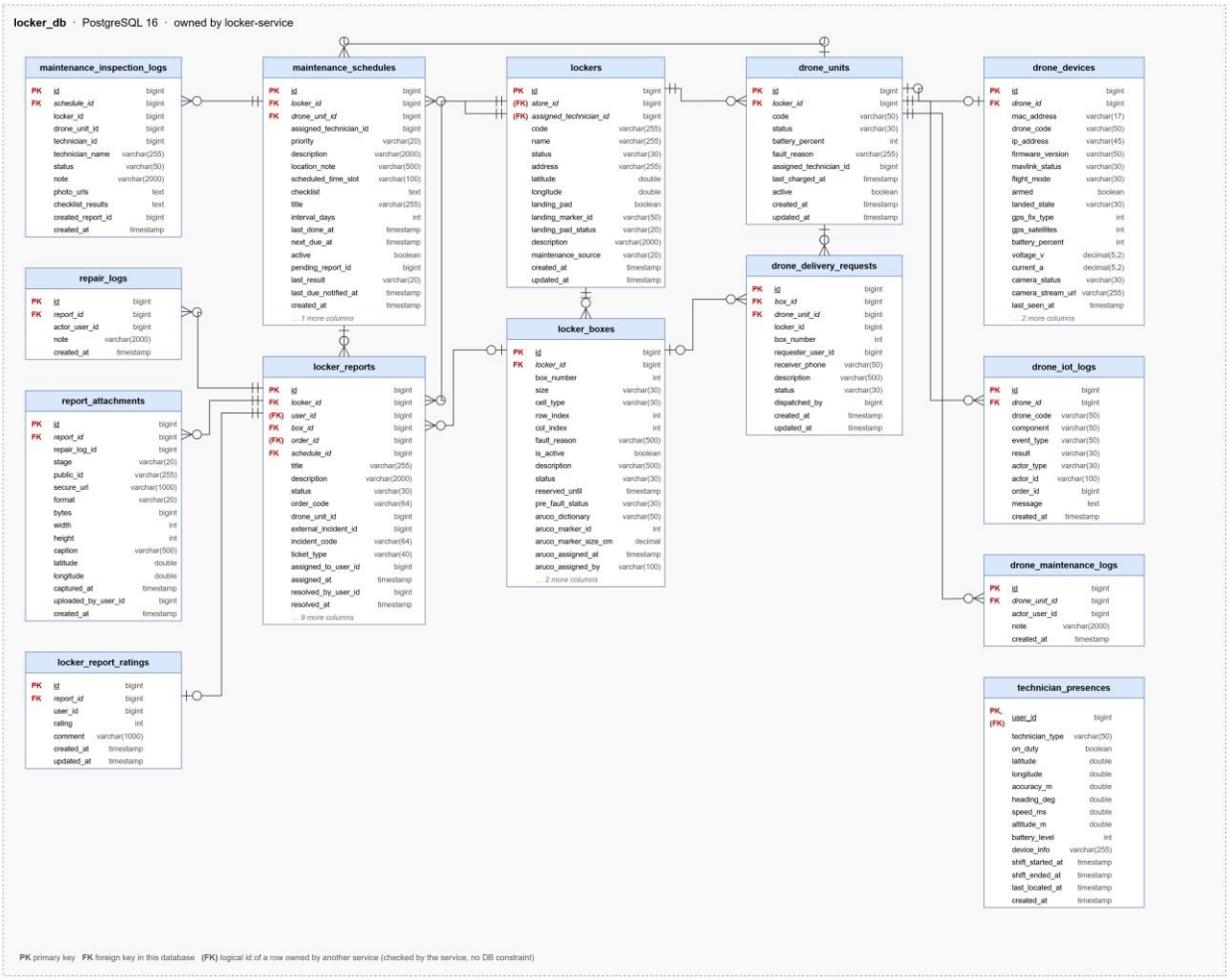


Figure 2.4: Database diagram of locker_db

Table 2.4: Table descriptions of locker_db

No	Table	Description
01	drone_delivery_requests	Request to deliver to a drone box of a locker. - Primary keys: id - Foreign keys: box_id -> locker_boxes, drone_unit_id -> drone_units
02	drone_devices	Hardware device (Pi + flight controller) installed on a drone. - Primary keys: id - Foreign keys: drone_id -> drone_units
03	drone_iot_logs	IoT events reported by a drone device. - Primary keys: id - Foreign keys: drone_id -> drone_units
04	drone_maintenance_logs	Maintenance note of a drone. - Primary keys: id - Foreign keys: drone_unit_id -> drone_units

No	Table	Description
05	drone_units	Drone of the fleet: code, station locker, status, battery and responsible technician. - Primary keys: id - Foreign keys: locker_id -> lockers
06	locker_boxes	Box of a locker: number, size, cell type (STANDARD / XL / DRONE), grid position and status. - Primary keys: id - Foreign keys: locker_id -> lockers
07	locker_report_ratings	Reporter's rating of the technician after a ticket is resolved. - Primary keys: id - Foreign keys: report_id -> locker_reports
08	locker_reports	Ticket for a box, locker or drone: category, status (OPEN / IN_PROGRESS / RESOLVED / REJECTED), assignee, SLA and resolution. - Primary keys: id - Foreign keys: locker_id -> lockers, box_id -> locker_boxes, user_id -> user_profiles (logical, user-service), order_id -> orders (logical, order-service), schedule_id -> maintenance_schedules
09	lockers	Locker cabinet: code, store, address, GPS, status (ACTIVE / MAINTENANCE / OFFLINE), landing pad and responsible technician. - Primary keys: id - Foreign keys: store_id -> stores (logical, store-service), assigned_technician_id -> user_profiles (logical, user-service)
10	maintenance_inspection_logs	Result of one inspection with Pass / Fail per checklist item and photos. - Primary keys: id - Foreign keys: schedule_id -> maintenance_schedules
11	maintenance_schedules	Periodic inspection plan of a locker or a drone with checklist, interval and assigned technician. - Primary keys: id - Foreign keys: locker_id -> lockers, drone_unit_id -> drone_units
12	repair_logs	Progress note of a ticket written by the technician. - Primary keys: id - Foreign keys: report_id -> locker_reports
13	report_attachments	Photo of a ticket (before / during / after repair) with GPS and capture time. - Primary keys: id - Foreign keys: report_id -> locker_reports
14	technician_presences	Shift state, last GPS location and penalty level of a locker technician. - Primary keys: user_id - Foreign keys: user_id -> user_profiles (logical, user-service)

2.5 order_db (order-service)

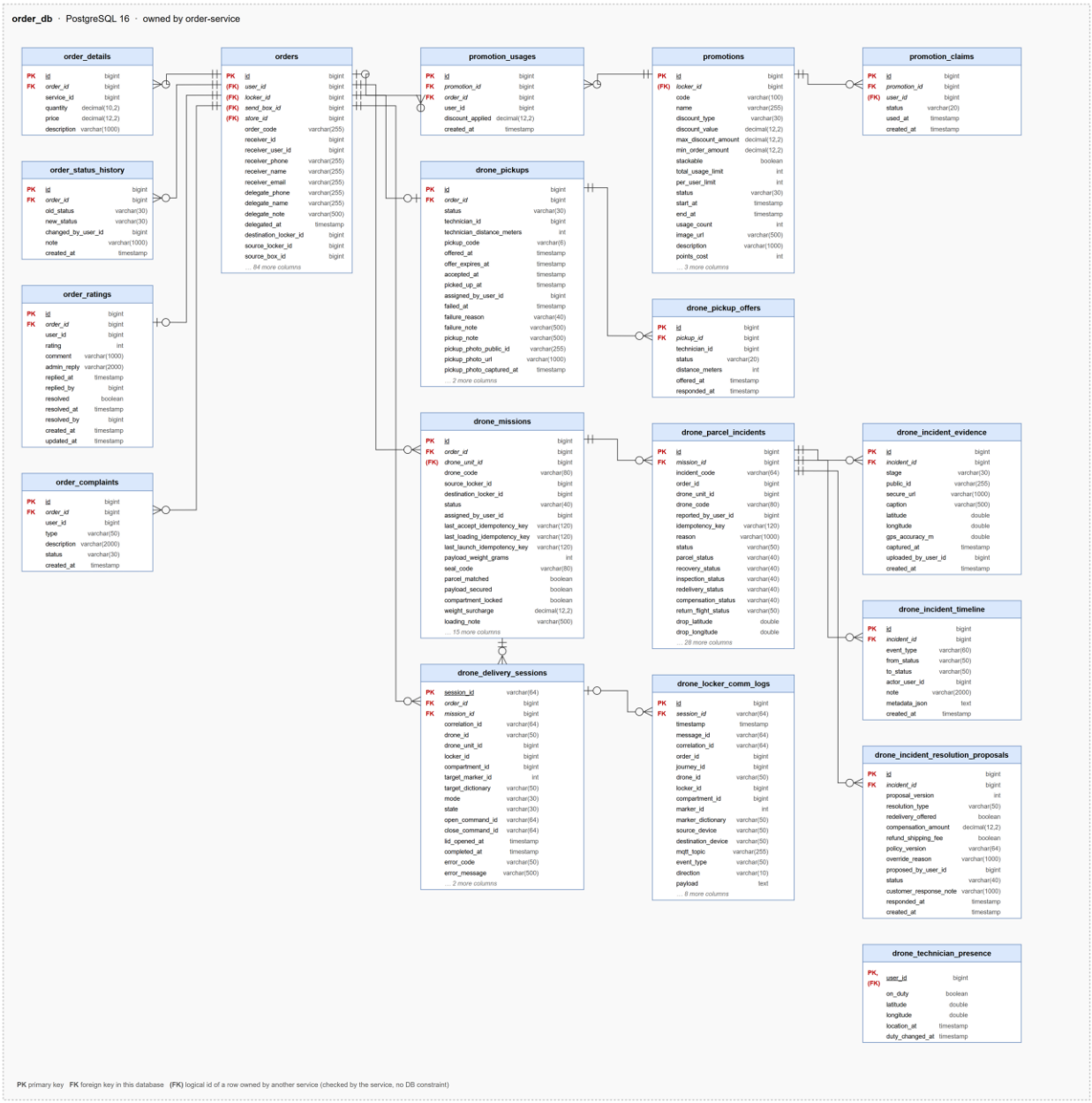


Figure 2.5: Database diagram of order_db

Table 2.5: Table descriptions of order_db

No	Table	Description
01	drone_delivery_sessions	Delivery session between a drone and the receiver locker (landing, compartment, lid). - Primary keys: session_id - Foreign keys: order_id -> orders, mission_id -> drone_missions

No	Table	Description
02	drone_incident_evidence	Photo or telemetry evidence of a parcel drop incident. - Primary keys: id - Foreign keys: incident_id -> drone_parcel_incidents
03	drone_incident_resolution_proposals	Redelivery or compensation proposal of an incident and its approval. - Primary keys: id - Foreign keys: incident_id -> drone_parcel_incidents
04	drone_incident_timeline	Event timeline of a parcel drop incident. - Primary keys: id - Foreign keys: incident_id -> drone_parcel_incidents
05	drone_locker_comm_logs	MQTT messages exchanged between a drone and a locker during a delivery. - Primary keys: id - Foreign keys: session_id -> drone_delivery_sessions
06	drone_missions	One flight of a drone order: drone, route, loading data (weight, seal code, checklist), launch time, stages and end reason. - Primary keys: id - Foreign keys: order_id -> orders, drone_unit_id -> drone_units (logical, locker-service)
07	drone_parcel_incidents	Parcel drop incident: location, evidence, recovery assignment, resolution and compensation. - Primary keys: id - Foreign keys: mission_id -> drone_missions
08	drone_pickup_offers	Pickup offer sent to a drone technician and its answer. - Primary keys: id - Foreign keys: pickup_id -> drone_pickups
09	drone_pickups	Home pickup of a drone order: location, technician, photo and deadlines. - Primary keys: id - Foreign keys: order_id -> orders
10	drone_technician_presence	Shift state and last GPS location of a drone technician. - Primary keys: user_id - Foreign keys: user_id -> user_profiles (logical, user-service)
11	order_complaints	Complaint about an order with type, content and handling status. - Primary keys: id - Foreign keys: order_id -> orders
12	order_details	Item line of an order (parcel description, size, weight). - Primary keys: id - Foreign keys: order_id -> orders
13	order_ratings	Customer rating of a completed order with the admin reply. - Primary keys: id - Foreign keys: order_id -> orders

No	Table	Description
14	order_status_history	Every status change of an order with actor and note. - Primary keys: id - Foreign keys: order_id -> orders
15	orders	Order of type SEND, RENTAL, STORAGE or DRONE_DELIVERY: code, customer, receiver, lockers and boxes, prices, PIN, deadlines, payment and status. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service), locker_id -> lockers (logical, locker-service), send_box_id -> locker_boxes (logical, locker-service), store_id -> stores (logical, store-service)
16	promotion_claims	Voucher saved or exchanged by a user. - Primary keys: id - Foreign keys: promotion_id -> promotions, user_id -> user_profiles (logical, user-service)
17	promotion_usages	Use of a promotion in an order. - Primary keys: id - Foreign keys: promotion_id -> promotions, order_id -> orders
18	promotions	Promotion definition: code, discount, period, limits, locker scope and point price. - Primary keys: id - Foreign keys: locker_id -> lockers (logical, locker-service)

2.6 payment_db (payment-service)

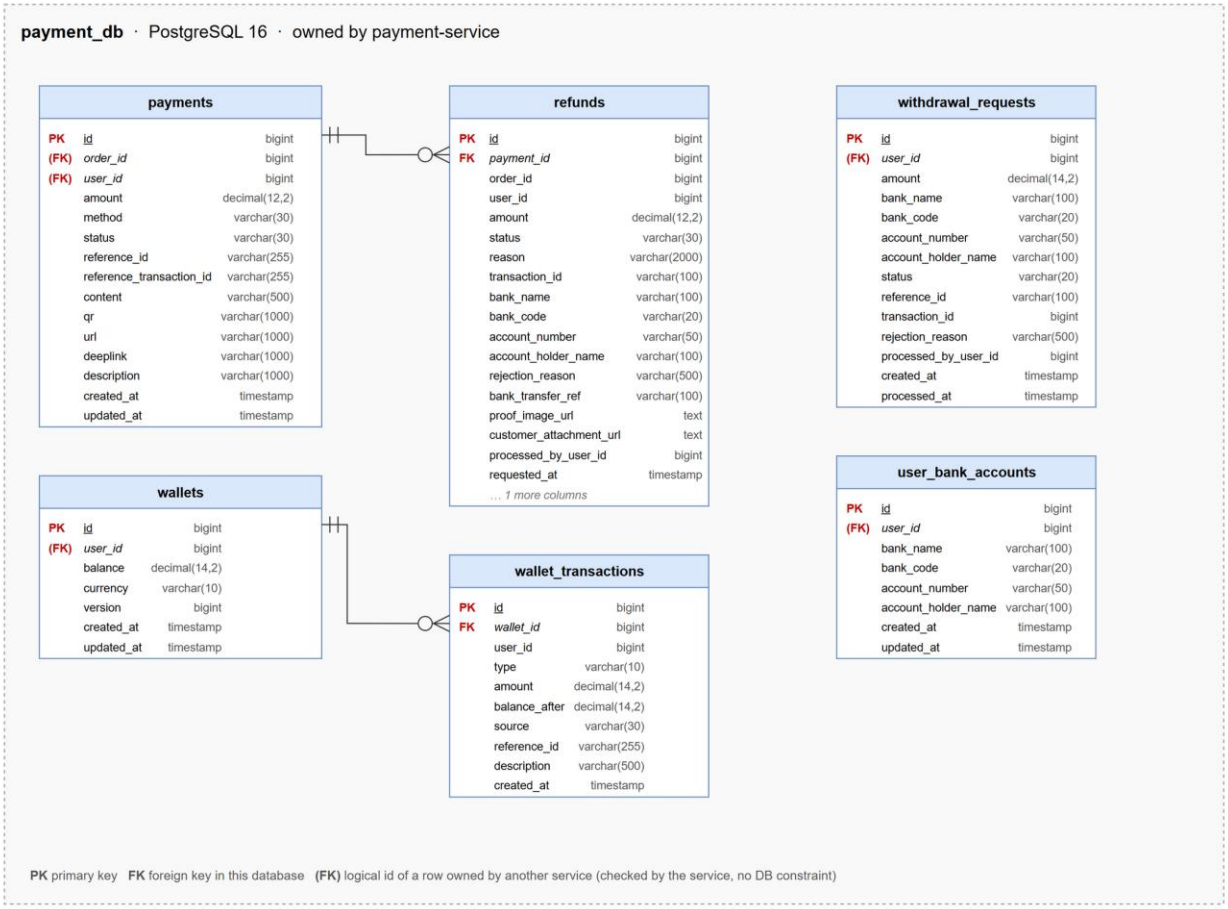


Figure 2.6: Database diagram of payment_db

Table 2.6: Table descriptions of payment_db

No	Table	Description
01	payments	Payment attempt of an order: method, amount, status, VietQR content and transaction id. - Primary keys: id - Foreign keys: order_id -> orders (logical, order-service), user_id -> user_profiles (logical, user-service)
02	refunds	Refund request of a payment with bank account, evidence, transfer receipt and status. - Primary keys: id - Foreign keys: payment_id -> payments
03	user_bank_accounts	Bank account saved by a user for refunds and withdrawals. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
04	wallet_transactions	Ledger entry of a wallet with type, amount and balance after. - Primary keys: id - Foreign keys: wallet_id -> wallets

No	Table	Description
05	wallets	Lock.R wallet balance of a user. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
06	withdrawal_requests	Withdrawal request from the wallet to a bank account and its approval status. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)

2.7 notification_db (notification-service)

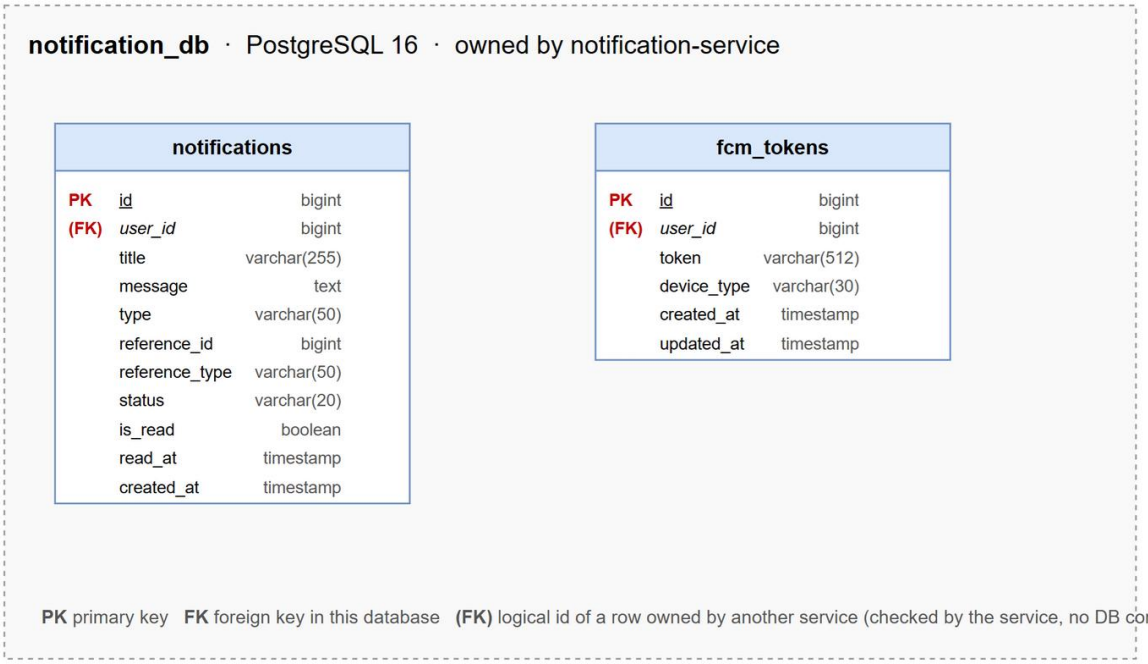


Figure 2.7: Database diagram of notification_db

Table 2.7: Table descriptions of notification_db

No	Table	Description
01	fcm_tokens	Firebase push token of a user's device. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
02	notifications	In-app notification of a user with type, content, related object and read flag. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)

2.8 iot_db (iot-service)

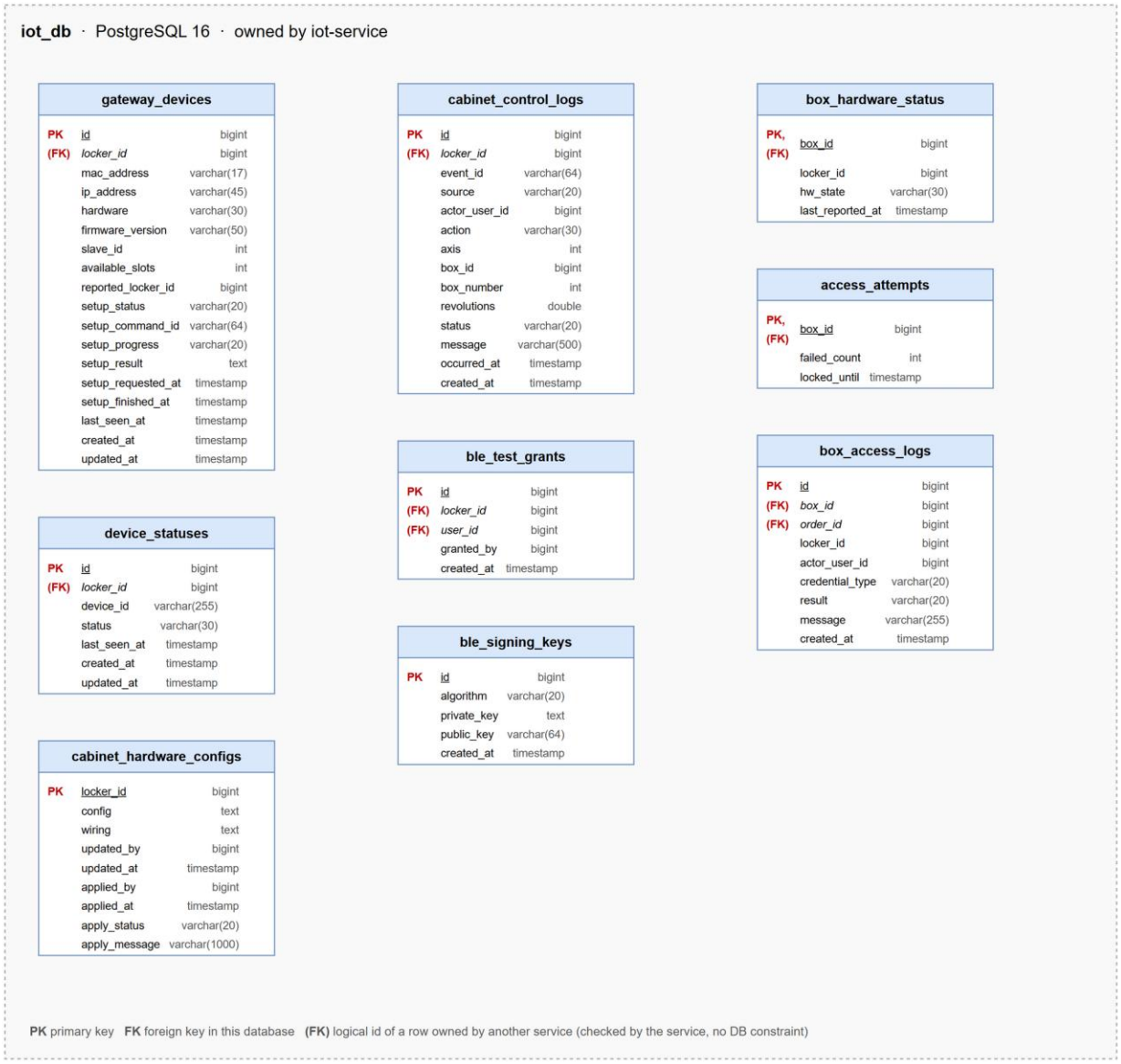


Figure 2.8: Database diagram of iot_db

Table 2.8: Table descriptions of iot_db

No	Table	Description
01	access_attempts	Wrong-PIN counter and lock-out time of a box. - Primary keys: box_id - Foreign keys: box_id -> locker_boxes (logical, locker-service)
02	ble_signing_keys	Ed25519 key pair used to sign Bluetooth open tokens. - Primary keys: id - Foreign keys: none

No	Table	Description
03	ble_test_grants	Permission of a customer account to test opening boxes of a locker by Bluetooth. - Primary keys: id - Foreign keys: locker_id -> lockers (logical, locker-service), user_id -> user_profiles (logical, user-service)
04	box_access_logs	Every open attempt of a box with credential type, order and result. - Primary keys: id - Foreign keys: box_id -> locker_boxes (logical, locker-service), order_id -> orders (logical, order-service)
05	box_hardware_status	Current door state (open / closed) of each box reported by the controller. - Primary keys: box_id - Foreign keys: box_id -> locker_boxes (logical, locker-service)
06	cabinet_control_logs	Control actions of a cabinet (box open, lid move, Bluetooth open) from the web or done by the Pi, with result. - Primary keys: id - Foreign keys: locker_id -> lockers (logical, locker-service)
07	cabinet_hardware_configs	Hardware configuration of a cabinet (GPIO / RS485 wiring, lid tuning) edited by technicians and applied to the Pi, with apply status. - Primary keys: locker_id - Foreign keys: none
08	device_statuses	Online / offline events of a locker controller. - Primary keys: id - Foreign keys: locker_id -> lockers (logical, locker-service)
09	gateway_devices	Raspberry Pi controller of a locker: MAC, firmware, assigned locker and last seen time. - Primary keys: id - Foreign keys: locker_id -> lockers (logical, locker-service)

2.9 loyalty_db (loyalty-service)

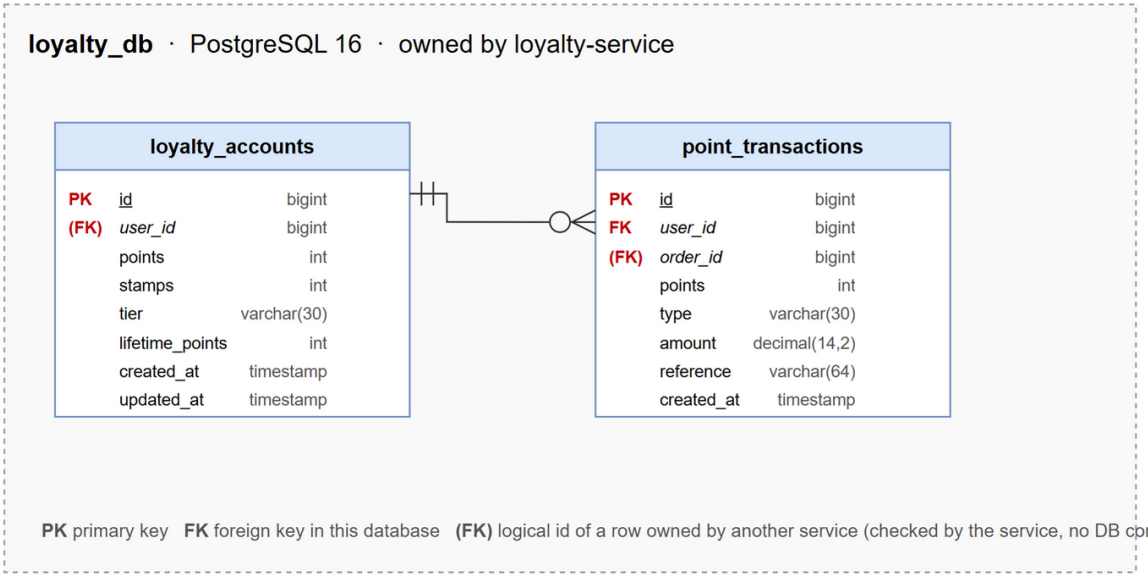


Figure 2.9: Database diagram of loyalty_db

Table 2.9: Table descriptions of loyalty_db

No	Table	Description
01	loyalty_accounts	Points, lifetime points and tier of a user. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
02	point_transactions	Every earn / redeem / reversal of loyalty points. - Primary keys: id - Foreign keys: user_id -> loyalty_accounts, order_id -> orders (logical, order-service)

2.10 assistant_db (assistant-service)

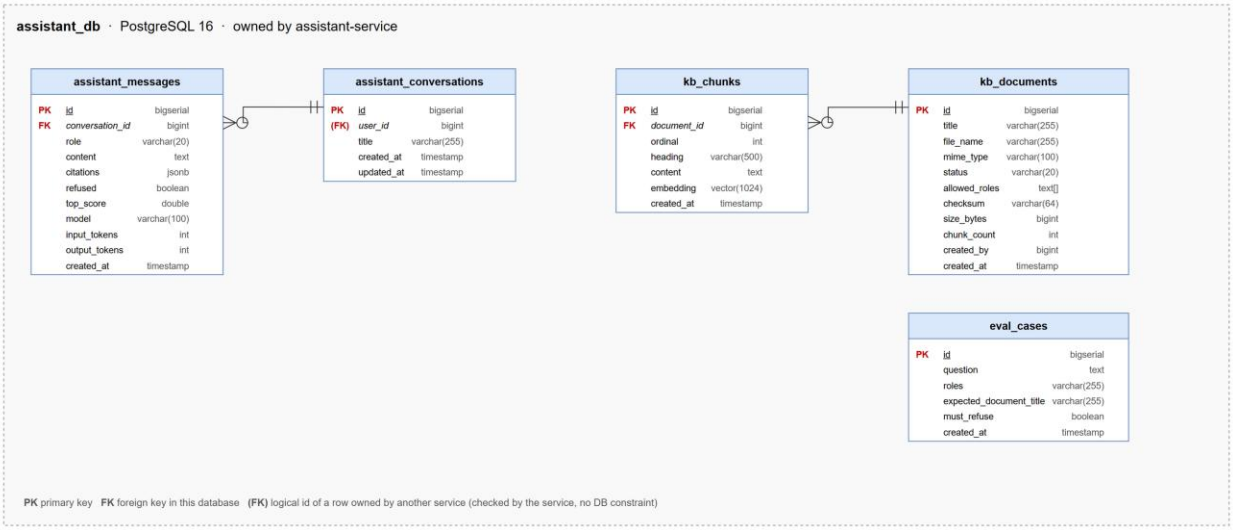


Figure 2.10: Database diagram of assistant_db

Table 2.10: Table descriptions of assistant_db

No	Table	Description
01	assistant_conversations	A conversation of a user with the AI assistant. - Primary keys: id - Foreign keys: user_id -> user_profiles (logical, user-service)
02	assistant_messages	Question or answer in a conversation, with cited sources and rating. - Primary keys: id - Foreign keys: conversation_id -> assistant_conversations
03	eval_cases	Evaluation question used to measure retrieval quality of the AI assistant. - Primary keys: id - Foreign keys: none
04	kb_chunks	Embedded chunk of a knowledge base document used for retrieval. - Primary keys: id - Foreign keys: document_id -> kb_documents
05	kb_documents	Knowledge base document of the AI assistant with file, allowed roles and indexing status. - Primary keys: id - Foreign keys: none

3. Detailed Design

This section gives, for every feature, the class diagram of the classes that take part, the sequence diagram(s) of its main flows and the specification of the controller and service classes. Controllers receive requests already authenticated by the API gateway (X-User-Id, X-User-Roles), services hold the business logic, Feign clients call other services and repositories access the database of the service. Names are the real class and method names in the source code.

3.1 Authentication

Customers and technicians sign in with email or phone and password, with a Firebase ID token (phone OTP / Google) or with an email OTP; administrators sign in with a password followed by an email OTP (2FA). auth-service owns the credentials and issues a JWT access token (24 h) and a hashed, revocable refresh token (30 days); roles are read from user-service.

3.1.1 Class Diagram

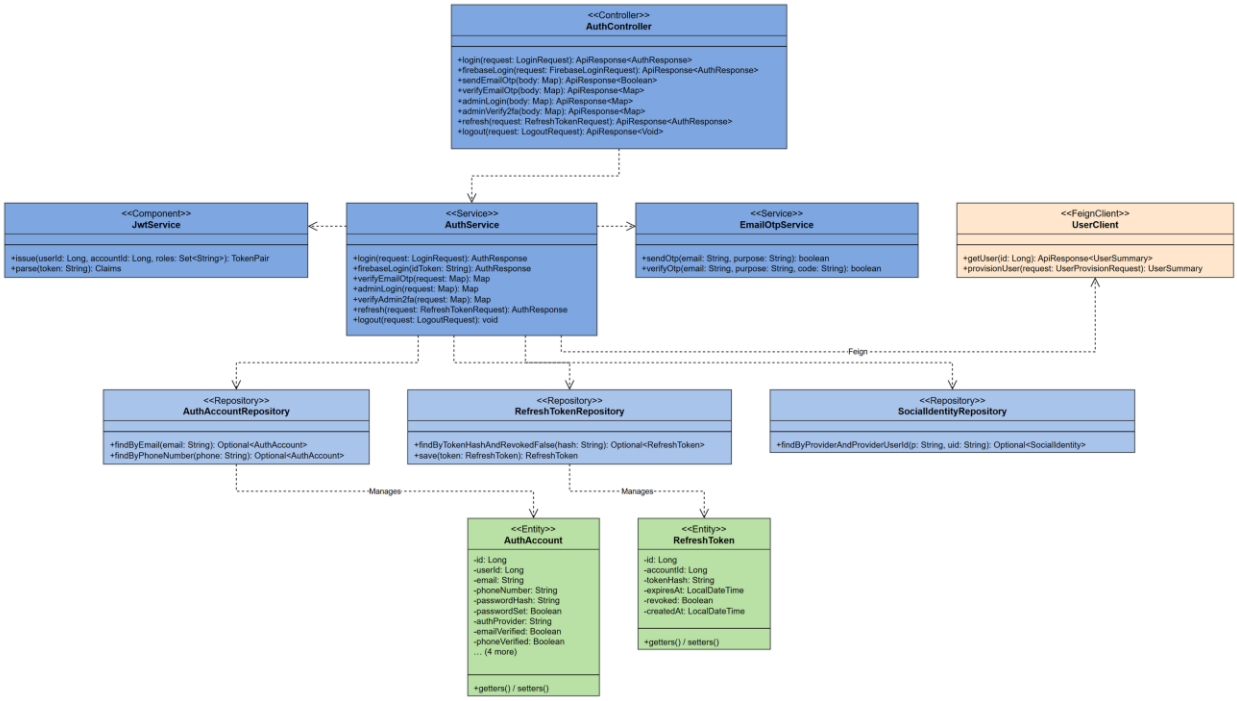


Figure 3.1.1: Class diagram - Authentication

3.1.2 Sign in with Password

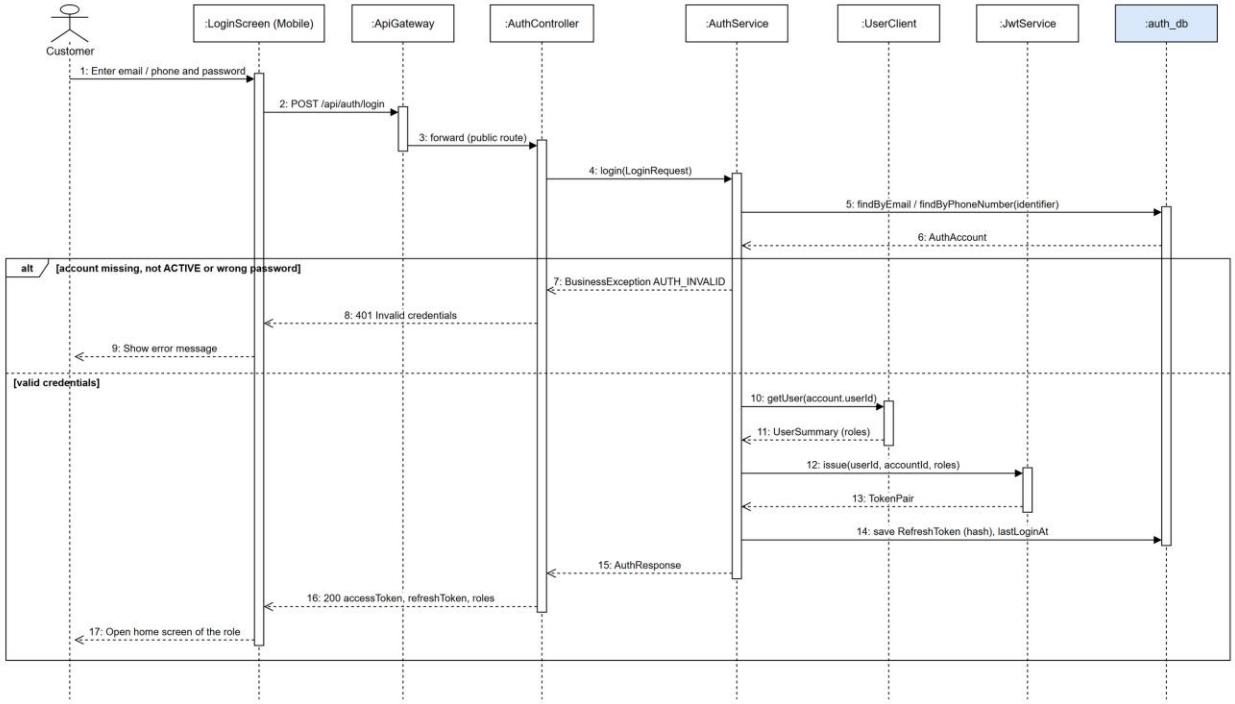


Figure 3.1.2: Sequence diagram - Sign in with Password

3.1.3 Sign in with Phone Number (Firebase OTP)

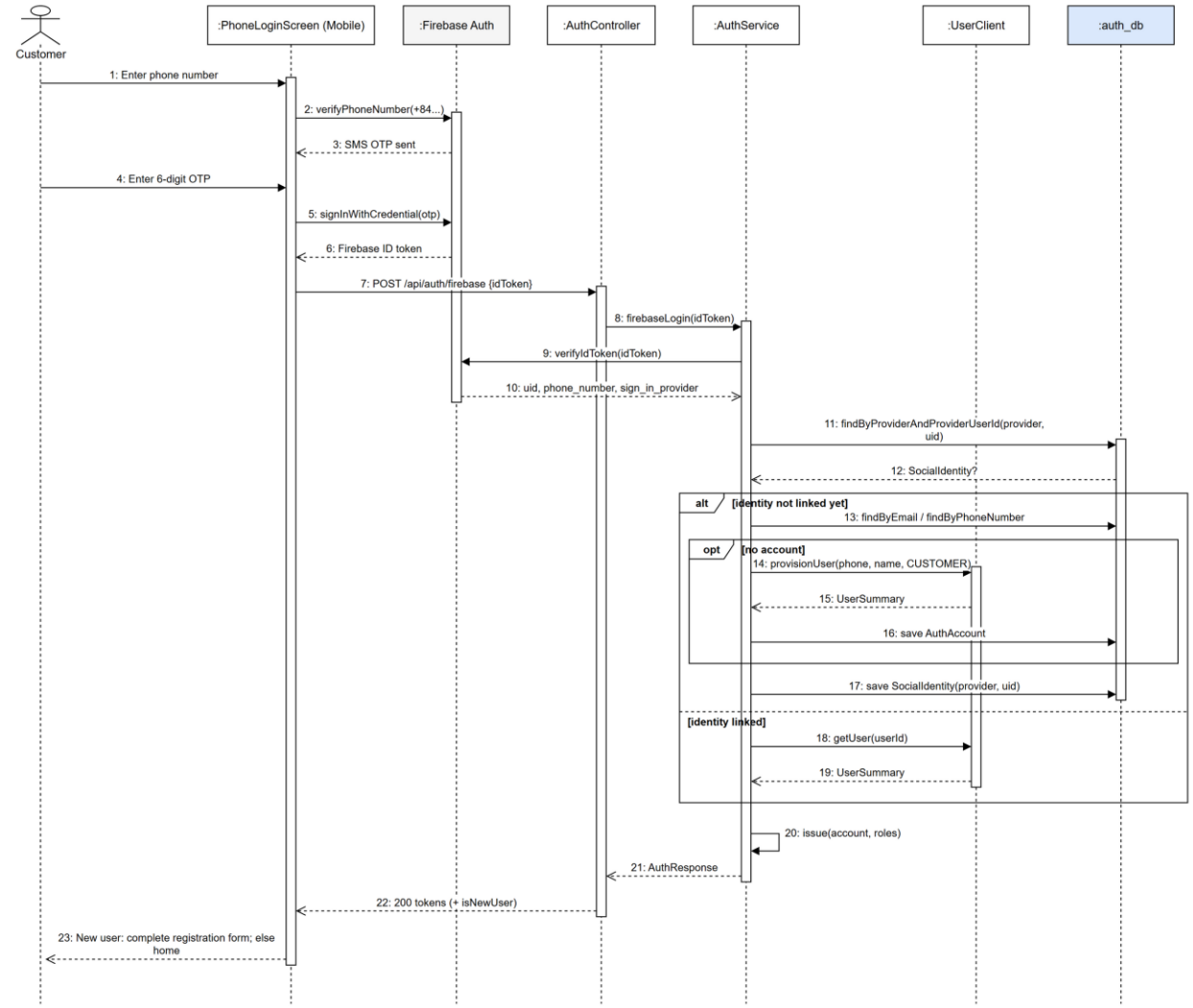


Figure 3.1.3: Sequence diagram - Sign in with Phone Number (Firebase OTP)

3.1.4 Administrator Sign in with 2FA

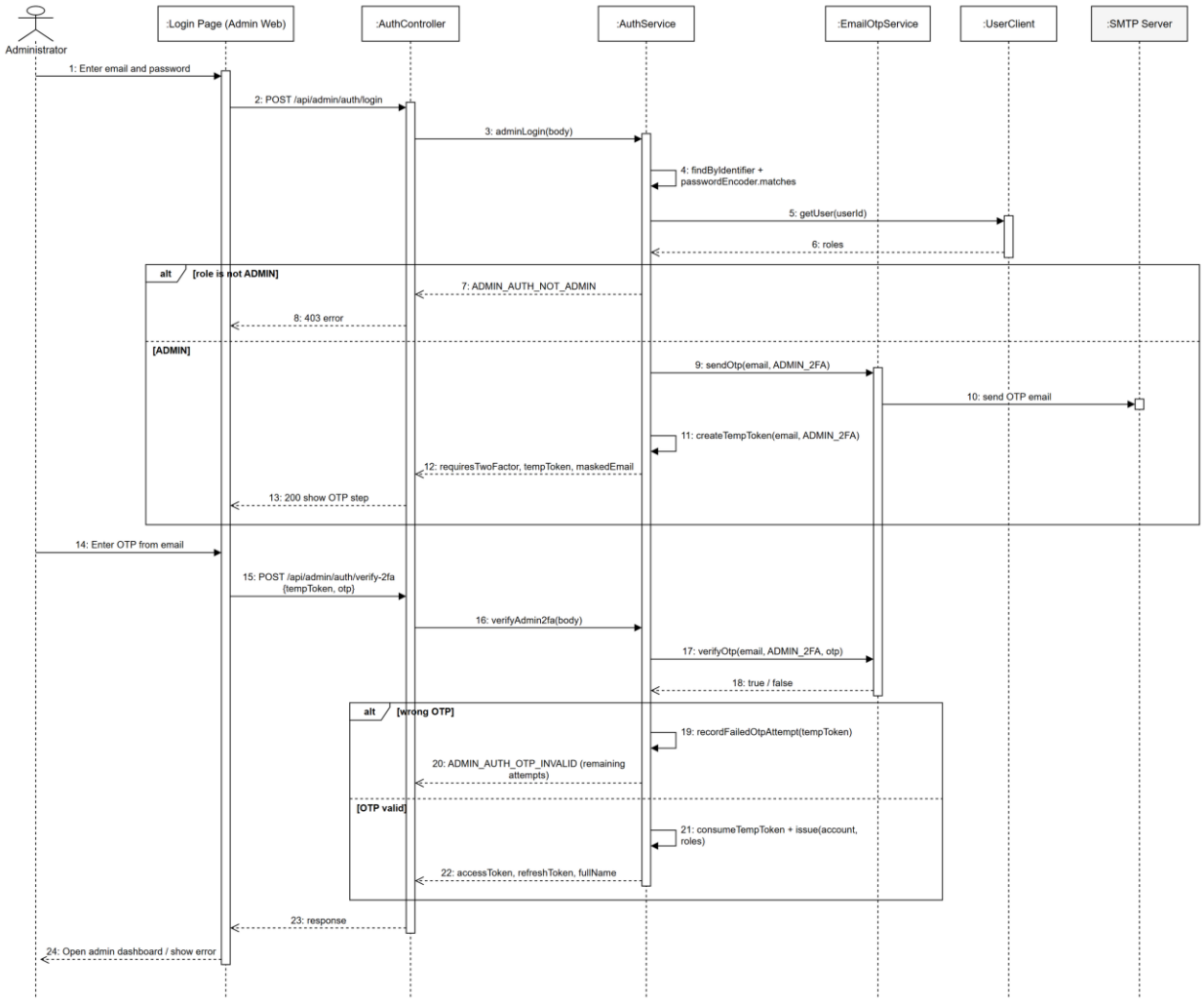


Figure 3.1.4: Sequence diagram - Administrator Sign in with 2FA

3.1.5 Refresh Token and Sign out

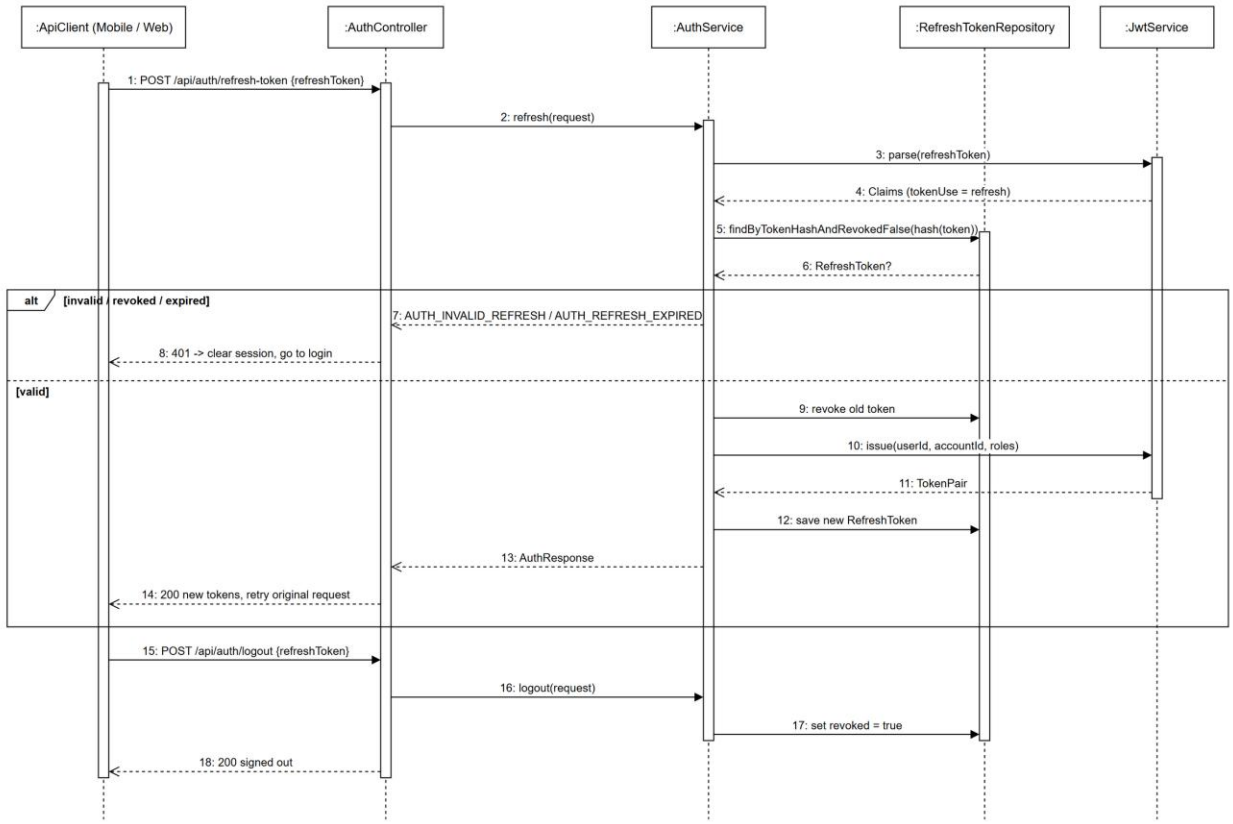


Figure 3.1.5: Sequence diagram - Refresh Token and Sign out

3.1.6 Class Specification

3.1.6.a AuthController

Table 3.1.6.a: AuthController description

No	Method	Description
01	login(request: LoginRequest): ApiResponse<AuthResponse>	Signs in a customer or technician by email / phone and password.
02	firebaseLogin(request: FirebaseLoginRequest): ApiResponse<AuthResponse>	Exchanges a verified Firebase ID token (phone OTP, Google) for Lock.R tokens.
03	sendEmailOtp(body: Map): ApiResponse<Boolean>	Sends a 6-digit login / registration OTP to an email address.
04	verifyEmailOtp(body: Map): ApiResponse<Map>	Verifies the email OTP; returns tokens, or a temp token when the account still has to be registered.
05	adminLogin(body: Map): ApiResponse<Map>	Admin step 1: checks password and ADMIN role, sends the 2FA OTP and returns a temp token.
06	adminVerify2fa(body: Map): ApiResponse<Map>	Admin step 2: verifies the OTP bound to the temp token and issues tokens.
07	refresh(request: RefreshTokenRequest): ApiResponse<AuthResponse>	Rotates the refresh token and issues a new access token.

No	Method	Description
08	logout(request: LogoutRequest): ApiResponse<Void>	Revokes the refresh token of the device.

3.1.6.b AuthService

Table 3.1.6.b: AuthService description

No	Method	Description
01	login(request: LoginRequest): AuthResponse	Finds the account by identifier, checks ACTIVE status and the BCrypt hash, loads roles and issues tokens.
02	firebaseLogin(idToken: String): AuthResponse	Verifies the ID token with Firebase Admin, links or provisions the account through SocialIdentity and issues tokens.
03	verifyEmailOtp(request: Map): Map	Checks the OTP with EmailOtpService; known email -> tokens, new email -> temp token.
04	adminLogin(request: Map): Map	Validates admin credentials, sends the ADMIN_2FA OTP and creates a short-lived temp token.
05	verifyAdmin2fa(request: Map): Map	Verifies the OTP (limited attempts), consumes the temp token and issues tokens.
06	refresh(request: RefreshTokenRequest): AuthResponse	Validates the stored refresh-token hash, revokes it and issues a new pair.
07	logout(request: LogoutRequest): void	Marks the refresh token as revoked.

3.1.6.c JwtService

Table 3.1.6.c: JwtService description

No	Method	Description
01	issue(userId: Long, accountId: Long, roles: Set<String>): TokenPair	Signs the access token (sub, accountId, roles, tokenUse) and creates a random refresh token.
02	parse(token: String): Claims	Verifies the HS signature and returns the claims.

3.1.6.d EmailOtpService

Table 3.1.6.d: EmailOtpService description

No	Method	Description
01	sendOtp(email: String, purpose: String): boolean	Generates an OTP, stores its hash with expiry and sends it by SMTP.
02	verifyOtp(email: String, purpose: String, code: String): boolean	Matches the latest unused OTP of the purpose and marks it used.

3.2 Kiosk Login by QR

A customer signs in at the locker kiosk by scanning a QR code with the mobile app. The kiosk creates a short-lived session, the logged-in app confirms it with the short code, and the kiosk polls until it receives a kiosk-scoped login grant.

3.2.1 Class Diagram

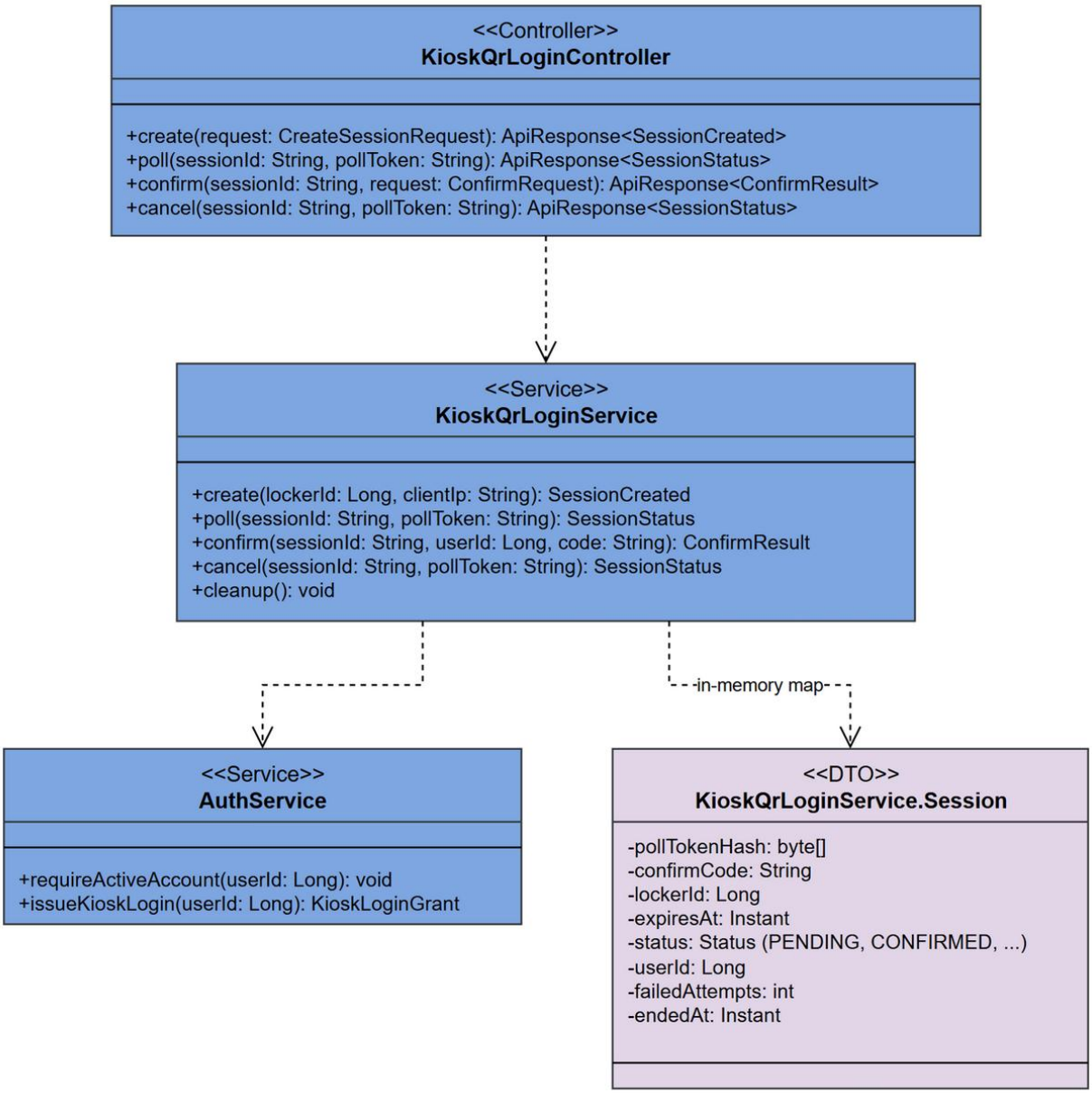


Figure 3.2.1: Class diagram - Kiosk Login by QR

3.2.2 Kiosk Login by QR

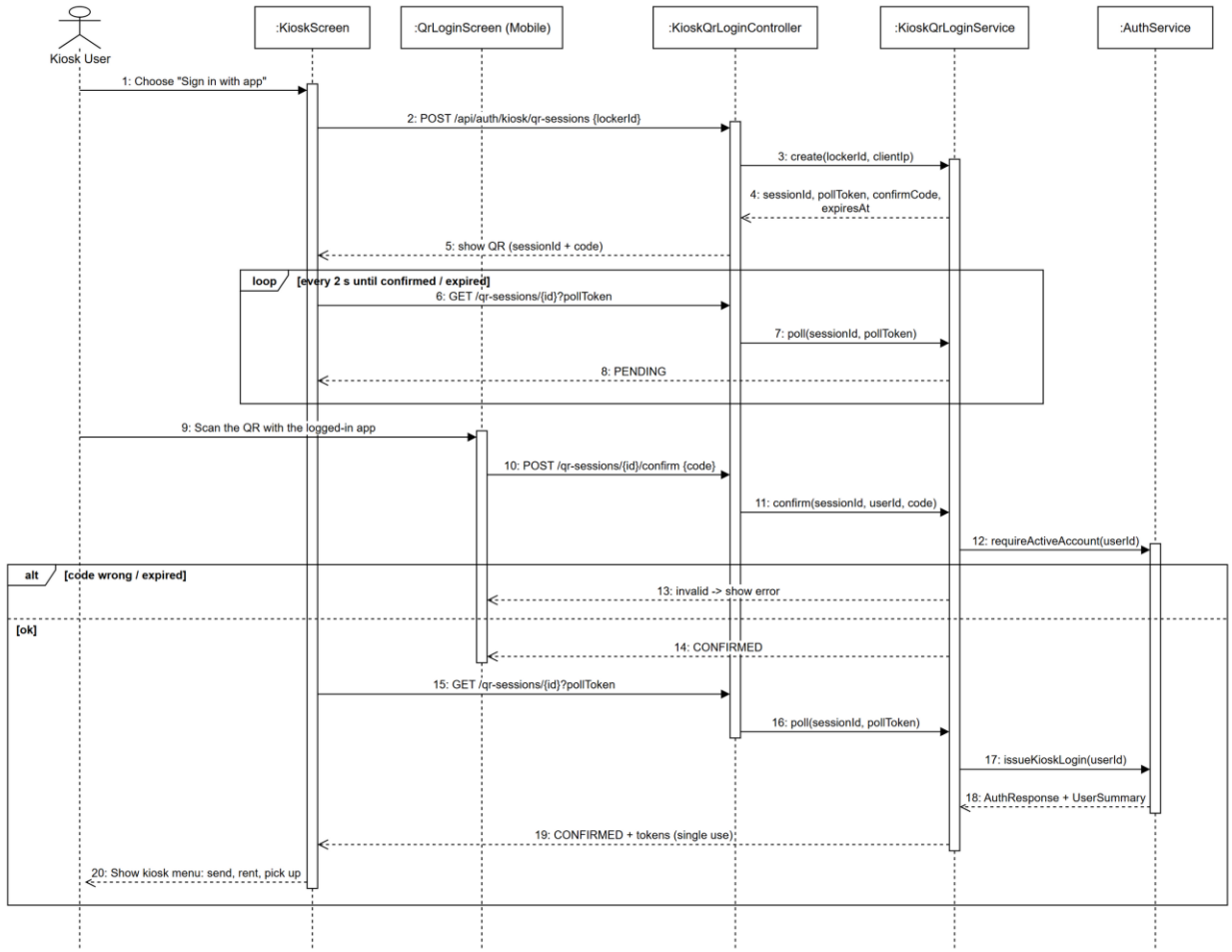


Figure 3.2.2: Sequence diagram - Kiosk Login by QR

3.2.3 Class Specification

3.2.3.a KioskQrLoginController

Table 3.2.3.a: KioskQrLoginController description

No	Method	Description
01	create(request: CreateSessionRequest): ApiResponse<SessionCreated>	Kiosk opens a QR login session for its locker.
02	poll(sessionId: String, pollToken: String): ApiResponse<SessionStatus>	Kiosk polls the session; returns tokens once confirmed.
03	confirm(sessionId: String, request: ConfirmRequest): ApiResponse<ConfirmResult>	Logged-in app confirms the session with the confirm code.
04	cancel(sessionId: String, pollToken: String): ApiResponse<SessionStatus>	Kiosk cancels the session.

3.2.3.b KioskQrLoginService

Table 3.2.3.b: KioskQrLoginService description

No	Method	Description
01	create(lockerId: Long, clientIp: String): SessionCreated	Rate-limits per locker and creates an in-memory session with a random poll token (stored hashed) and a confirm code.
02	poll(sessionId: String, pollToken: String): SessionStatus	Checks the poll token, expires old sessions and, when CONFIRMED, issues the kiosk login once.
03	confirm(sessionId: String, userId: Long, code: String): ConfirmResult	Matches the confirm code, requires an active account and binds the user to the session.
04	cancel(sessionId: String, pollToken: String): SessionStatus	Ends the session as CANCELLED.
05	cleanup(): void	Scheduled removal of expired sessions.

3.3 Profile and User Management

Every user keeps a profile (name, email, phone, avatar) in user-service. Images are uploaded straight to Cloudinary with a signature issued by the backend and verified before the URL is stored. Administrators create users and technicians, lock accounts and change roles; credential changes are pushed to auth-service through Feign.

3.3.1 Class Diagram

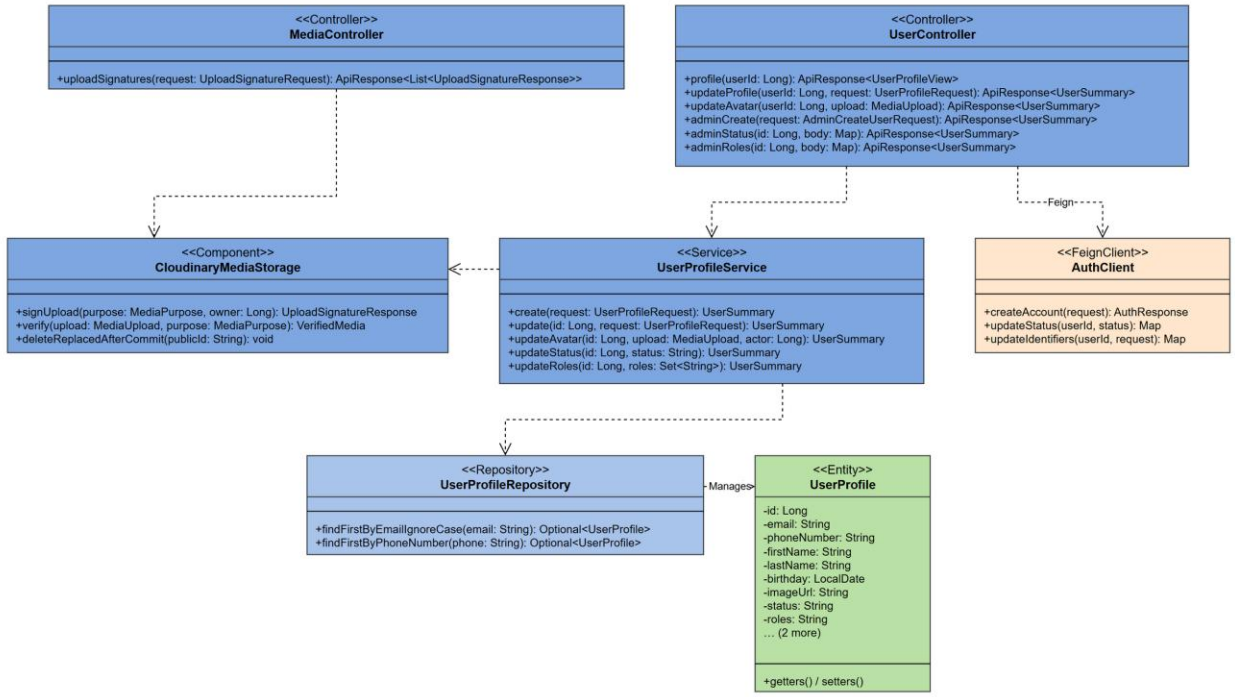


Figure 3.3.1: Class diagram - Profile and User Management

3.3.2 Update Profile Avatar

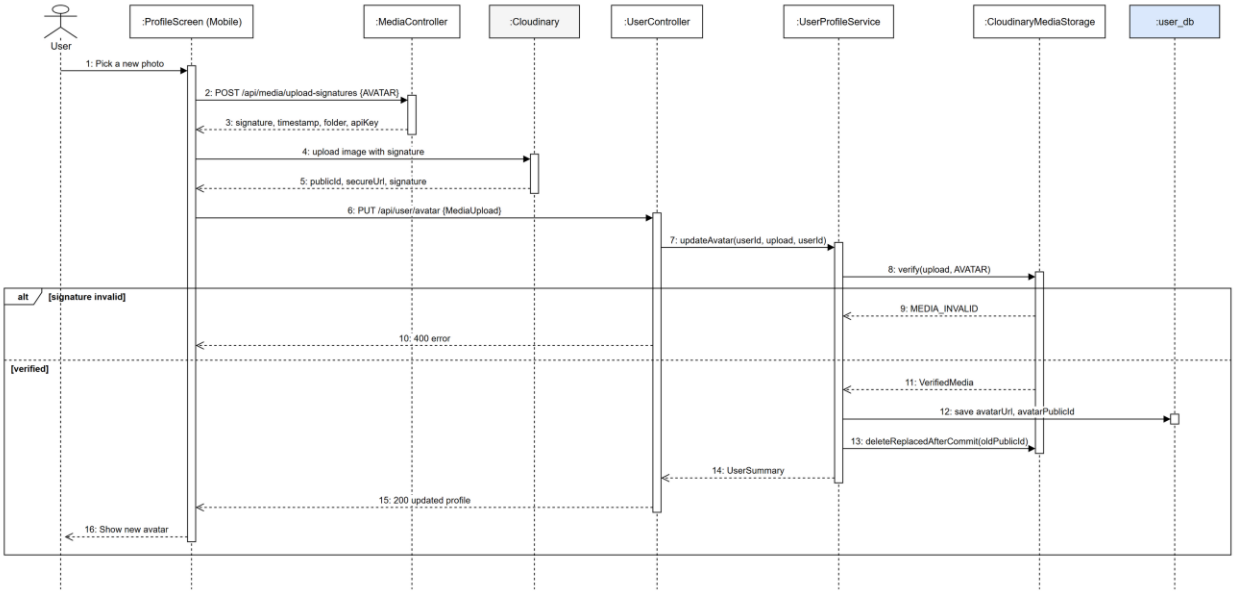


Figure 3.3.2: Sequence diagram - Update Profile Avatar

3.3.3 Administrator Creates and Deactivates a User

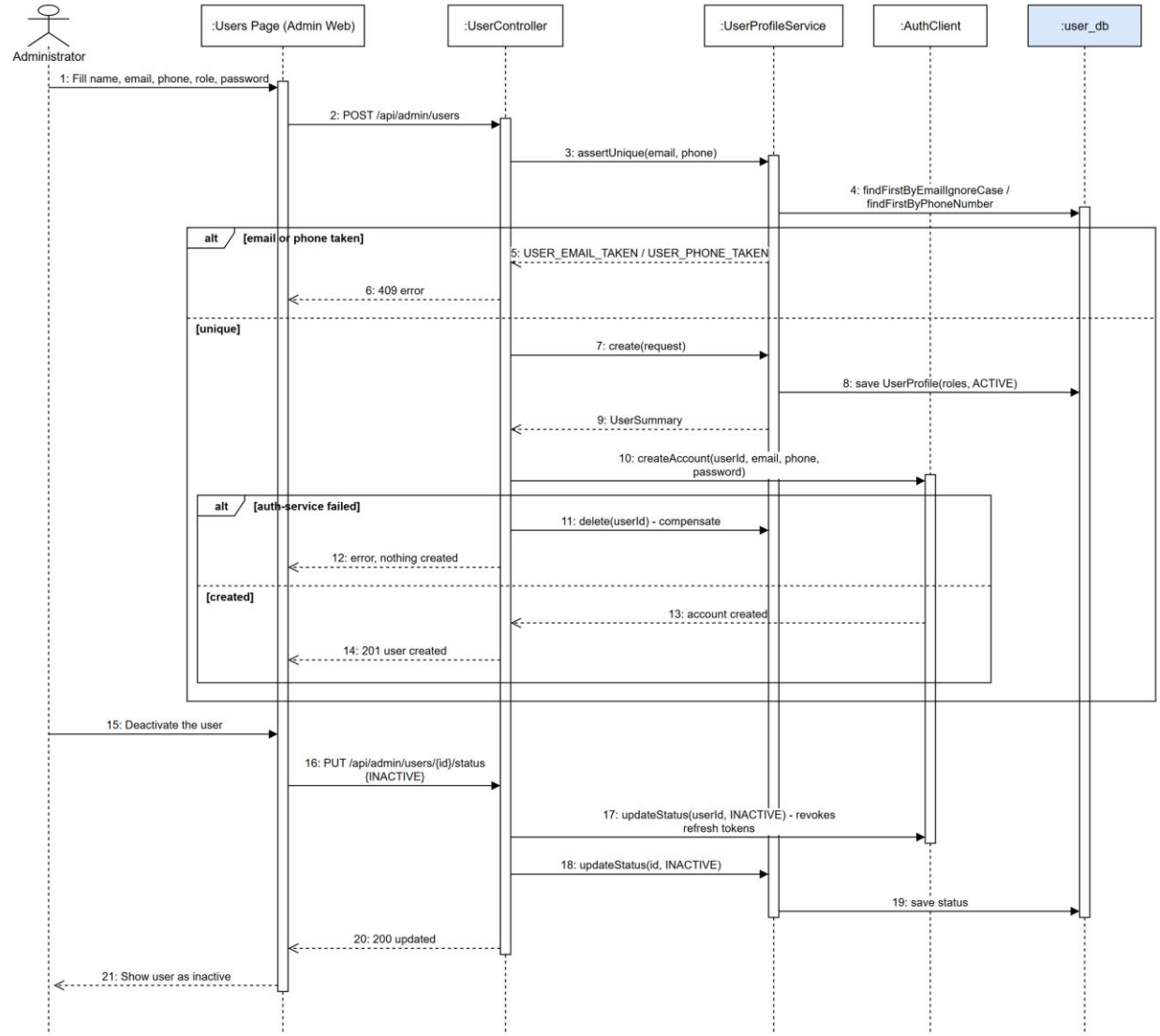


Figure 3.3.3: Sequence diagram - Administrator Creates and Deactivates a User

3.3.4 Class Specification

3.3.4.a UserController

Table 3.3.4.a: UserController description

No	Method	Description
01	profile(userId: Long): ApiResponse<UserProfileView>	Returns the profile of the signed-in user.
02	updateProfile(userId: Long, request: UserProfileRequest): ApiResponse<UserSummary>	Updates name, email, phone (checked for uniqueness).
03	updateAvatar(userId: Long, upload: MediaUpload): ApiResponse<UserSummary>	Stores a verified Cloudinary avatar.

No	Method	Description
04	adminCreate(request: AdminCreateUserRequest): ApiResponse<UserSummary>	Admin creates a customer or technician account.
05	adminStatus(id: Long, body: Map): ApiResponse<UserSummary>	Admin deactivates / reactivates a user.
06	adminRoles(id: Long, body: Map): ApiResponse<UserSummary>	Admin changes the roles of a user.

3.3.4.b MediaController

Table 3.3.4.b: MediaController description

No	Method	Description
01	uploadSignatures(request: UploadSignatureRequest): ApiResponse<List<UploadSignatureResponse>>	Issues signed Cloudinary upload parameters for a media purpose.

3.3.4.c UserProfileService

Table 3.3.4.c: UserProfileService description

No	Method	Description
01	create(request: UserProfileRequest): UserSummary	Validates unique email / phone and saves the profile.
02	update(id: Long, request: UserProfileRequest): UserSummary	Updates profile fields after uniqueness checks.
03	updateAvatar(id: Long, upload: MediaUpload, actor: Long): UserSummary	Verifies the Cloudinary upload and replaces the old image after commit.
04	updateStatus(id: Long, status: String): UserSummary	Changes ACTIVE / INACTIVE status.
05	updateRoles(id: Long, roles: Set<String>): UserSummary	Replaces the role set.

3.4 Store Management and Nearby Stores

Stores are the places that host lockers. Customers and guests list stores, find the nearest ones by GPS and see their lockers and ratings; administrators create, edit, deactivate stores and set their image.

3.4.1 Class Diagram

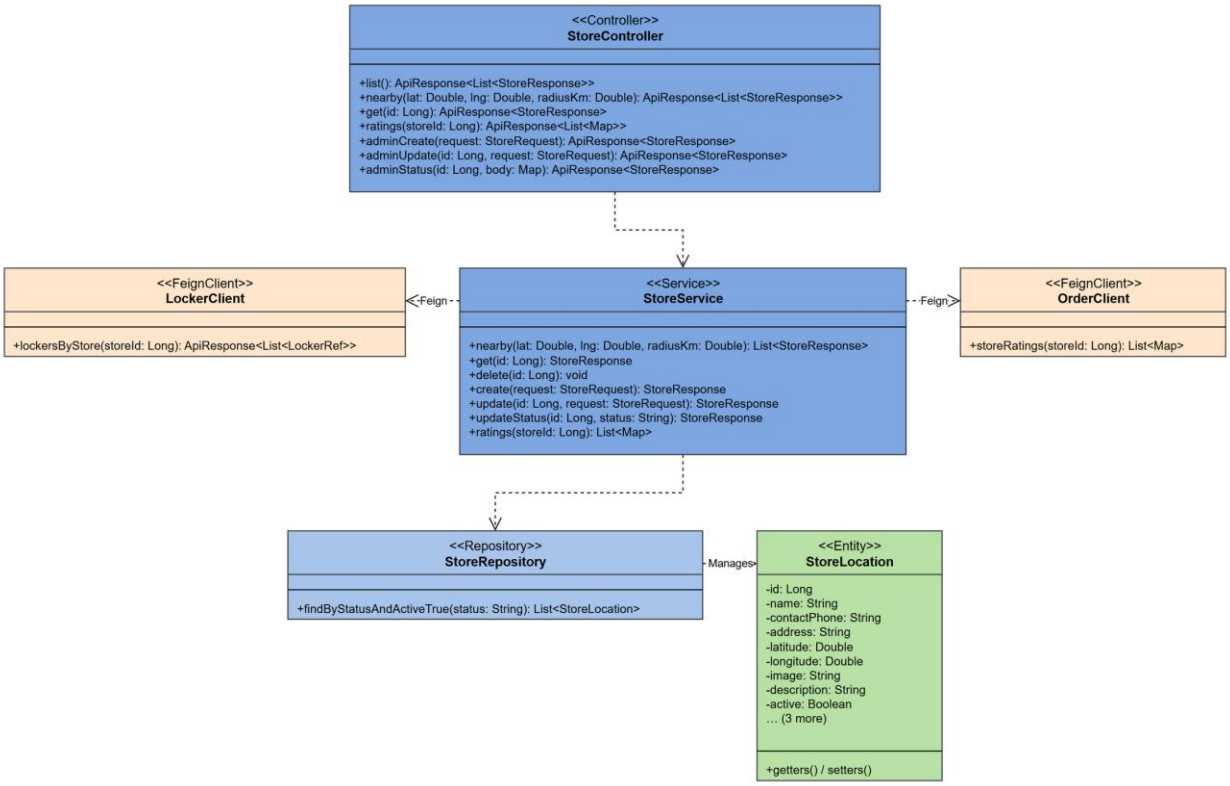


Figure 3.4.1: Class diagram - Store Management and Nearby Stores

3.4.2 Find Nearby Stores

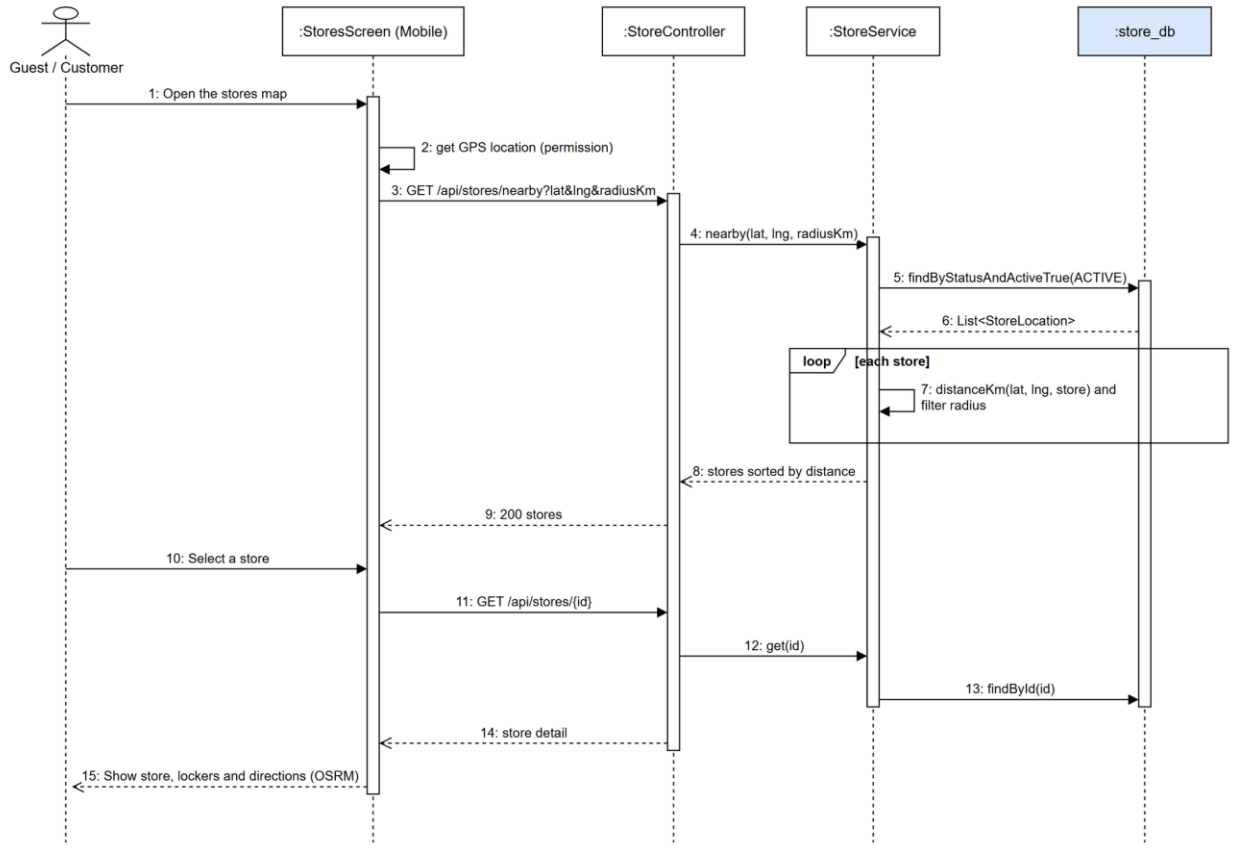


Figure 3.4.2: Sequence diagram - Find Nearby Stores

3.4.3 Class Specification

3.4.3.a StoreController

Table 3.4.3.a: StoreController description

No	Method	Description
01	list(): ApiResponse<List<StoreResponse>>	Lists active stores.
02	nearby(lat: Double, lng: Double, radiusKm: Double): ApiResponse<List<StoreResponse>>	Lists active stores within a radius ordered by distance.
03	get(id: Long): ApiResponse<StoreResponse>	Store detail with its lockers.
04	ratings(storeId: Long): ApiResponse<List<Map>>	Ratings of orders made at the store.
05	adminCreate(request: StoreRequest): ApiResponse<StoreResponse>	Admin creates a store.
06	adminUpdate(id: Long, request: StoreRequest): ApiResponse<StoreResponse>	Admin edits a store.
07	adminStatus(id: Long, body: Map): ApiResponse<StoreResponse>	Admin activates / deactivates a store.

3.4.3.b StoreService

Table 3.4.3.b: StoreService description

No	Method	Description
01	nearby(lat: Double, lng: Double, radiusKm: Double): List<StoreResponse>	Filters active stores by haversine distance and sorts them.
02	get(id: Long): StoreResponse	Loads one store.
03	delete(id: Long): void	Deletes a store after checking with locker-service that no locker still belongs to it.
04	create(request: StoreRequest): StoreResponse	Validates and saves a store, publishes catalog.changed.
05	update(id: Long, request: StoreRequest): StoreResponse	Updates fields and publishes catalog.changed.
06	updateStatus(id: Long, status: String): StoreResponse	Changes the store status.
07	ratings(storeId: Long): List<Map>	Reads store ratings from order-service.

3.5 Locker and Box Management

Administrators register locker cabinets for a store and define the box grid (number, size, cell type STANDARD / XL / DRONE, row / column). The locker layout combines the box states kept by locker-service with the door states and controller reported by iot-service, and is shown on the app, the kiosk and the admin web.

3.5.1 Class Diagram

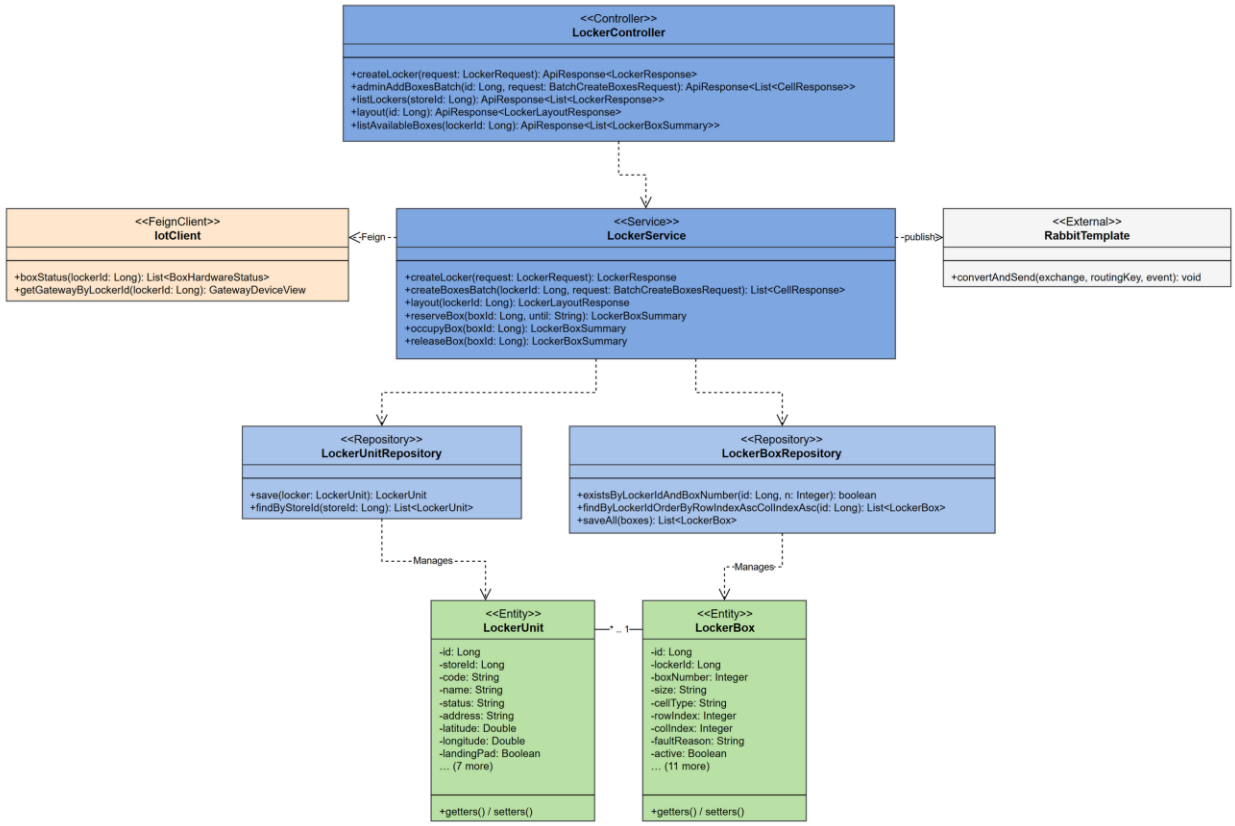


Figure 3.5.1: Class diagram - Locker and Box Management

3.5.2 Create Locker and Box Grid

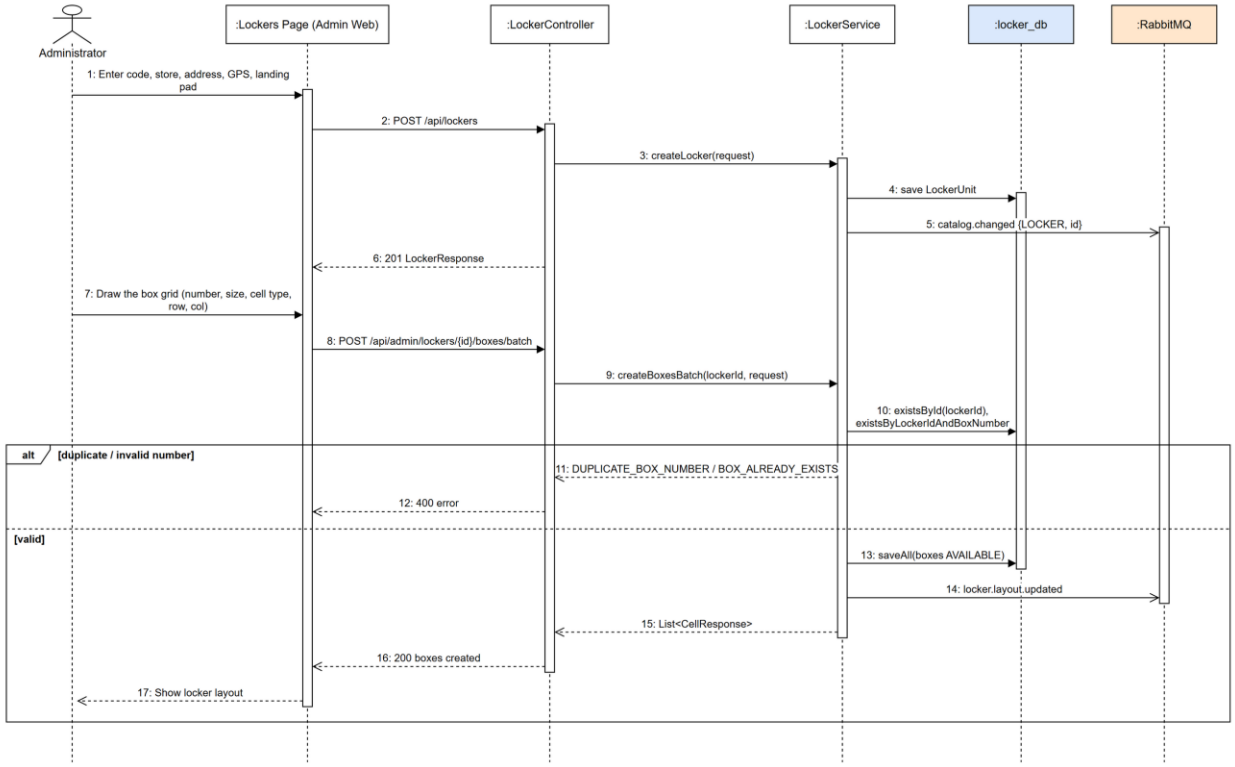


Figure 3.5.2: Sequence diagram - Create Locker and Box Grid

3.5.3 View Locker Layout

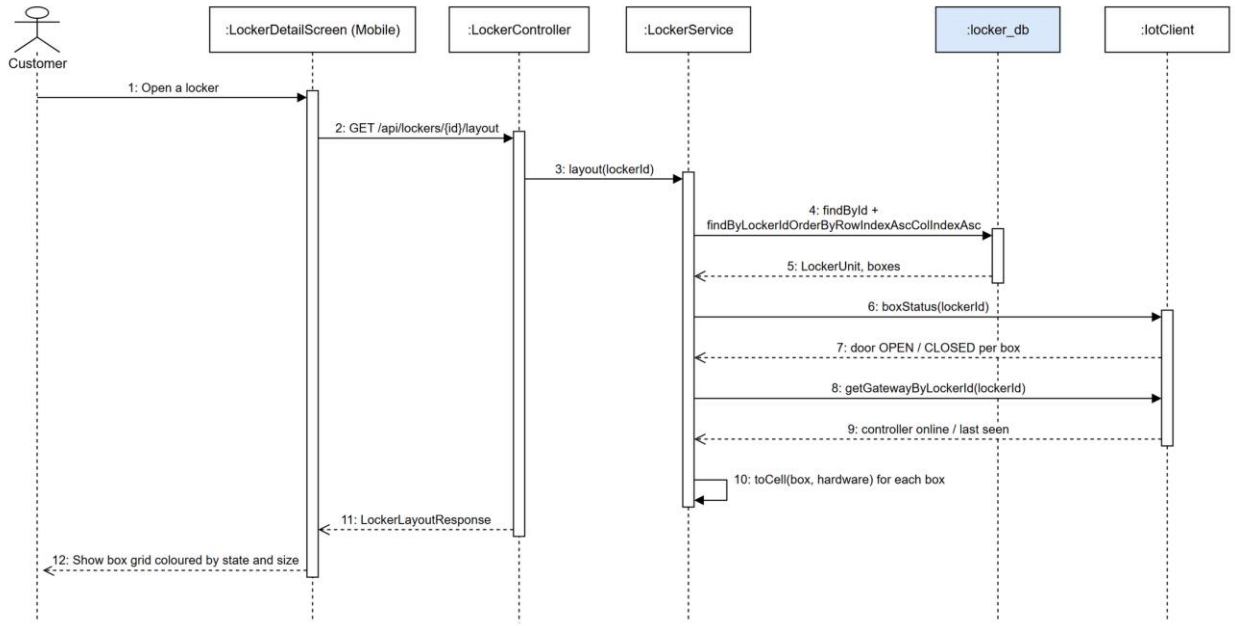


Figure 3.5.3: Sequence diagram - View Locker Layout

3.5.4 Class Specification

3.5.4.a LockerController

Table 3.5.4.a: LockerController description

No	Method	Description
01	createLocker(request: LockerRequest): ApiResponse<LockerResponse>	Admin creates a locker cabinet.
02	adminAddBoxesBatch(id: Long, request: BatchCreateBoxesRequest): ApiResponse<List<CellResponse>>	Admin adds several boxes to a locker grid.
03	listLockers(storeId: Long): ApiResponse<List<LockerResponse>>	Lists lockers (optionally of a store) with free-box counts.
04	layout(id: Long): ApiResponse<LockerLayoutResponse>	Returns the box grid with state, size and door state.
05	listAvailableBoxes(lockerId: Long): ApiResponse<List<LockerBoxSummary>>	Lists AVAILABLE boxes of a locker.

3.5.4.b LockerService

Table 3.5.4.b: LockerService description

No	Method	Description
01	createLocker(request: LockerRequest): LockerResponse	Saves the locker, applies the admin status and publishes catalog.changed.
02	createBoxesBatch(lockerId: Long, request: BatchCreateBoxesRequest): List<CellResponse>	Validates numbers (no duplicates), saves all boxes and publishes locker.layout.updated.
03	layout(lockerId: Long): LockerLayoutResponse	Builds the grid from boxes + hardware statuses + gateway of the locker.
04	reserveBox(boxId: Long, until: String): LockerBoxSummary	Marks a box RESERVED for a new order.
05	occupyBox(boxId: Long): LockerBoxSummary	Marks a box OCCUPIED after the parcel is dropped.
06	releaseBox(boxId: Long): LockerBoxSummary	Returns a box to AVAILABLE.

3.6 Send Parcel

A customer chooses a locker and a box size, enters the receiver (phone, name, optional email), applies vouchers or loyalty points and creates a SEND order. order-service reserves a box in locker-service, generates the order code and the 6-digit PIN, records the status history and notifies the sender and, when the receiver has an account, the receiver.

3.6.1 Class Diagram

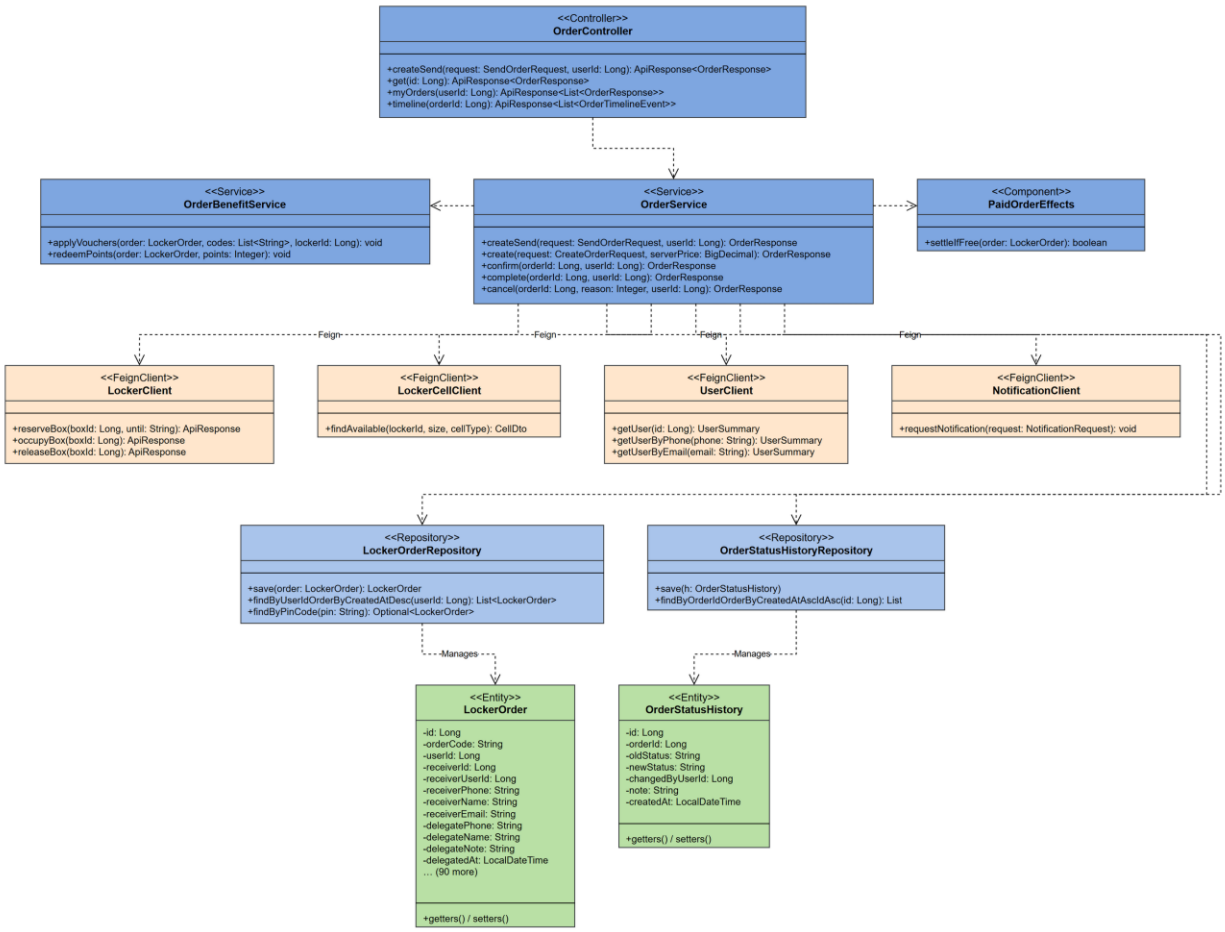


Figure 3.6.1: Class diagram - Send Parcel

3.6.2 Create Send Order

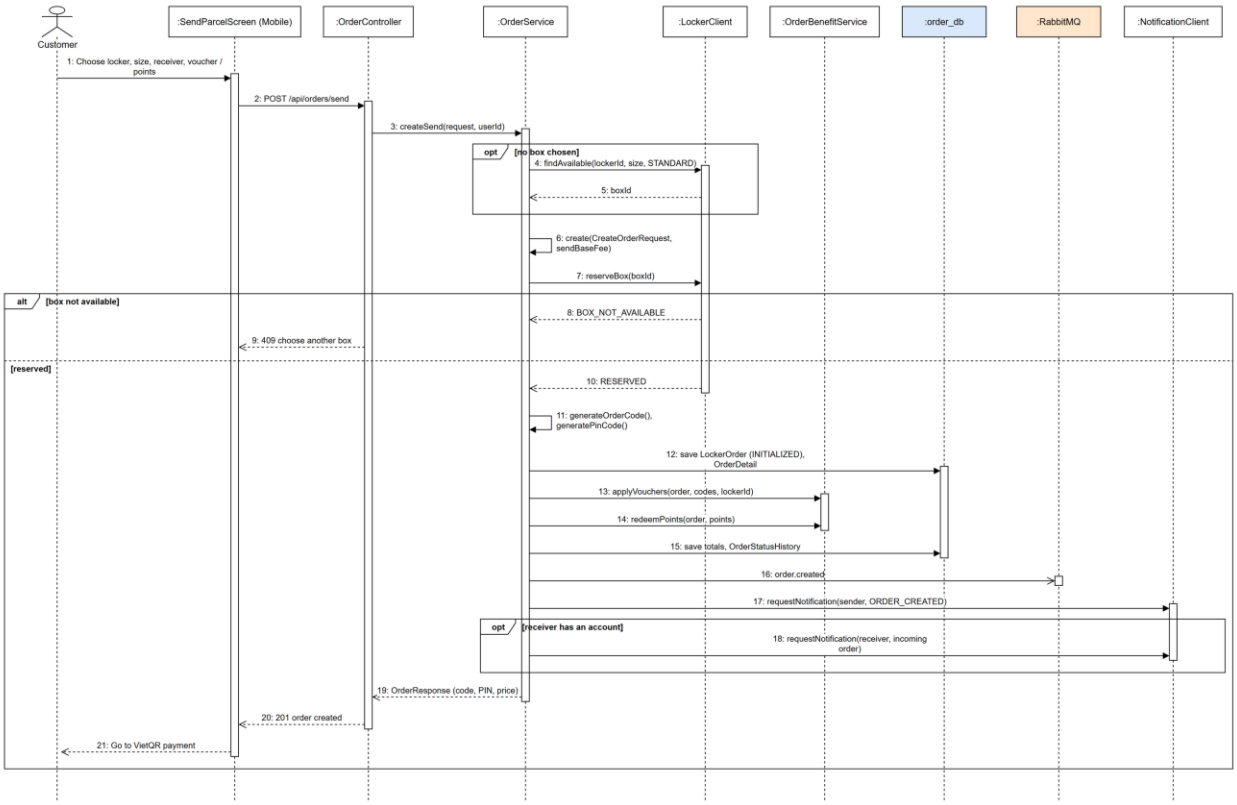


Figure 3.6.2: Sequence diagram - Create Send Order

3.6.3 Class Specification

3.6.3.a OrderController

Table 3.6.3.a: OrderController description

No	Method	Description
01	createSend(request: SendOrderRequest, userId: Long): ApiResponse<OrderResponse>	Creates a SEND order for the signed-in customer.
02	get(id: Long): ApiResponse<OrderResponse>	Order detail with PIN / QR for the owner.
03	myOrders(userId: Long): ApiResponse<List<OrderResponse>>	Orders of the customer.
04	timeline(orderId: Long): ApiResponse<List<OrderTimelineEvent>>	Status history of an order.

3.6.3.b OrderService

Table 3.6.3.b: OrderService description

No	Method	Description
01	createSend(request: SendOrderRequest, userId: Long): OrderResponse	Finds a free cell when no box is given, creates the order with the SEND base fee and links the receiver account.

No	Method	Description
02	create(request: CreateOrderRequest, serverPrice: BigDecimal): OrderResponse	Generates code and PIN, reserves the box, saves details, applies vouchers / points, writes history, publishes order.created and notifies.
03	confirm(orderId: Long, userId: Long): OrderResponse	Sender confirms the drop: occupies the box, issues a new PIN and notifies the receiver.
04	complete(orderId: Long, userId: Long): OrderResponse	Completes the order after pickup and releases the box.
05	cancel(orderId: Long, reason: Integer, userId: Long): OrderResponse	Cancels an unpaid / undropped order and releases the box and benefits.

3.6.3.c OrderBenefitService

Table 3.6.3.c: OrderBenefitService description

No	Method	Description
01	applyVouchers(order: LockerOrder, codes: List<String>, lockerId: Long): void	Validates vouchers for the locker and writes promotion usages.
02	redeemPoints(order: LockerOrder, points: Integer): void	Redeems loyalty points as a discount through loyalty-service.

3.6.3.d PaidOrderEffects

Table 3.6.3.d: PaidOrderEffects description

No	Method	Description
01	settleIfFree(order: LockerOrder): boolean	Marks a zero-price order as paid immediately.

3.7 VietQR Payment (SePay)

Orders and wallet top-ups are paid by bank transfer through SePay. payment-service creates a PENDING payment with a unique reference and returns a VietQR image and transfer details. When the bank transfer arrives, SePay calls the webhook; the payment becomes COMPLETED only from that verified webhook, then payment.completed is published and order-service marks the order PAID.

3.7.1 Class Diagram

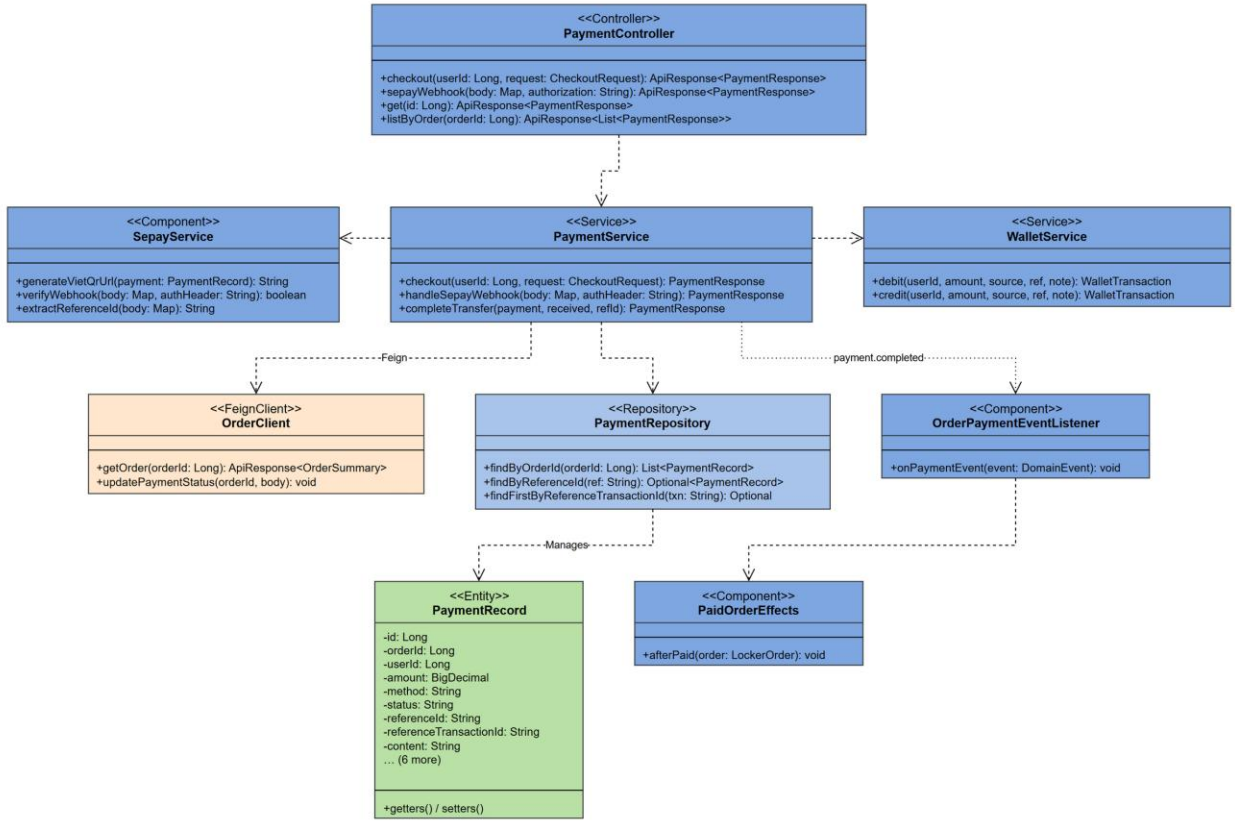


Figure 3.7.1: Class diagram - VietQR Payment (SePay)

3.7.2 Checkout with VietQR

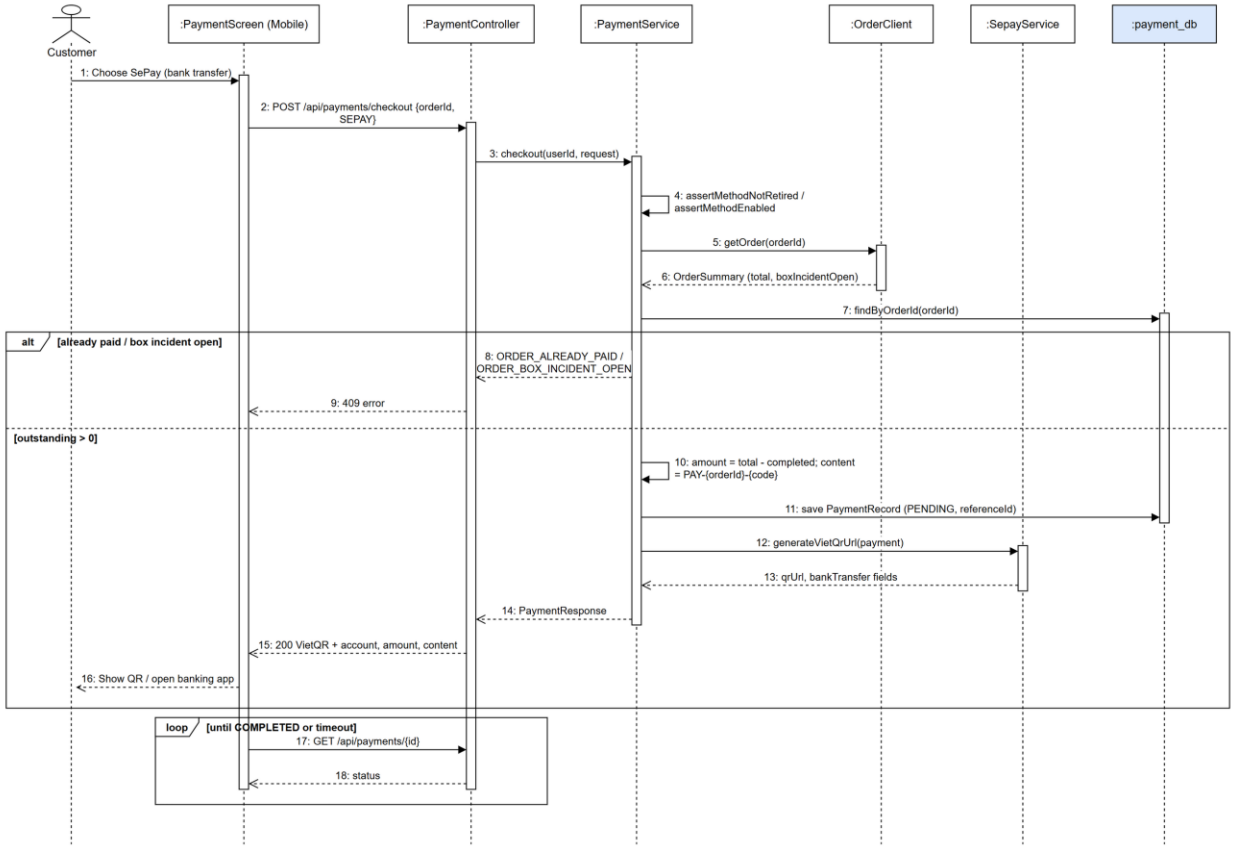


Figure 3.7.2: Sequence diagram - Checkout with VietQR

3.7.3 SePay Webhook Completes the Payment

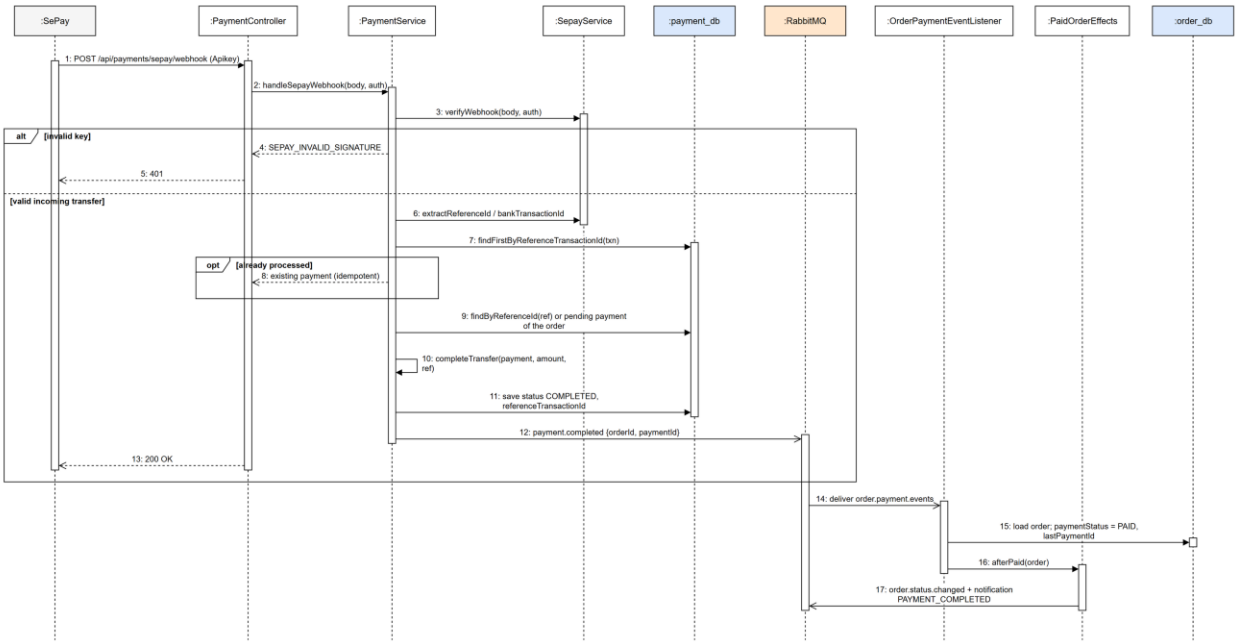


Figure 3.7.3: Sequence diagram - SePay Webhook Completes the Payment

3.7.4 Class Specification

3.7.4.a PaymentController

Table 3.7.4.a: PaymentController description

No	Method	Description
01	checkout(userId: Long, request: CheckoutRequest): ApiResponse<PaymentResponse>	Creates / reuses the pending payment of an order (SEPAY or WALLET).
02	sepayWebhook(body: Map, authorization: String): ApiResponse<PaymentResponse>	Receives the SePay bank-transfer webhook.
03	get(id: Long): ApiResponse<PaymentResponse>	Payment status polled by the app.
04	listByOrder(orderId: Long): ApiResponse<List<PaymentResponse>>	Payments of an order.

3.7.4.b PaymentService

Table 3.7.4.b: PaymentService description

No	Method	Description
01	checkout(userId: Long, request: CheckoutRequest): PaymentResponse	Checks method and order, computes the outstanding amount, builds the transfer content and creates a PENDING payment; WALLET is debited at once.
02	handleSepayWebhook(body: Map, authHeader: String): PaymentResponse	Verifies API key, ignores outgoing transfers and duplicates, finds the payment by reference and completes it.

No	Method	Description
03	completeTransfer(payment, received, refId): PaymentResponse	Marks COMPLETED, credits any surplus to the wallet and publishes payment.completed.

3.7.4.c SepayService

Table 3.7.4.c: SepayService description

No	Method	Description
01	generateVietQrUrl(payment: PaymentRecord): String	Builds the VietQR image URL (bank BIN, account, amount, content).
02	verifyWebhook(body: Map, authHeader: String): boolean	Checks the SePay API key.
03	extractReferenceId(body: Map): String	Reads the payment reference from the transfer content.

3.7.4.d OrderPaymentEventListener

Table 3.7.4.d: OrderPaymentEventListener description

No	Method	Description
01	onPaymentEvent(event: DomainEvent): void	Consumes payment.completed in order-service and marks the order PAID.

3.7.4.e PaidOrderEffects

Table 3.7.4.e: PaidOrderEffects description

No	Method	Description
01	afterPaid(order: LockerOrder): void	Publishes order.status.changed, notifies the customer and starts drone pickup for paid drone orders.

3.8 Drop and Pick Up Parcel at the Kiosk

At the locker the sender enters the order PIN / scans the QR to open the reserved box and drop the parcel; the receiver uses the PIN sent by SMS / email to open the box and take it. iot-service validates the code against order-service, enforces the wrong-PIN lockout, sends the open command over MQTT to the Raspberry Pi and waits for the result before updating the order and the box.

3.8.1 Class Diagram

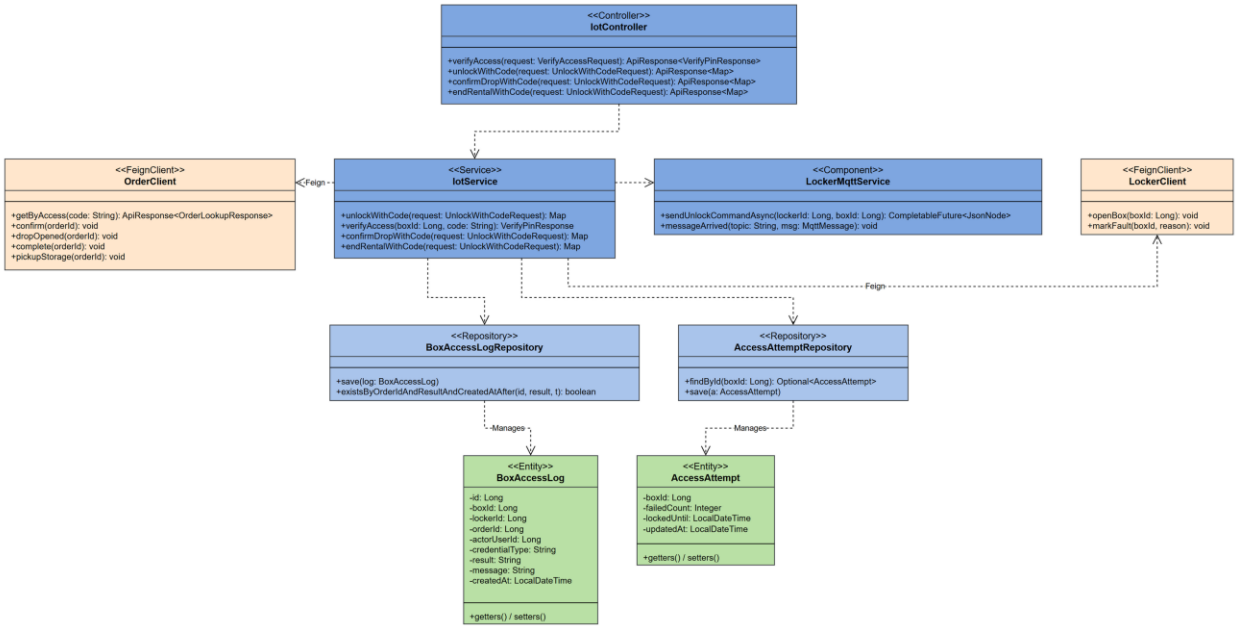


Figure 3.8.1: Class diagram - Drop and Pick Up Parcel at the Kiosk

3.8.2 Open Box with PIN at the Kiosk

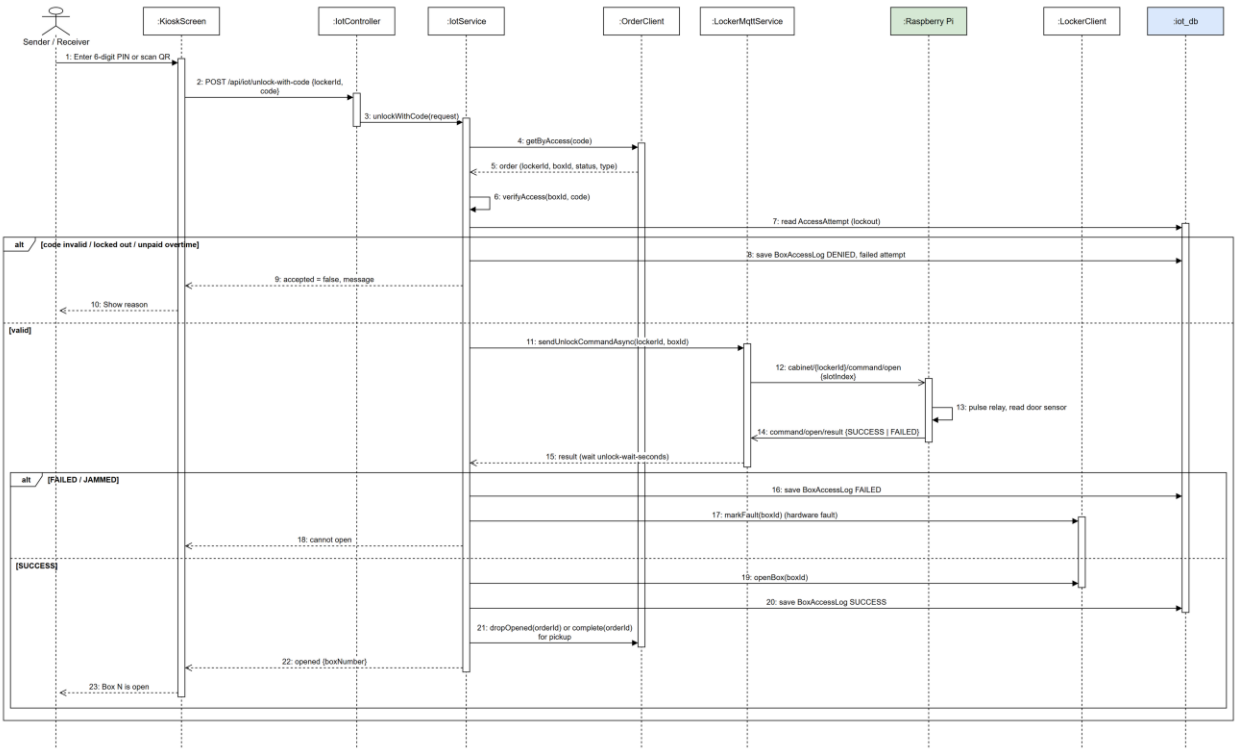


Figure 3.8.2: Sequence diagram - Open Box with PIN at the Kiosk

3.8.3 Confirm Parcel Drop

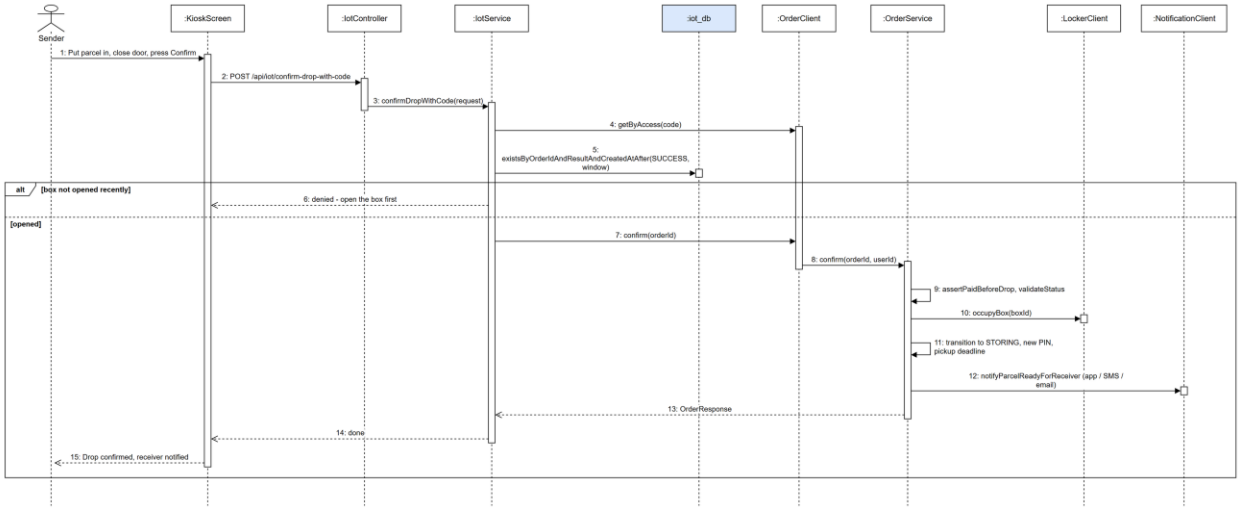


Figure 3.8.3: Sequence diagram - Confirm Parcel Drop

3.8.4 Class Specification

3.8.4.a IotController

Table 3.8.4.a: IotController description

No	Method	Description
01	verifyAccess(request: VerifyAccessRequest): ApiResponse<VerifyPinResponse>	Checks a PIN / QR code for a box (public kiosk route).
02	unlockWithCode(request: UnlockWithCodeRequest): ApiResponse<Map>	Opens the box of the order identified by the code.
03	confirmDropWithCode(request: UnlockWithCodeRequest): ApiResponse<Map>	Confirms the parcel was dropped after the door was opened.
04	endRentalWithCode(request: UnlockWithCodeRequest): ApiResponse<Map>	Ends a rental after the customer took the items.

3.8.4.b IotService

Table 3.8.4.b: IotService description

No	Method	Description
01	unlockWithCode(request: UnlockWithCodeRequest): Map	Looks up the order by access code, verifies the box and code, sends the unlock command and records the access.
02	verifyAccess(boxId: Long, code: String): VerifyPinResponse	Checks the code, payment rules and the wrong-attempt lockout (AccessAttempt).
03	confirmDropWithCode(request: UnlockWithCodeRequest): Map	Requires a recent successful open, then confirms the order in order-service.
04	endRentalWithCode(request: UnlockWithCodeRequest): Map	Requires a recent open, then calls pickupStorage to end the rental.

3.8.4.c LockerMqttService

Table 3.8.4.c: LockerMqttService description

No	Method	Description
01	sendUnlockCommandAsync(lockerId: Long, boxId: Long): CompletableFuture<JsonNode>	Publishes cabinet/{lockerId}/command/open and completes when /result arrives.
02	messageArrived(topic: String, msg: MqttMessage): void	Handles results, heartbeats and door status from the Pi.

3.9 Rent Locker, Extend and End Rental

A customer rents a box for a number of hours (price by cell type and hours from business settings), extends the rental from the app and ends it at the kiosk or on the phone. Overtime is assessed before the box can be opened again.

3.9.1 Class Diagram

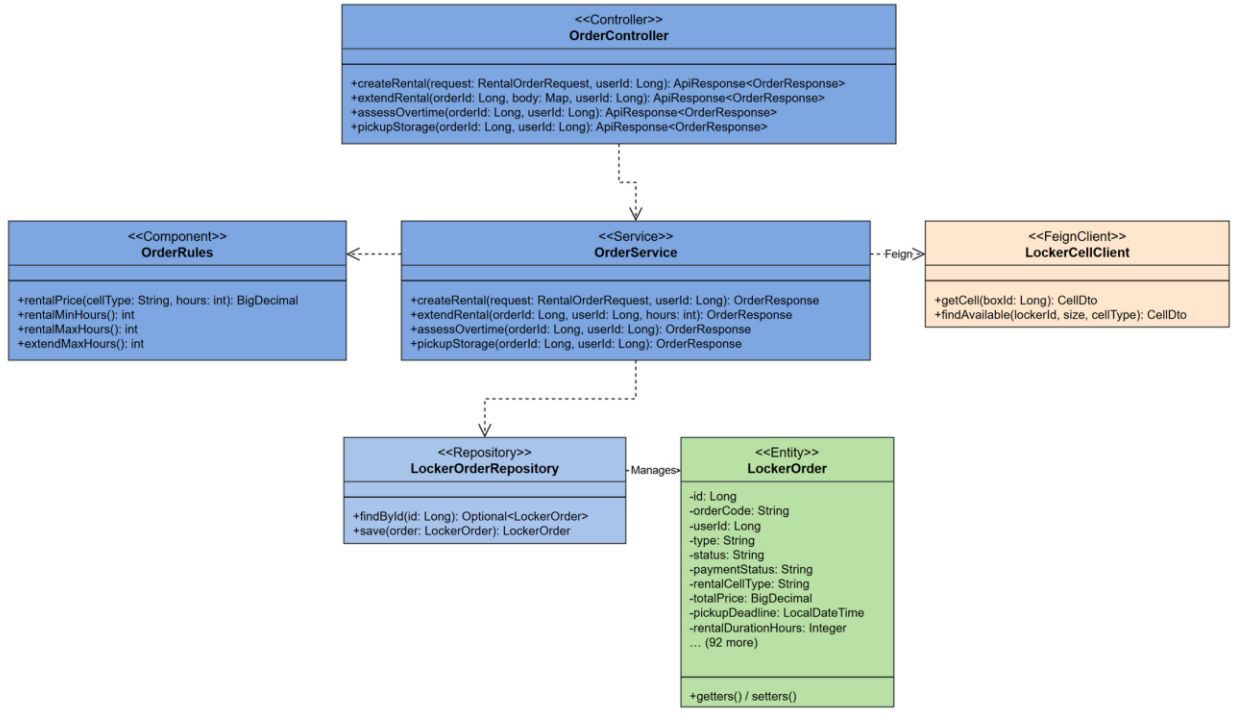


Figure 3.9.1: Class diagram - Rent Locker, Extend and End Rental

3.9.2 Create Rental Order

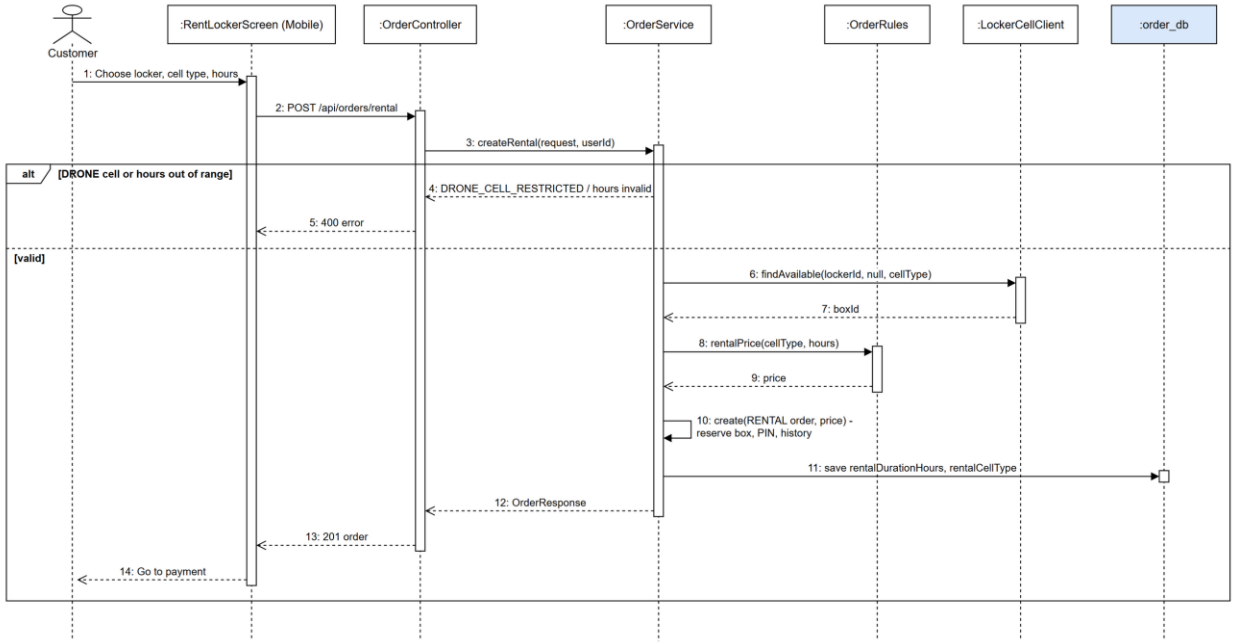


Figure 3.9.2: Sequence diagram - Create Rental Order

3.9.3 Extend Rental

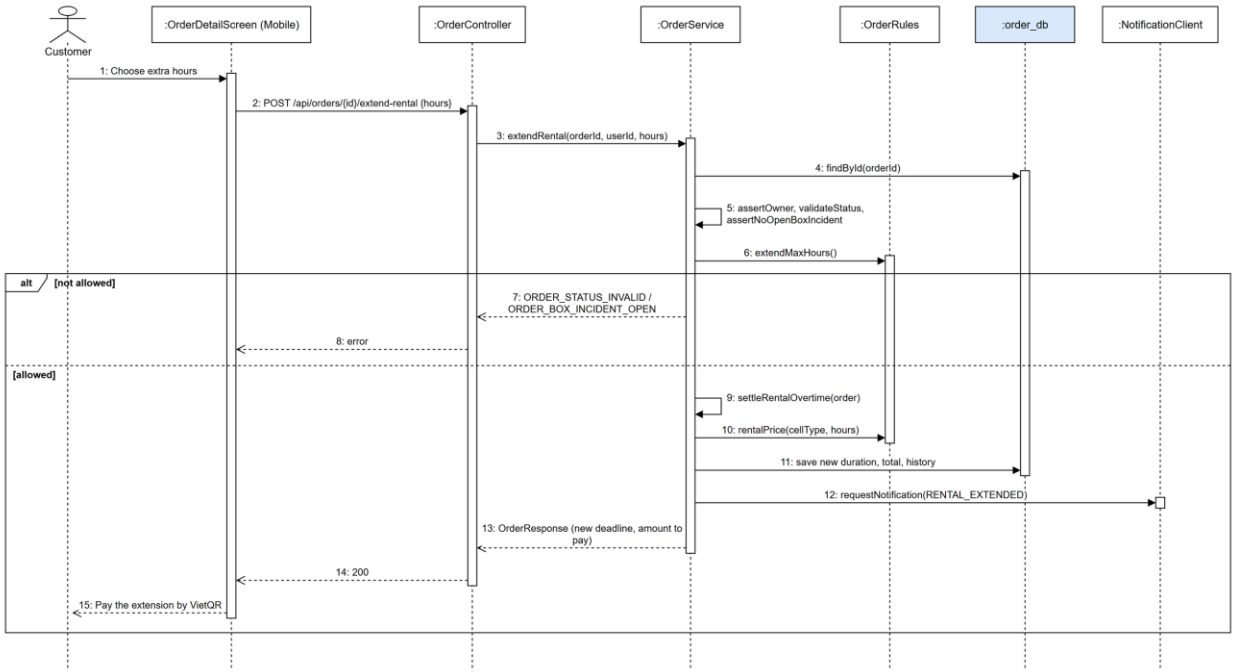


Figure 3.9.3: Sequence diagram - Extend Rental

3.9.4 End Rental at the Kiosk

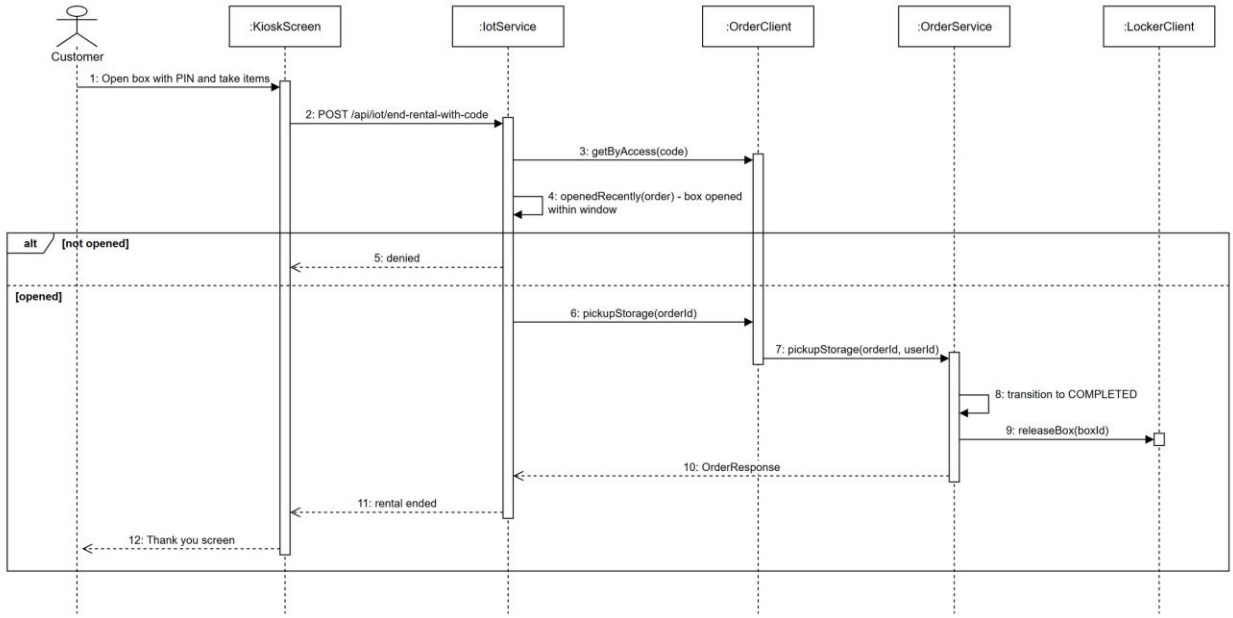


Figure 3.9.4: Sequence diagram - End Rental at the Kiosk

3.9.5 Class Specification

3.9.5.a OrderController

Table 3.9.5.a: OrderController description

No	Method	Description
01	createRental(request: RentalOrderRequest, userId: Long): ApiResponse<OrderResponse>	Creates a RENTAL order.
02	extendRental(orderId: Long, body: Map, userId: Long): ApiResponse<OrderResponse>	Adds hours to an active rental.
03	assessOvertime(orderId: Long, userId: Long): ApiResponse<OrderResponse>	Computes overtime fee before reopening.
04	pickupStorage(orderId: Long, userId: Long): ApiResponse<OrderResponse>	Ends the rental after the items are taken.

3.9.5.b OrderService

Table 3.9.5.b: OrderService description

No	Method	Description
01	createRental(request: RentalOrderRequest, userId: Long): OrderResponse	Rejects DRONE cells, checks min / max hours, prices by cell type and hours and creates the order.
02	extendRental(orderId: Long, userId: Long, hours: int): OrderResponse	Checks owner, status, open incidents and max hours, settles overtime and adds the extension price.
03	assessOvertime(orderId: Long, userId: Long): OrderResponse	Adds the overtime fee that must be paid before unlocking.

No	Method	Description
04	pickupStorage(orderId: Long, userId: Long): OrderResponse	Completes the rental and releases the box.

3.10 Delegate Pickup and Reset PIN

The owner of an order may hand the pickup over to another person (phone, name, note) or request a new PIN. A new PIN is generated, the old one stops working, and the delegate is notified when the phone belongs to an account.

3.10.1 Class Diagram

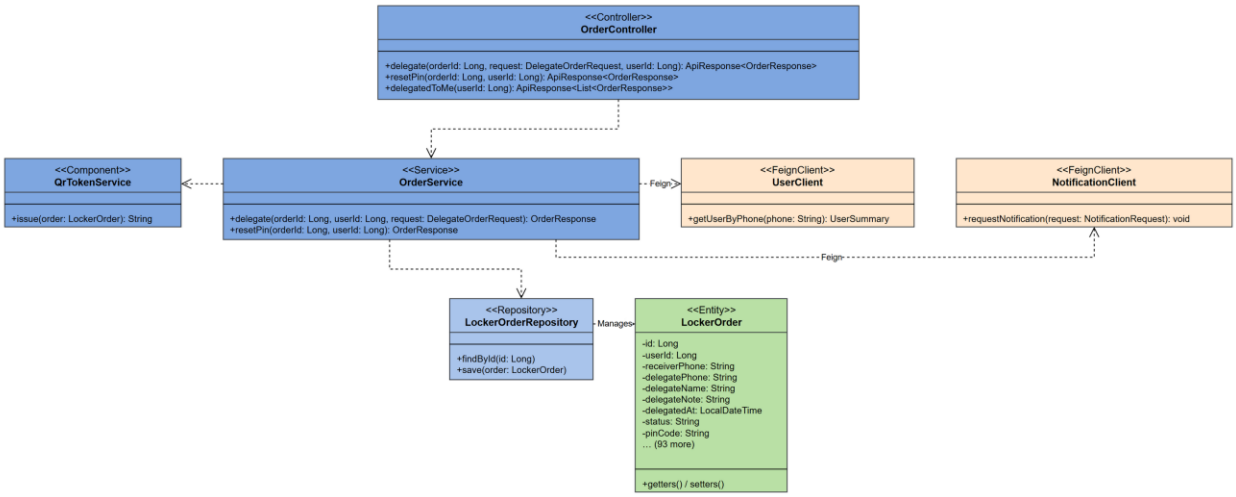


Figure 3.10.1: Class diagram - Delegate Pickup and Reset PIN

3.10.2 Delegate Pickup

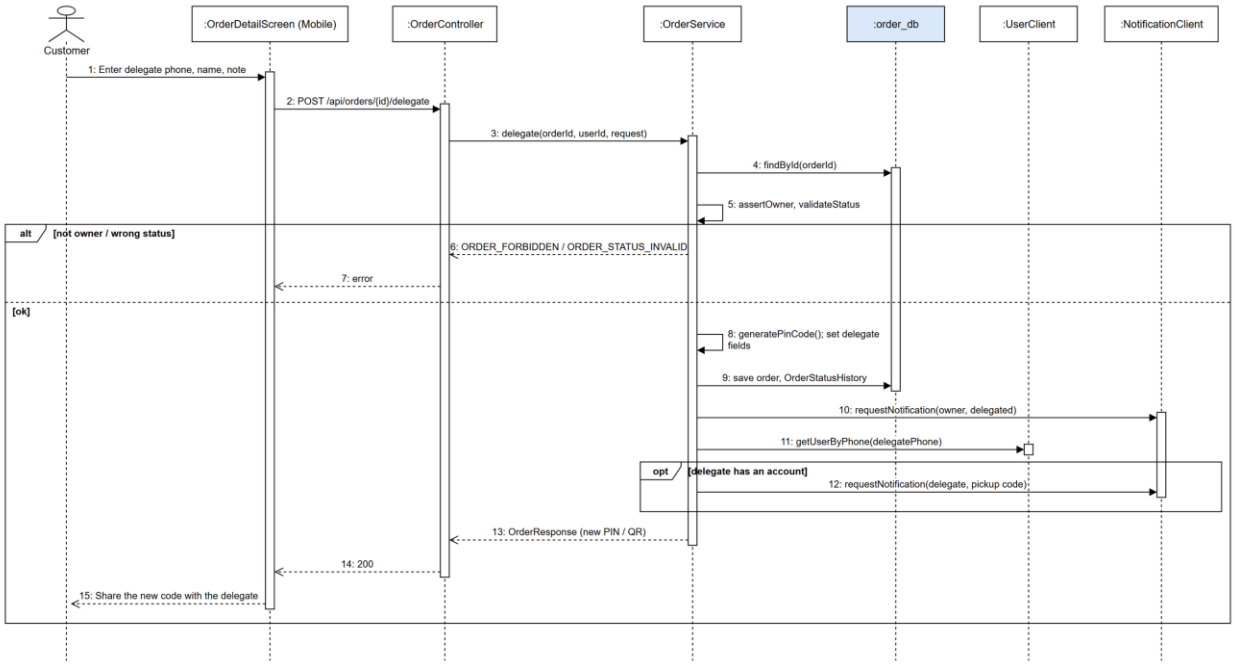


Figure 3.10.2: Sequence diagram - Delegate Pickup

3.10.3 Reset PIN

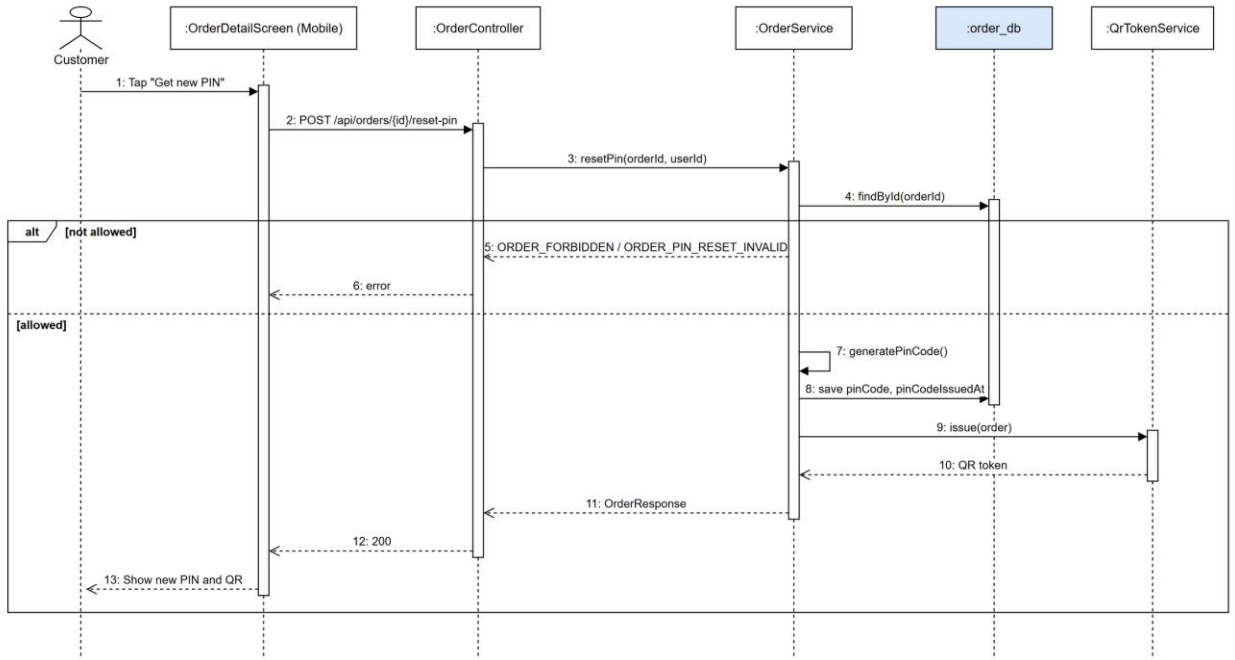


Figure 3.10.3: Sequence diagram - Reset PIN

3.10.4 Class Specification

3.10.4.a OrderController

Table 3.10.4.a: OrderController description

No	Method	Description
01	delegate(orderId: Long, request: DelegateOrderRequest, userId: Long): ApiResponse<OrderResponse>	Delegates pickup to another person.
02	resetPin(orderId: Long, userId: Long): ApiResponse<OrderResponse>	Issues a new PIN for the order.
03	delegatedToMe(userId: Long): ApiResponse<List<OrderResponse>>	Orders delegated to the current user.

3.10.4.b OrderService

Table 3.10.4.b: OrderService description

No	Method	Description
01	delegate(orderId: Long, userId: Long, request: DelegateOrderRequest): OrderResponse	Checks owner and status, stores delegate data, generates a new PIN, writes history and notifies both sides.
02	resetPin(orderId: Long, userId: Long): OrderResponse	Checks owner and status and generates a new PIN.

3.10.4.c QrTokenService

Table 3.10.4.c: QrTokenService description

No	Method	Description
01	issue(order: LockerOrder): String	Signs the QR token that encodes the order access.

3.11 Rate Order, Complaint and Feedback Handling

After an order is completed the customer rates it (1-5 stars, comment) or files a complaint. Administrators read the feedback, see satisfaction analytics and reply; the customer is notified of the reply.

3.11.1 Class Diagram

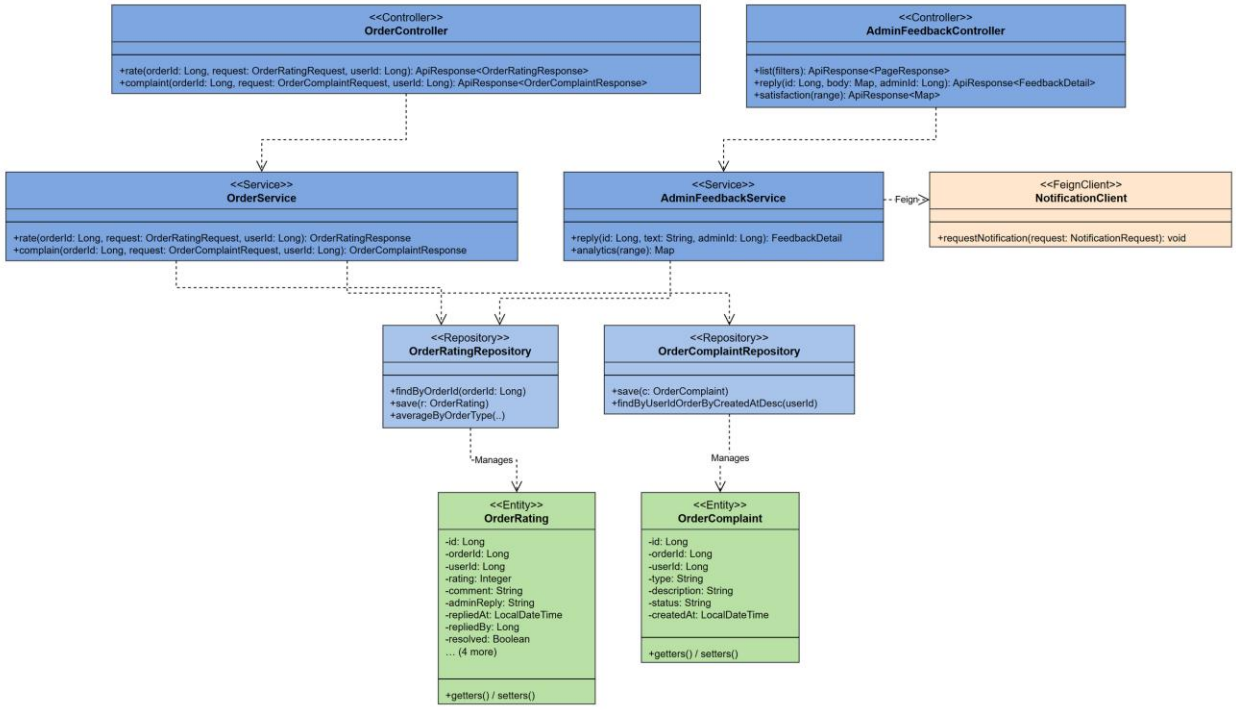


Figure 3.11.1: Class diagram - Rate Order, Complaint and Feedback Handling

3.11.2 Rate an Order

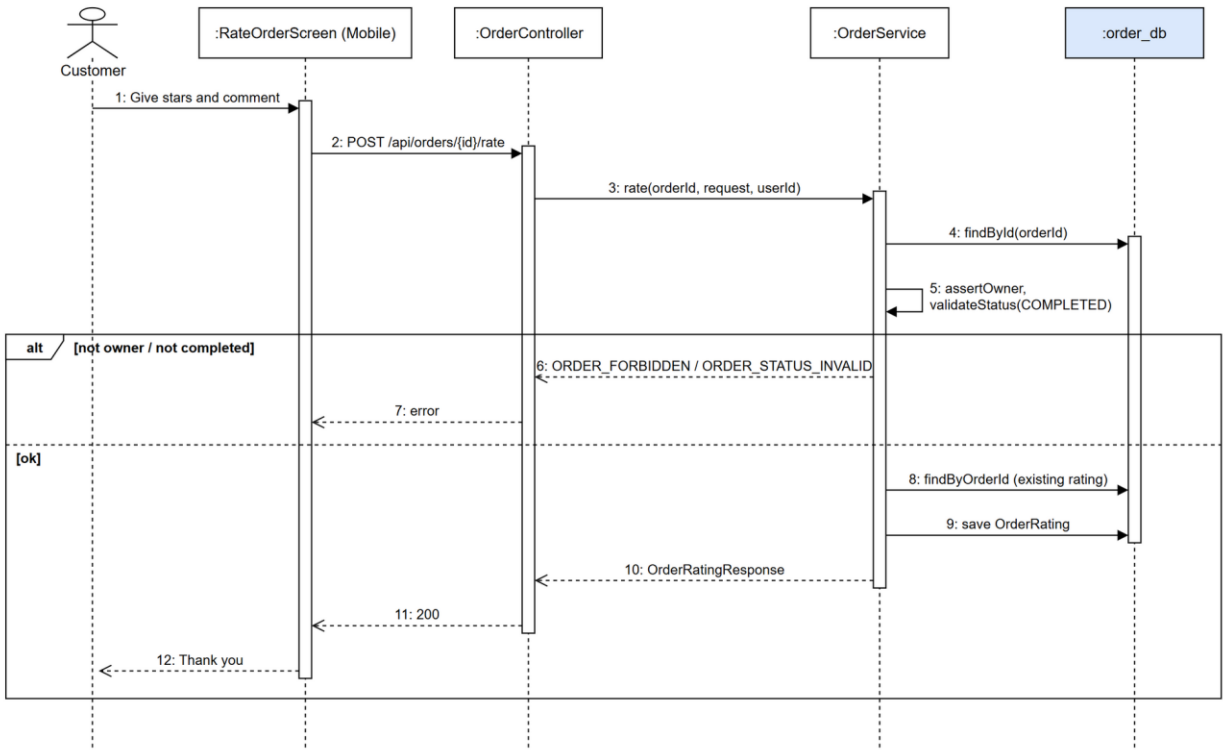


Figure 3.11.2: Sequence diagram - Rate an Order

3.11.3 Administrator Replies to Feedback

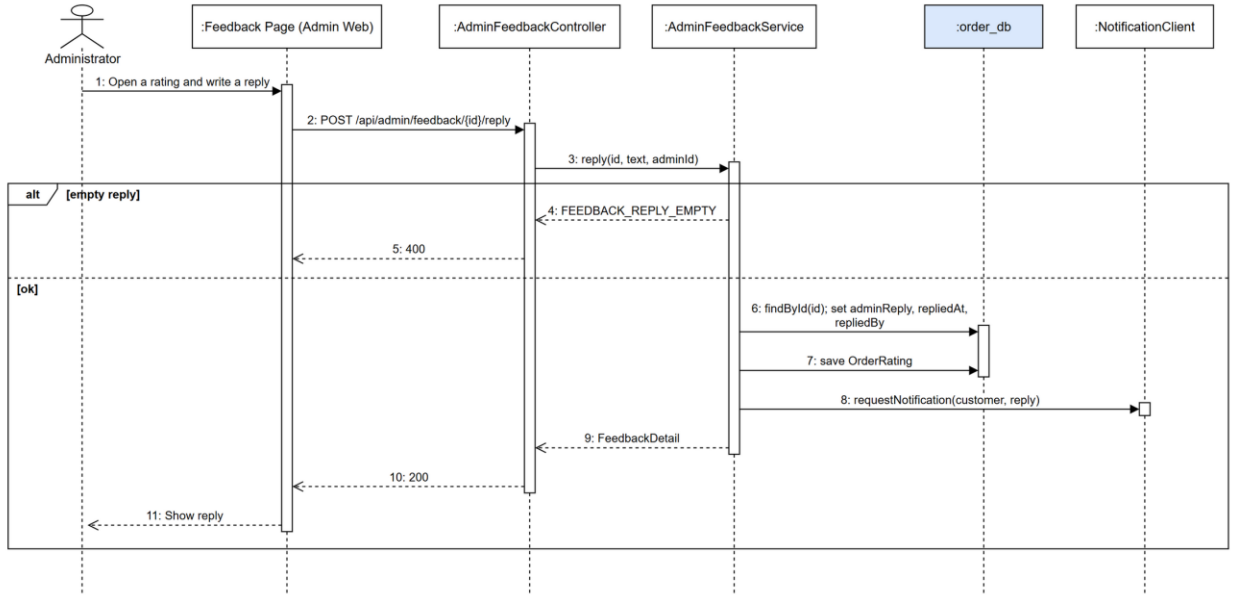


Figure 3.11.3: Sequence diagram - Administrator Replies to Feedback

3.11.4 Class Specification

3.11.4.a OrderController

Table 3.11.4.a: OrderController description

No	Method	Description
01	rate(orderId: Long, request: OrderRatingRequest, userId: Long): ApiResponse<OrderRatingResponse>	Customer rates a completed order.
02	complaint(orderId: Long, request: OrderComplaintRequest, userId: Long): ApiResponse<OrderComplaintResponse>	Customer files a complaint.

3.11.4.b AdminFeedbackController

Table 3.11.4.b: AdminFeedbackController description

No	Method	Description
01	list(filters): ApiResponse<PageResponse>	Admin lists ratings and complaints.
02	reply(id: Long, body: Map, adminId: Long): ApiResponse<FeedbackDetail>	Admin replies to a rating.
03	satisfaction(range): ApiResponse<Map>	Satisfaction analytics.

3.11.4.c OrderService

Table 3.11.4.c: OrderService description

No	Method	Description
01	rate(orderId: Long, request: OrderRatingRequest, userId: Long): OrderRatingResponse	Checks owner and COMPLETED status, creates or updates the rating.
02	complain(orderId: Long, request: OrderComplaintRequest, userId: Long): OrderComplaintResponse	Checks owner and saves the complaint.

3.11.4.d AdminFeedbackService

Table 3.11.4.d: AdminFeedbackService description

No	Method	Description
01	reply(id: Long, text: String, adminId: Long): FeedbackDetail	Saves the admin reply and notifies the customer.
02	analytics(range): Map	Counts ratings, complaint types and averages by order type.

3.12 Wallet Top-up and Withdrawal

Each customer has a Lock.R wallet with a ledger. The wallet is topped up by VietQR (SePay), used to pay orders, credited by refunds, and can be withdrawn to a saved bank account after admin approval. Only real money (top-ups and refunds) can be withdrawn.

3.12.1 Class Diagram

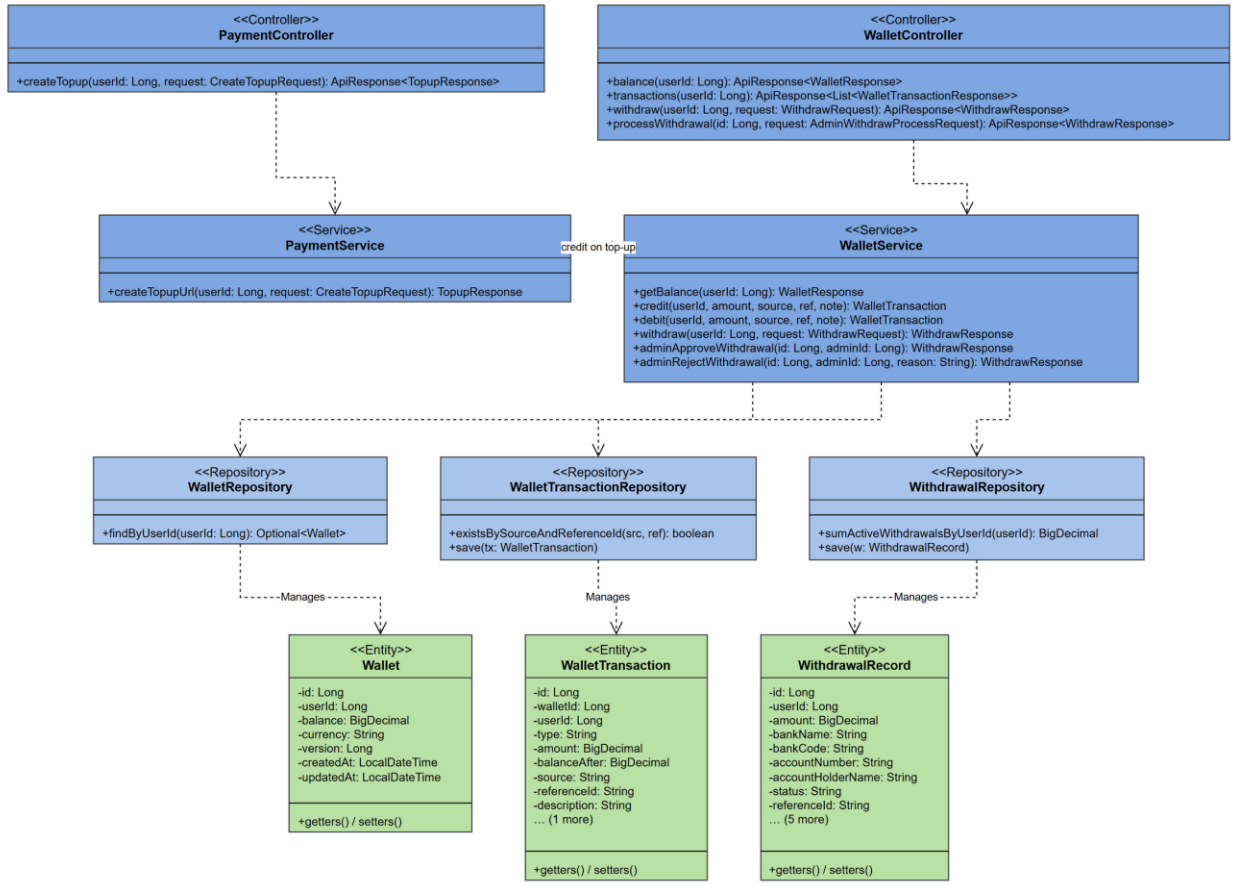


Figure 3.12.1: Class diagram - Wallet Top-up and Withdrawal

3.12.2 Top Up Wallet

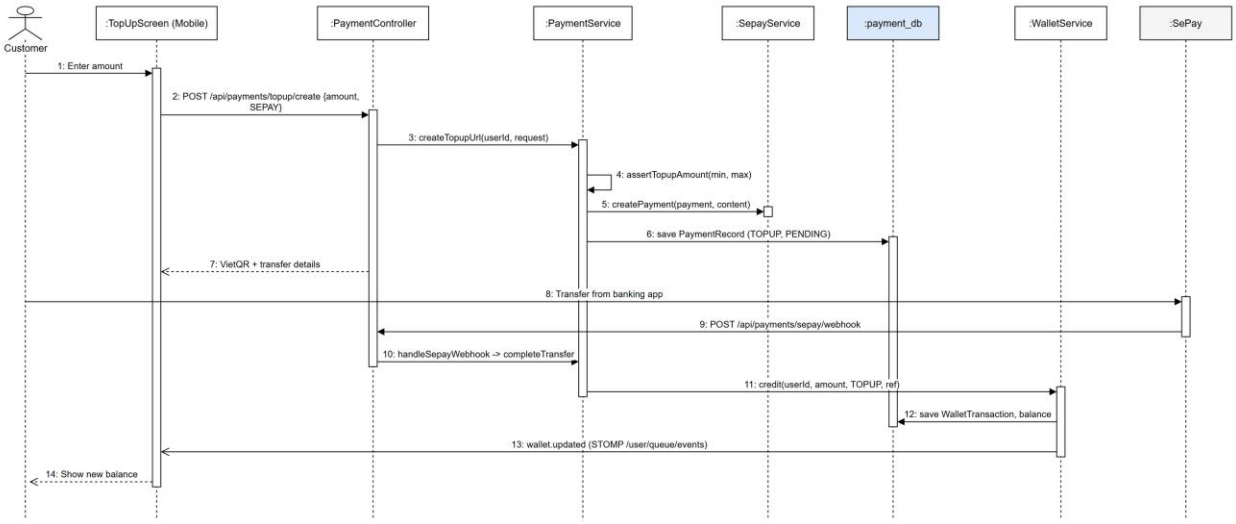


Figure 3.12.2: Sequence diagram - Top Up Wallet

3.12.3 Withdraw to Bank and Admin Approval

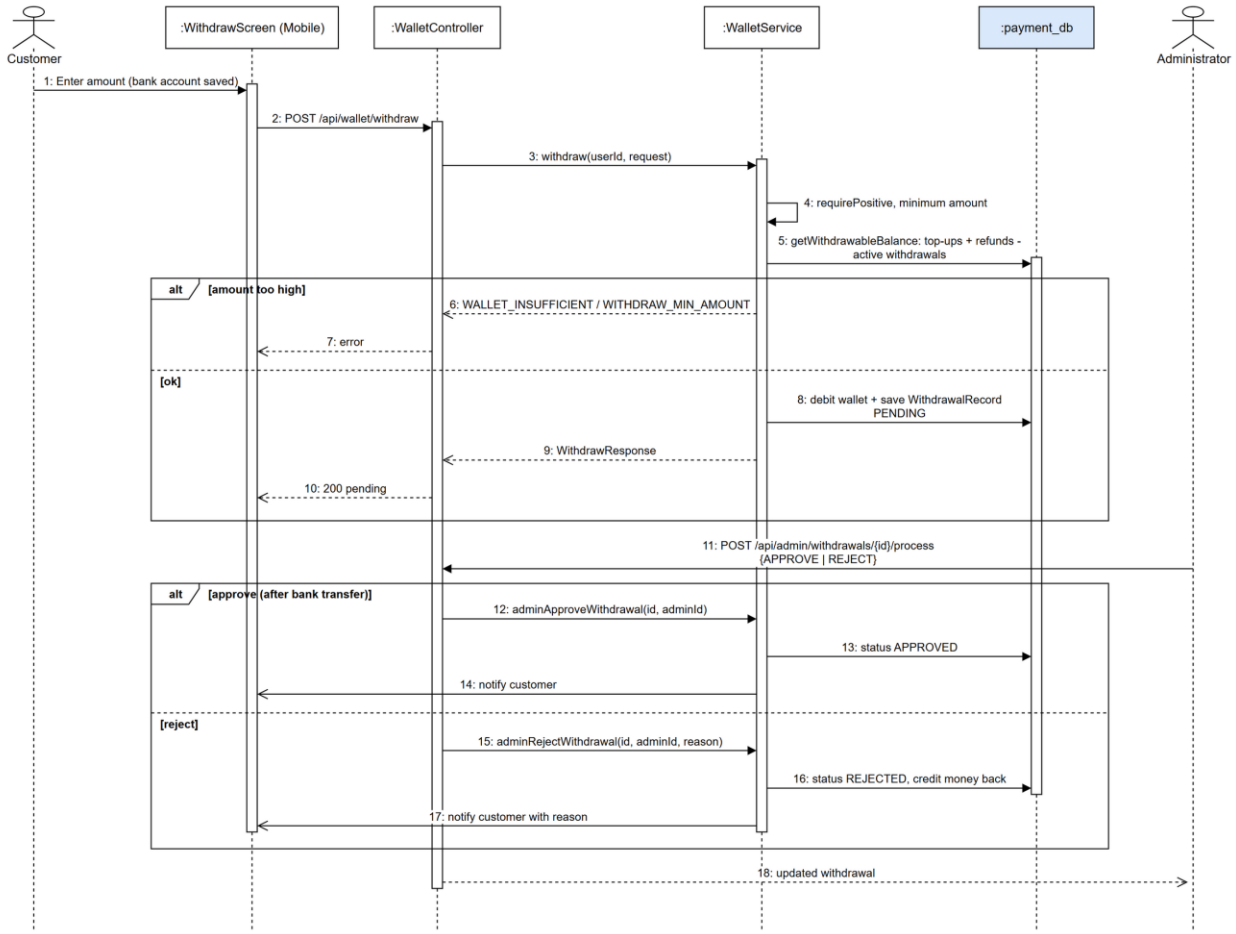


Figure 3.12.3: Sequence diagram - Withdraw to Bank and Admin Approval

3.12.4 Class Specification

3.12.4.a WalletController

Table 3.12.4.a: WalletController description

No	Method	Description
01	balance(userId: Long): ApiResponse<WalletResponse>	Wallet balance.
02	transactions(userId: Long): ApiResponse<List<WalletTransactionResponse>>	Wallet ledger.
03	withdraw(userId: Long, request: WithdrawRequest): ApiResponse<WithdrawResponse>	Requests a withdrawal to the bank account.
04	processWithdrawal(id: Long, request: AdminWithdrawProcessRequest): ApiResponse<WithdrawResponse>	Admin approves / rejects a withdrawal.

3.12.4.b PaymentController

Table 3.12.4.b: PaymentController description

No	Method	Description
01	createTopup(userId: Long, request: CreateTopupRequest): ApiResponse<TopupResponse>	Creates a VietQR top-up payment.

3.12.4.c WalletService

Table 3.12.4.c: WalletService description

No	Method	Description
01	getBalance(userId: Long): WalletResponse	Balance and withdrawable balance.
02	credit(userId, amount, source, ref, note): WalletTransaction	Adds money (idempotent by source + reference) and publishes wallet.updated.
03	debit(userId, amount, source, ref, note): WalletTransaction	Removes money if the balance is enough.
04	withdraw(userId: Long, request: WithdrawRequest): WithdrawResponse	Checks minimum and withdrawable amount, debits the wallet and saves a PENDING withdrawal.
05	adminApproveWithdrawal(id: Long, adminId: Long): WithdrawResponse	Marks the withdrawal APPROVED after the bank transfer and notifies.
06	adminRejectWithdrawal(id: Long, adminId: Long, reason: String): WithdrawResponse	Rejects and credits the money back.

3.13 Refund Requests

A customer asks for a refund of a paid order within the allowed days, to a saved bank account. The order is marked REFUND_PENDING; an administrator transfers the money, enters the bank transfer reference and approves (or rejects). payment.refunded lets loyalty-service take back the points earned.

3.13.1 Class Diagram

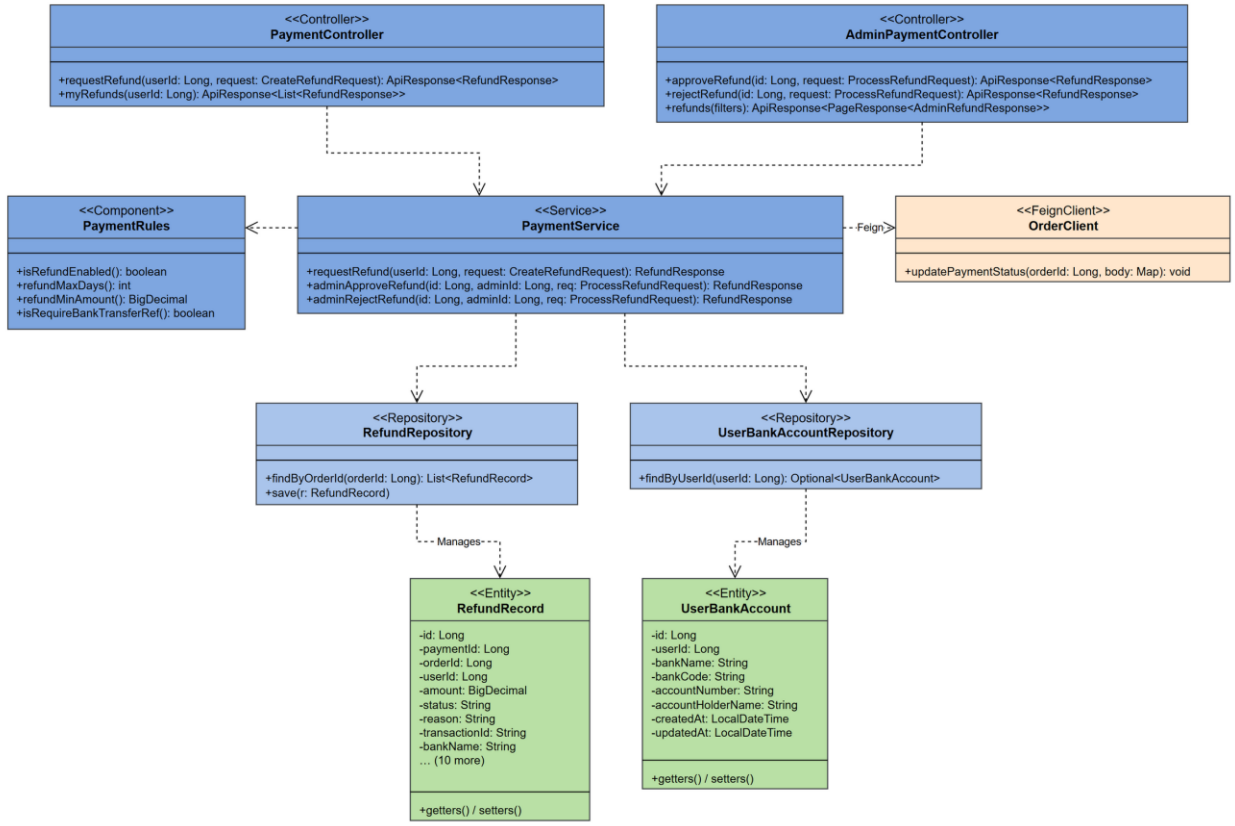


Figure 3.13.1: Class diagram - Refund Requests

3.13.2 Request and Approve a Refund

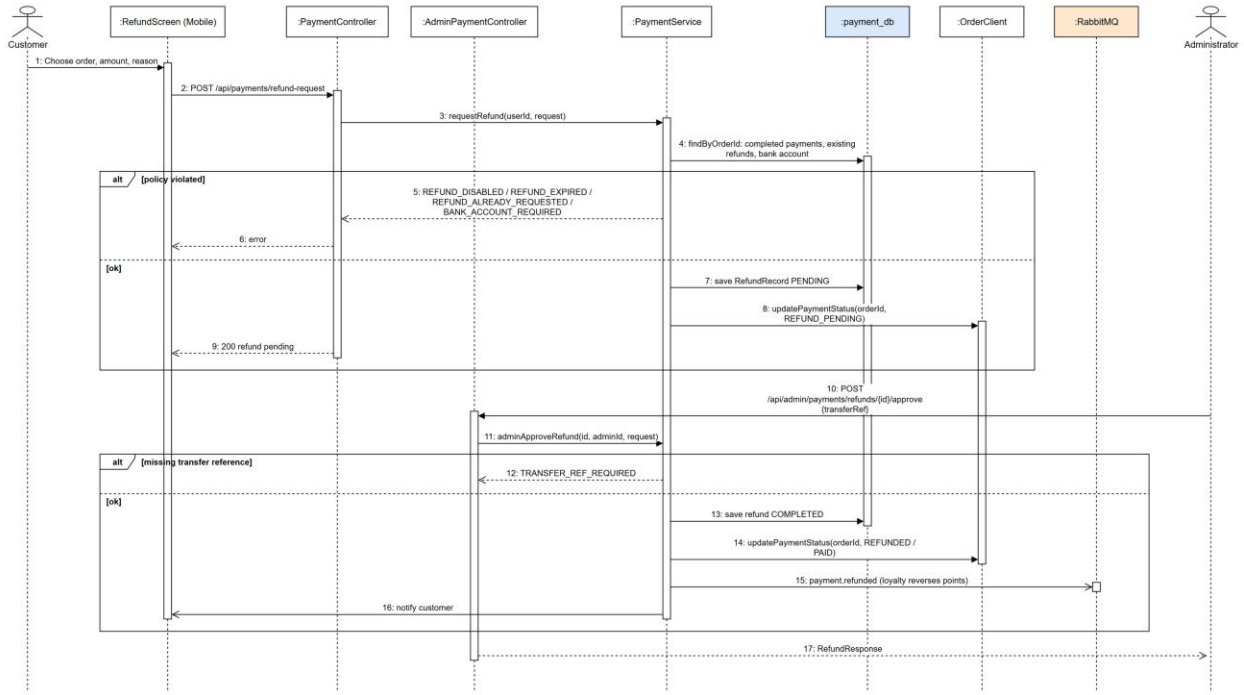


Figure 3.13.2: Sequence diagram - Request and Approve a Refund

3.13.3 Class Specification

3.13.3.a PaymentController

Table 3.13.3.a: PaymentController description

No	Method	Description
01	requestRefund(userId: Long, request: CreateRefundRequest): ApiResponse<RefundResponse>	Customer requests a refund.
02	myRefunds(userId: Long): ApiResponse<List<RefundResponse>>	Refunds of the customer.

3.13.3.b AdminPaymentController

Table 3.13.3.b: AdminPaymentController description

No	Method	Description
01	approveRefund(id: Long, request: ProcessRefundRequest): ApiResponse<RefundResponse>	Admin approves a refund after the bank transfer.
02	rejectRefund(id: Long, request: ProcessRefundRequest): ApiResponse<RefundResponse>	Admin rejects a refund with a reason.
03	refunds(filters): ApiResponse<PageResponse<AdminRefundResponse>>	Admin lists refund requests.

3.13.3.c PaymentService

Table 3.13.3.c: PaymentService description

No	Method	Description
01	requestRefund(userId: Long, request: CreateRefundRequest): RefundResponse	Checks the refund policy (enabled, max days, amount), bank account and duplicates, saves the refund and marks the order REFUND_PENDING.
02	adminApproveRefund(id: Long, adminId: Long, req: ProcessRefundRequest): RefundResponse	Requires the transfer reference, completes the refund, syncs the order and publishes payment.refunded.
03	adminRejectRefund(id: Long, adminId: Long, req: ProcessRefundRequest): RefundResponse	Rejects with a reason and restores the order payment status.

3.14 Promotions, Vouchers and Loyalty Points

Administrators define promotions (code, discount, period, limits, locker scope, point price). Customers save free vouchers or exchange loyalty points for vouchers and apply them when ordering. loyalty-service awards 10 % of the paid amount as points when order.completed is received and reverses them on refunds.

3.14.1 Class Diagram

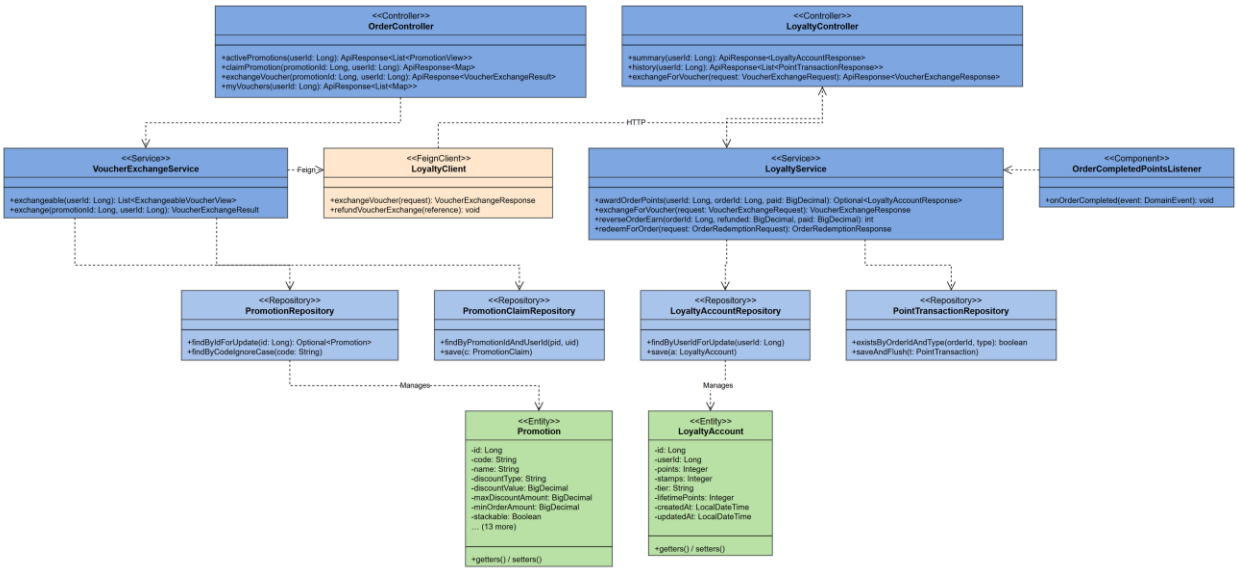


Figure 3.14.1: Class diagram - Promotions, Vouchers and Loyalty Points

3.14.2 Exchange Points for a Voucher

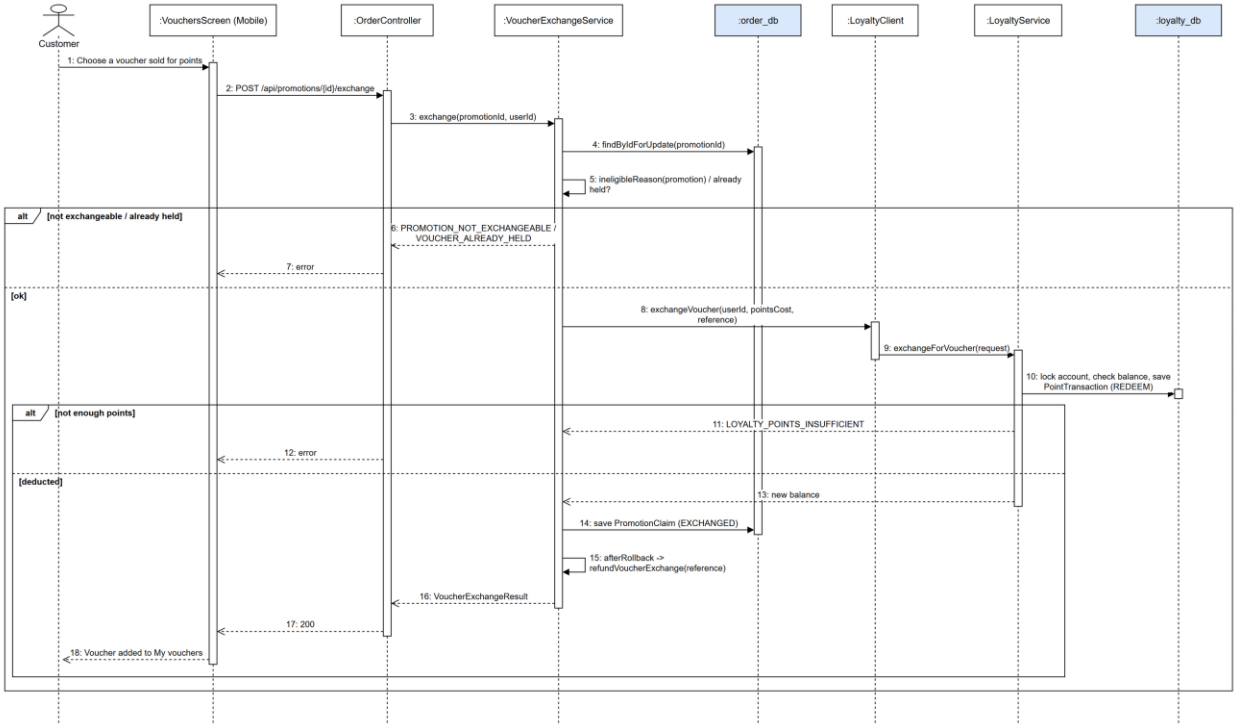


Figure 3.14.2: Sequence diagram - Exchange Points for a Voucher

3.14.3 Earn Points when an Order Completes

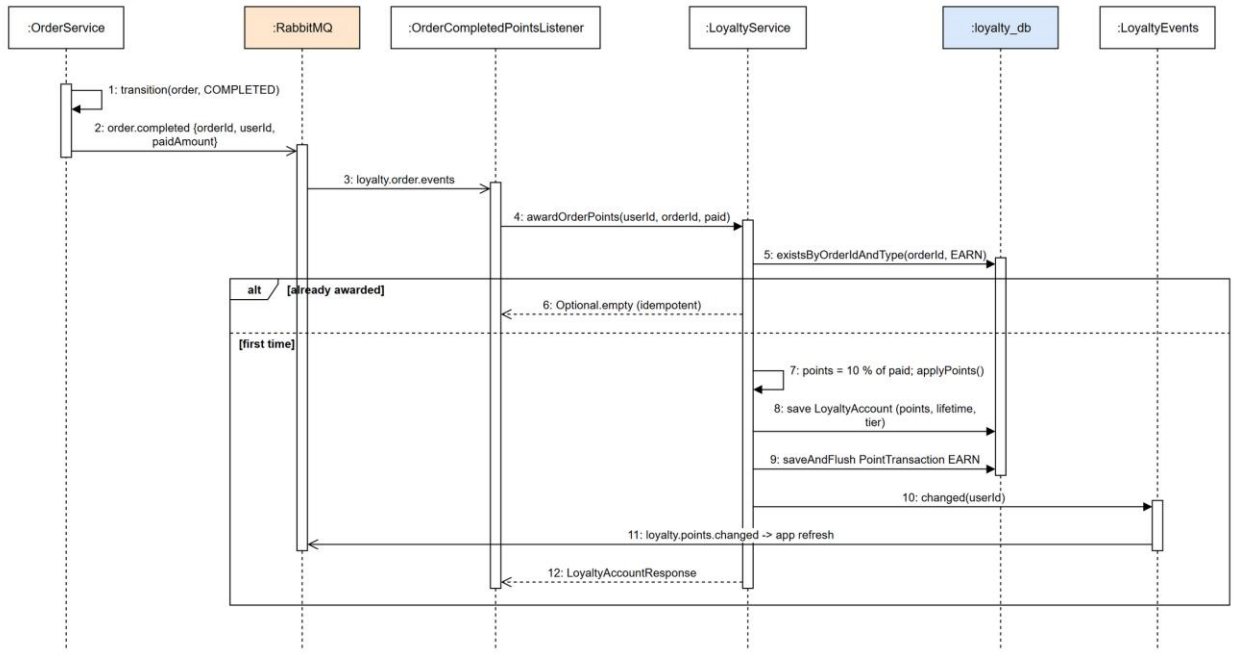


Figure 3.14.3: Sequence diagram - Earn Points when an Order Completes

3.14.4 Class Specification

3.14.4.a OrderController

Table 3.14.4.a: OrderController description

No	Method	Description
01	activePromotions(userId: Long): ApiResponse<List<PromotionView>>	Active promotions for the customer.
02	claimPromotion(promotionId: Long, userId: Long): ApiResponse<Map>	Saves a free voucher.
03	exchangeVoucher(promotionId: Long, userId: Long): ApiResponse<VoucherExchangeResult>	Exchanges points for a voucher.
04	myVouchers(userId: Long): ApiResponse<List<Map>>	Vouchers held by the customer.

3.14.4.b VoucherExchangeService

Table 3.14.4.b: VoucherExchangeService description

No	Method	Description
01	exchangeable(userId: Long): List<ExchangeableVoucherView>	Promotions that can be bought with points.
02	exchange(promotionId: Long, userId: Long): VoucherExchangeResult	Locks the promotion, checks eligibility, spends points in loyalty-service and saves the claim (refunds points on rollback).

3.14.4.c LoyaltyController

Table 3.14.4.c: LoyaltyController description

No	Method	Description
01	summary(userId: Long): ApiResponse<LoyaltyAccountResponse>	Points, lifetime points and tier.
02	history(userId: Long): ApiResponse<List<PointTransactionResponse>>	Point transactions.
03	exchangeForVoucher(request: VoucherExchangeRequest): ApiResponse<VoucherExchangeResponse>	Internal: spends points for a voucher.

3.14.4.d LoyaltyService

Table 3.14.4.d: LoyaltyService description

No	Method	Description
01	awardOrderPoints(userId: Long, orderId: Long, paid: BigDecimal): Optional<LoyaltyAccountResponse>	Adds earn points once per order and updates the tier.
02	exchangeForVoucher(request: VoucherExchangeRequest): VoucherExchangeResponse	Deducts points with a reference for idempotency.

No	Method	Description
03	reverseOrderEarn(orderId: Long, refunded: BigDecimal, paid: BigDecimal): int	Takes back points proportionally to a refund.
04	redeemForOrder(request: OrderRedemptionRequest): OrderRedemptionResponse	Uses points as an order discount.

3.14.4.e OrderCompletedPointsListener

Table 3.14.4.e: OrderCompletedPointsListener description

No	Method	Description
01	onOrderCompleted(event: DomainEvent): void	Consumes order.completed / payment.refunded and calls LoyaltyService.

3.15 Drone Delivery Booking

A customer books a drone delivery between two lockers with a landing pad or from a pickup address to a locker. order-service quotes the price, checks flight availability, open-order limit, parcel weight and weather, selects the source locker, reserves a DRONE box at the destination and creates a DRONE_DELIVERY order (idempotent by key).

3.15.1 Class Diagram

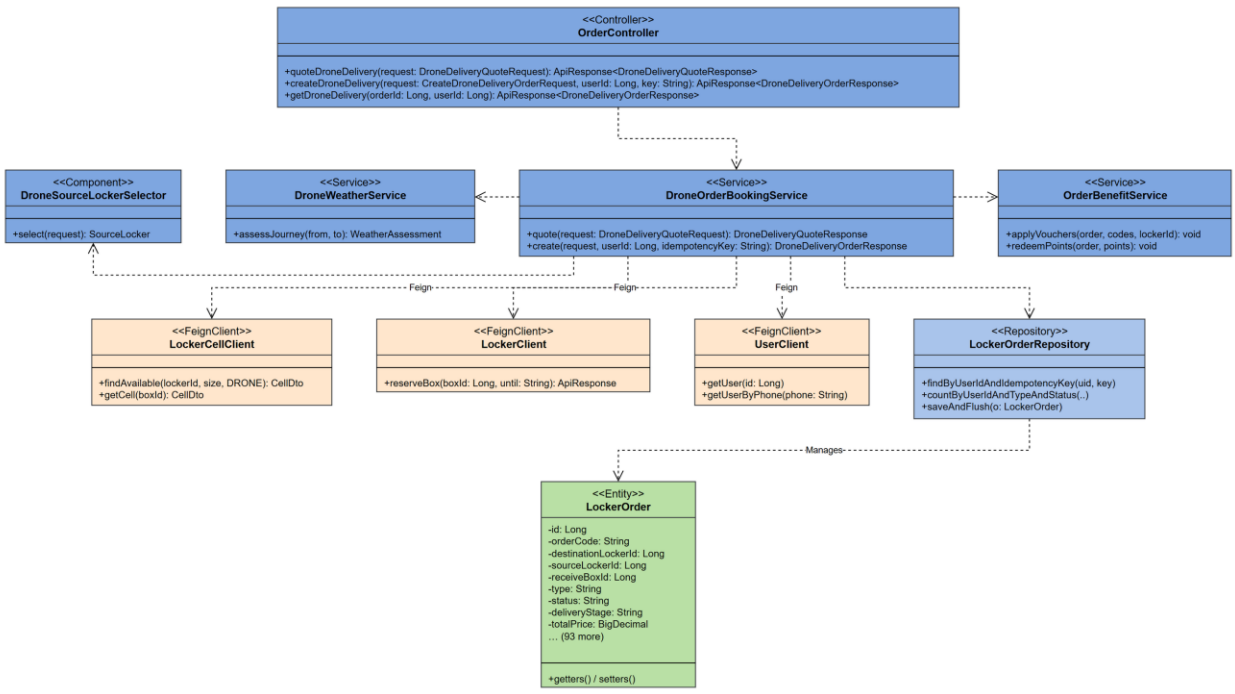


Figure 3.15.1: Class diagram - Drone Delivery Booking

3.15.2 Book a Drone Delivery

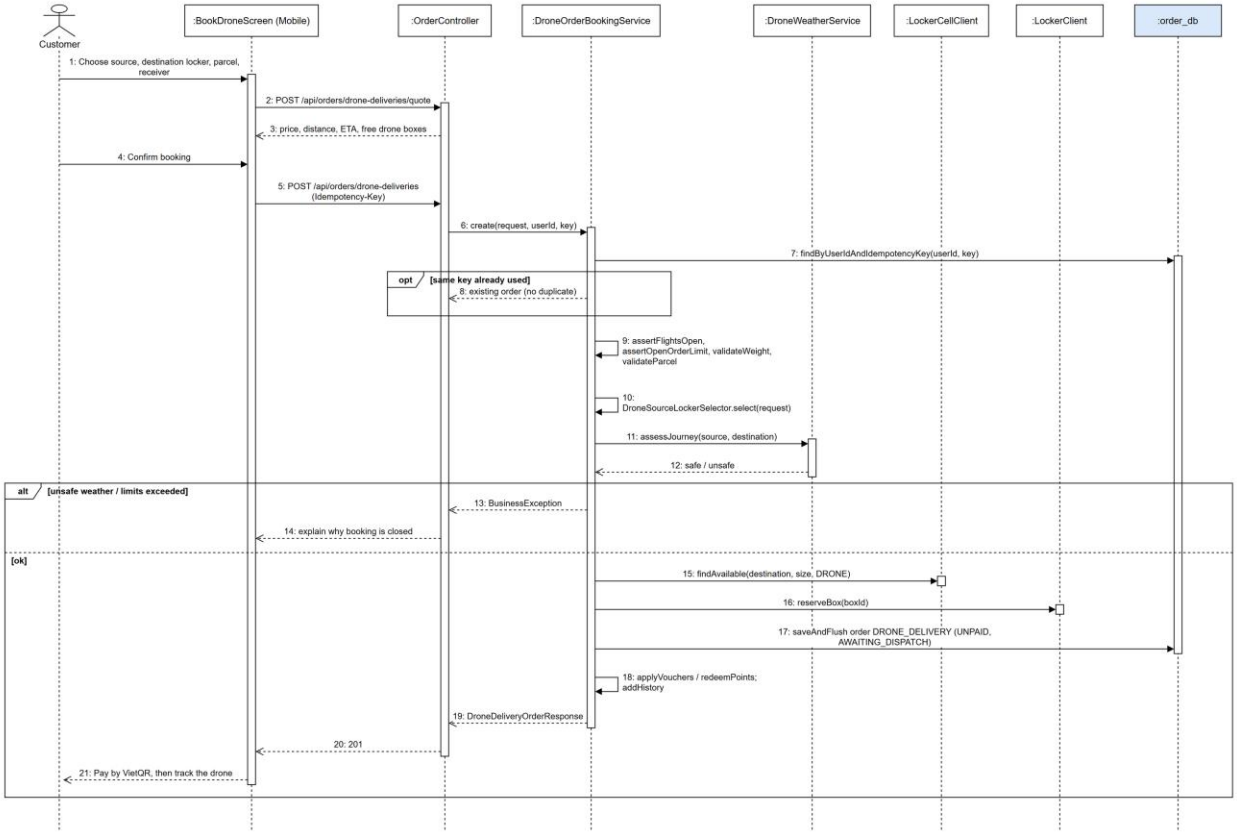


Figure 3.15.2: Sequence diagram - Book a Drone Delivery

3.15.3 Class Specification

3.15.3.a OrderController

Table 3.15.3.a: OrderController description

No	Method	Description
01	quoteDroneDelivery(request: DroneDeliveryQuoteRequest): ApiResponse<DroneDeliveryQuoteResponse>	Price, distance and ETA before booking.
02	createDroneDelivery(request: CreateDroneDeliveryOrderRequest, userId: Long, key: String): ApiResponse<DroneDeliveryOrderResponse>	Books a drone delivery.
03	getDroneDelivery(orderId: Long, userId: Long): ApiResponse<DroneDeliveryOrderResponse>	Booking and flight status.

3.15.3.b DroneOrderBookingService

Table 3.15.3.b: DroneOrderBookingService description

No	Method	Description
01	quote(request: DroneDeliveryQuoteRequest): DroneDeliveryQuoteResponse	Computes distance, price and availability.
02	create(request, userId: Long, idempotencyKey: String): DroneDeliveryOrderResponse	Validates limits, weight, parcel and weather, selects the source locker, reserves the destination box and saves the order.

3.15.3.c DroneSourceLockerSelector

Table 3.15.3.c: DroneSourceLockerSelector description

No	Method	Description
01	select(request): SourceLocker	Chooses the source locker / station for the flight.

3.15.3.d DroneWeatherService

Table 3.15.3.d: DroneWeatherService description

No	Method	Description
01	assessJourney(from, to): WeatherAssessment	Wind / rain check from Open-Meteo (fallback MET Norway).

3.16 Drone Home Pickup

When a drone order that starts at the customer's address is paid, a pickup is opened and offered to the nearest on-duty drone technician who is not busy. The technician accepts (or declines, then the next one is offered), meets the customer, checks the pickup code and takes a photo; the parcel is then ready to load.

3.16.1 Class Diagram

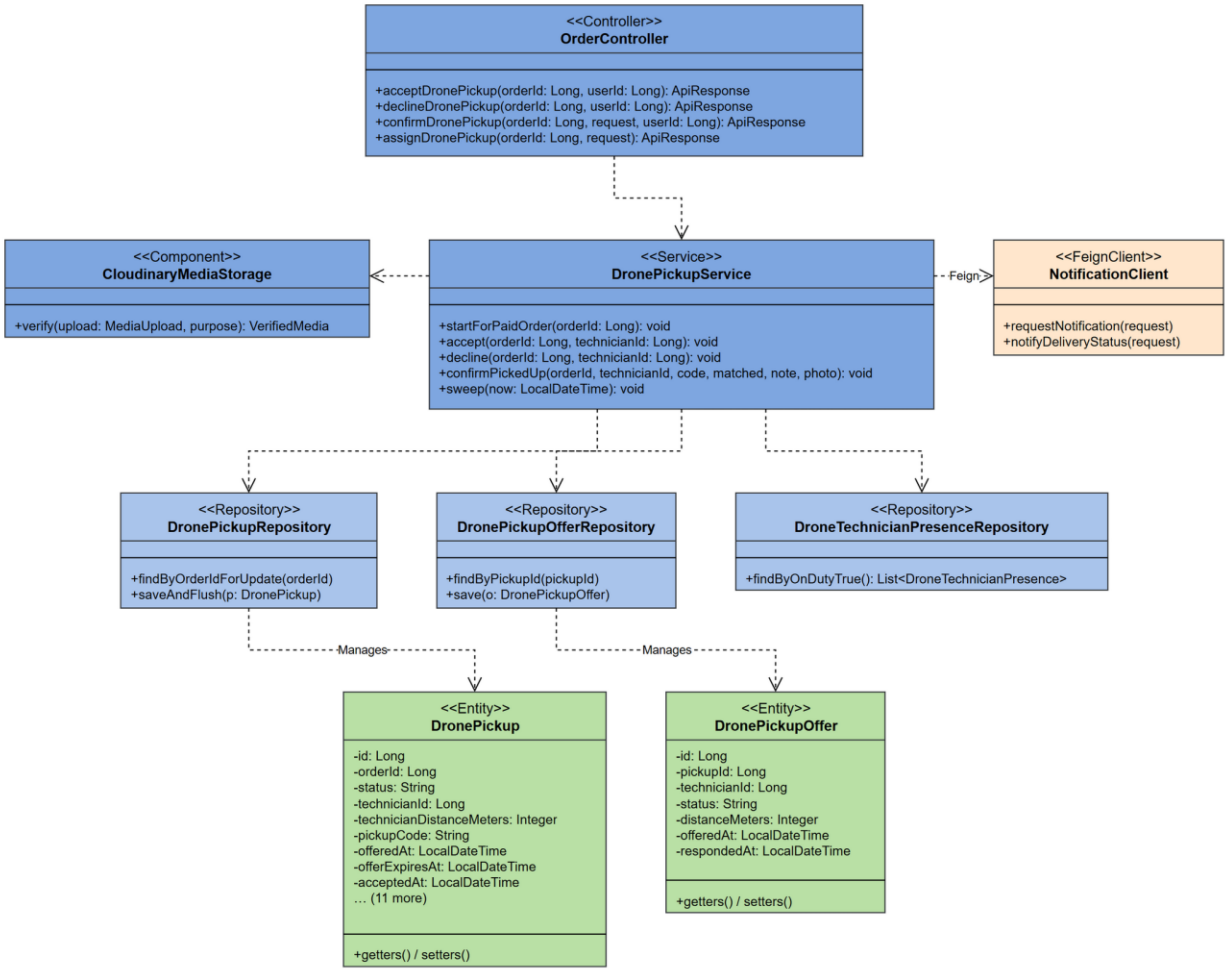


Figure 3.16.1: Class diagram - Drone Home Pickup

3.16.2 Offer, Accept and Confirm a Home Pickup

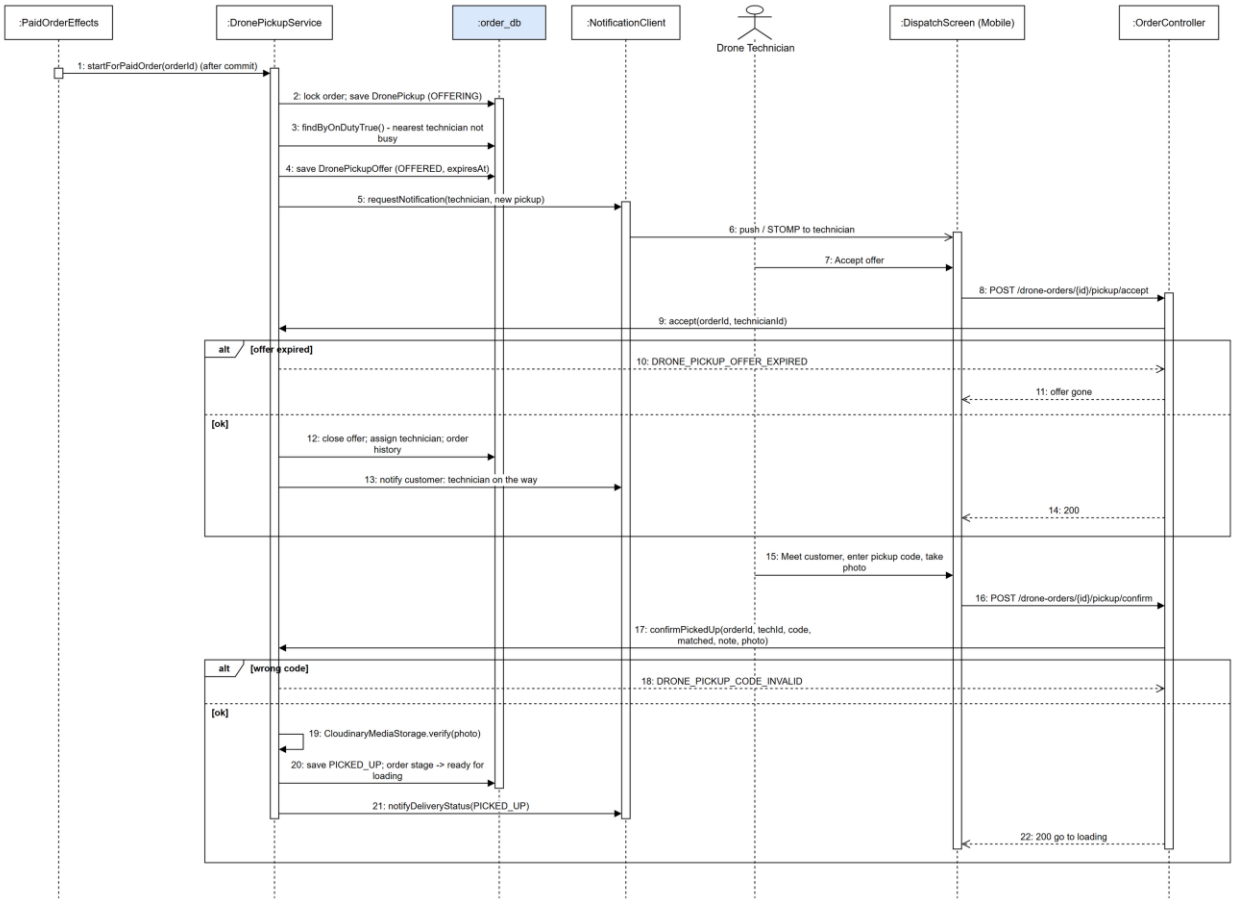


Figure 3.16.2: Sequence diagram - Offer, Accept and Confirm a Home Pickup

3.16.3 Class Specification

3.16.3.a OrderController

Table 3.16.3.a: OrderController description

No	Method	Description
01	acceptDronePickup(orderId: Long, userId: Long): ApiResponse	Technician accepts the pickup offer.
02	declineDronePickup(orderId: Long, userId: Long): ApiResponse	Technician declines the offer.
03	confirmDronePickup(orderId: Long, request, userId: Long): ApiResponse	Technician confirms the parcel was collected.
04	assignDronePickup(orderId: Long, request): ApiResponse	Admin assigns a technician manually.

3.16.3.b DronePickupService

Table 3.16.3.b: DronePickupService description

No	Method	Description
01	startForPaidOrder(orderId: Long): void	Creates the pickup and offers it to the next free technician.
02	accept(orderId: Long, technicianId: Long): void	Closes the offer and assigns the technician to the order.
03	decline(orderId: Long, technicianId: Long): void	Declines and offers the pickup to the next technician.
04	confirmPickedUp(orderId, technicianId, code, matched, note, photo): void	Checks the pickup code, verifies the photo and moves the order to the loading step.
05	sweep(now: LocalDateTime): void	Scheduled: expires unanswered offers and re-offers.

3.17 Drone Dispatch, Loading and Launch

A drone technician accepts a paid drone order and reserves a drone that passes the pre-flight checks (status, battery, landing pad and destination box). The parcel is weighed and sealed in the loading step (surcharge when heavier than declared), then the drone is launched after a weather check. The drone status is changed in locker-service.

3.17.1 Class Diagram

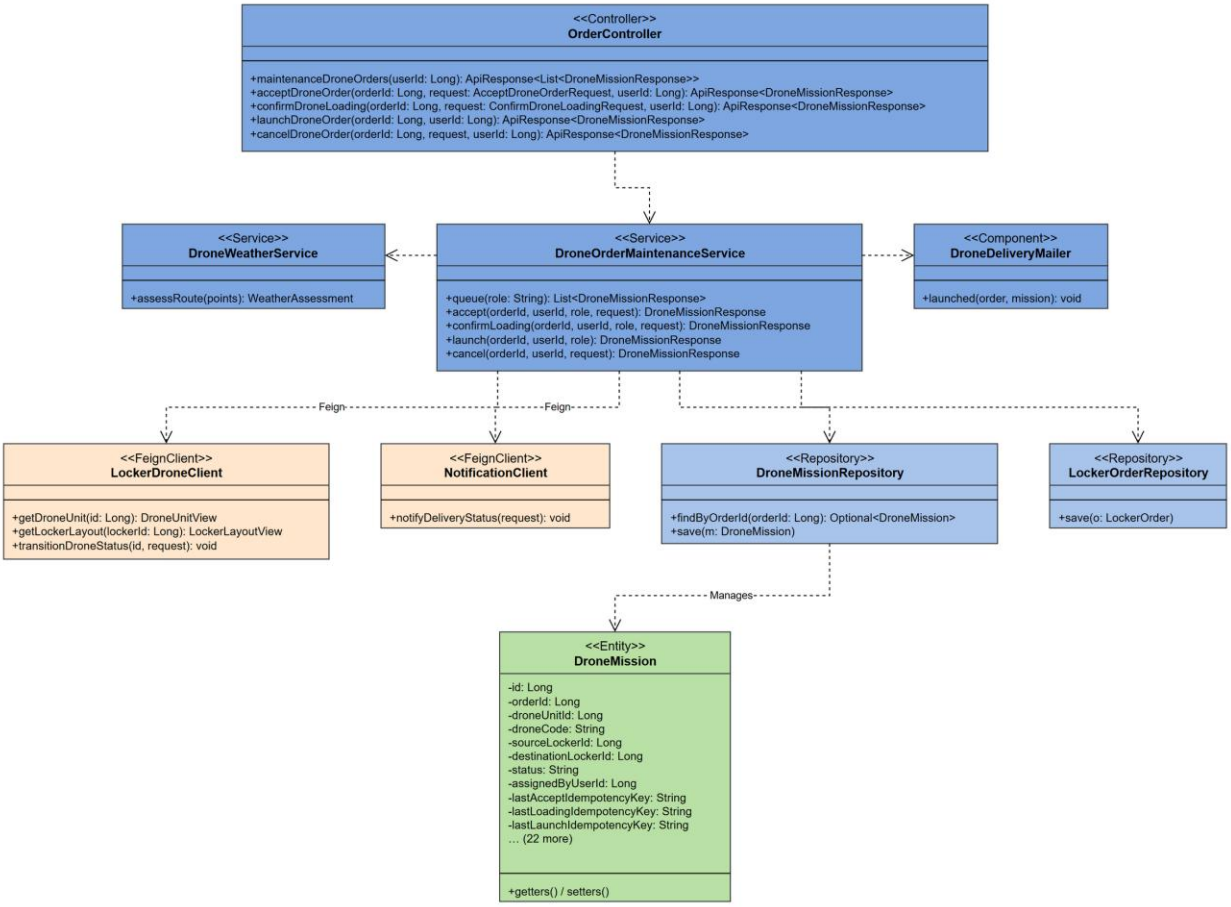


Figure 3.17.1: Class diagram - Drone Dispatch, Loading and Launch

3.17.2 Accept Drone Order and Confirm Loading

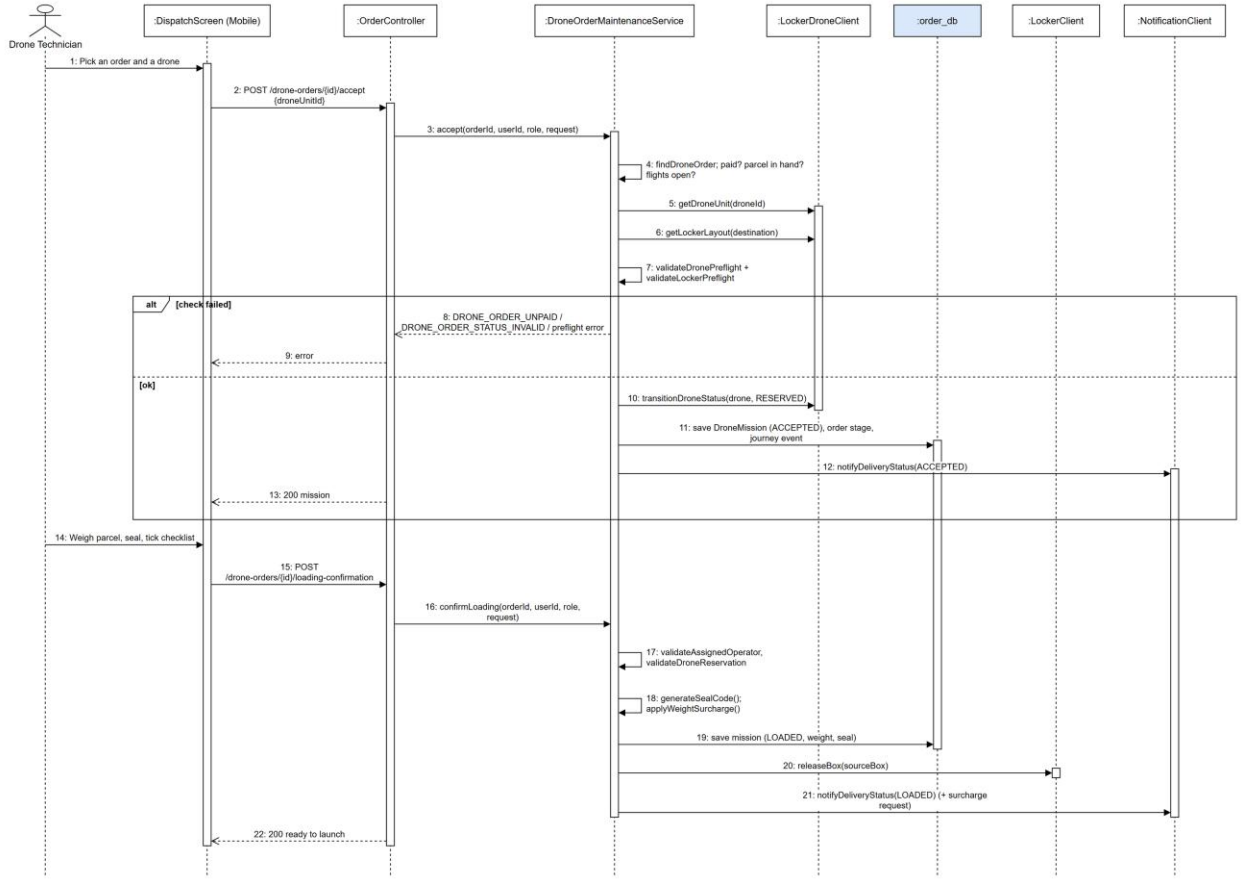


Figure 3.17.2: Sequence diagram - Accept Drone Order and Confirm Loading

3.17.3 Launch the Drone

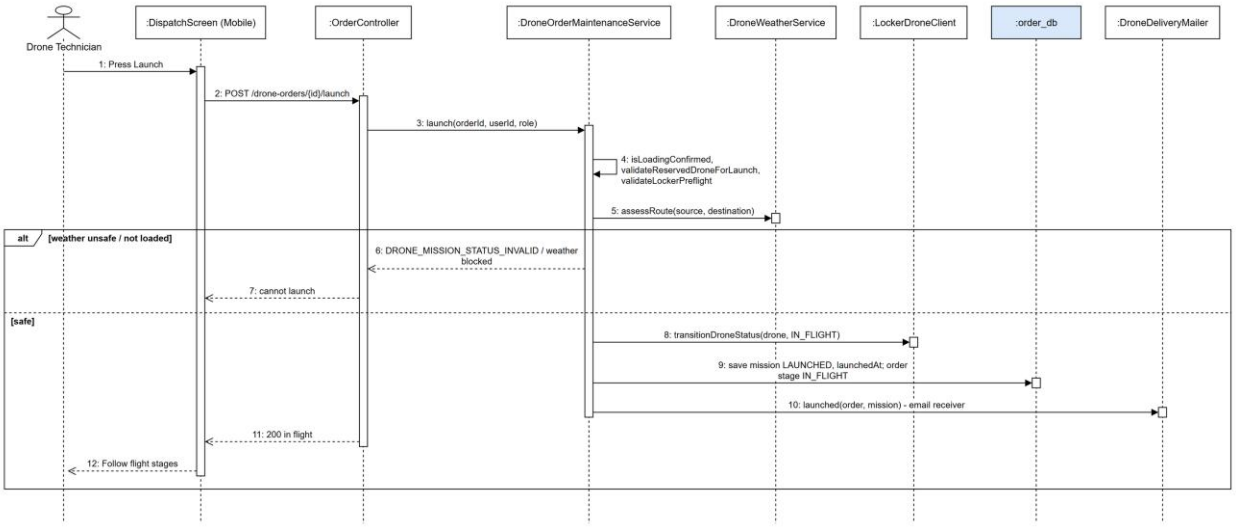


Figure 3.17.3: Sequence diagram - Launch the Drone

3.17.4 Class Specification

3.17.4.a OrderController

Table 3.17.4.a: OrderController description

No	Method	Description
01	maintenanceDroneOrders(userId: Long): ApiResponse<List<DroneMissionResponse>>	Dispatch queue of drone orders.
02	acceptDroneOrder(orderId: Long, request: AcceptDroneOrderRequest, userId: Long): ApiResponse<DroneMissionResponse>	Accepts an order with a drone.
03	confirmDroneLoading(orderId: Long, request: ConfirmDroneLoadingRequest, userId: Long): ApiResponse<DroneMissionResponse>	Confirms weight, seal and checklist.
04	launchDroneOrder(orderId: Long, userId: Long): ApiResponse<DroneMissionResponse>	Launches the flight.
05	cancelDroneOrder(orderId: Long, request, userId: Long): ApiResponse<DroneMissionResponse>	Cancels before launch.

3.17.4.b DroneOrderMaintenanceService

Table 3.17.4.b: DroneOrderMaintenanceService description

No	Method	Description
01	queue(role: String): List<DroneMissionResponse>	Orders waiting for dispatch / in progress.
02	accept(orderId, userId, role, request): DroneMissionResponse	Checks paid status, parcel in hand, flights open, drone and locker pre-flight; reserves the drone and creates the mission.
03	confirmLoading(orderId, userId, role, request): DroneMissionResponse	Checks operator and reservation, generates the seal code, applies a weight surcharge and releases the source box.
04	launch(orderId, userId, role): DroneMissionResponse	Re-checks pre-flight and weather, sets the drone IN_FLIGHT and the mission LAUNCHED, emails / notifies.
05	cancel(orderId, userId, request): DroneMissionResponse	Cancels the mission and frees the drone.

3.18 Drone Flight Tracking and Stage Progress

During the flight the dispatcher confirms the STANDARD stages (or the simulator advances DEMO orders) and the drone companion Pi sends HMAC-signed telemetry over MQTT. order-service updates the mission, writes the journey and broadcasts the live position to /topic/deliveries/{orderId}/position.

3.18.1 Class Diagram

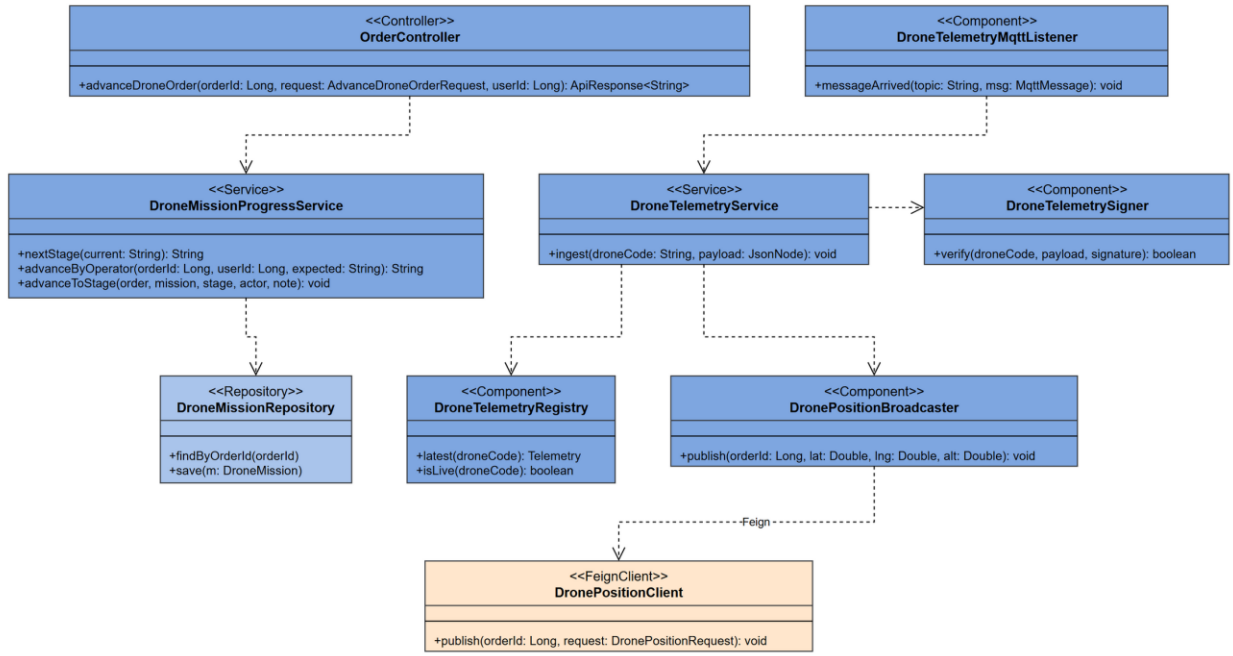


Figure 3.18.1: Class diagram - Drone Flight Tracking and Stage Progress

3.18.2 Advance Flight Stage and Live Position

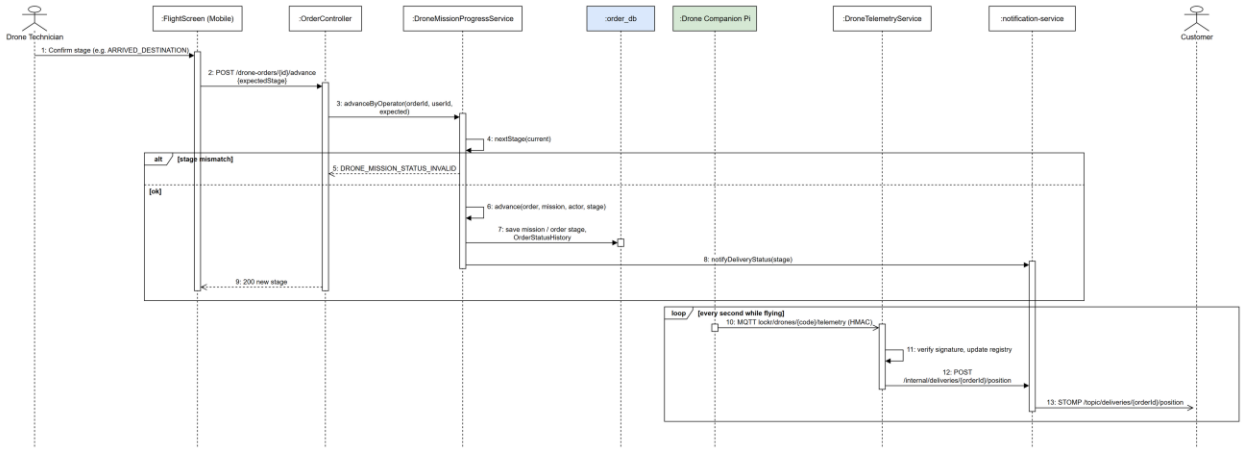


Figure 3.18.2: Sequence diagram - Advance Flight Stage and Live Position

3.18.3 Class Specification

3.18.3.a OrderController

Table 3.18.3.a: OrderController description

No	Method	Description
01	advanceDroneOrder(orderId: Long, request: AdvanceDroneOrderRequest, userId: Long): ApiResponse<String>	Dispatcher confirms the next flight stage.

3.18.3.b DroneMissionProgressService

Table 3.18.3.b: DroneMissionProgressService description

No	Method	Description
01	nextStage(current: String): String	Next stage of the STANDARD journey.
02	advanceByOperator(orderId: Long, userId: Long, expected: String): String	Validates the expected stage and advances it.
03	advanceToStage(order, mission, stage, actor, note): void	Saves mission / order stage and history, notifies and frees the drone at the end.

3.18.3.c DroneTelemetryMqttListener

Table 3.18.3.c: DroneTelemetryMqttListener description

No	Method	Description
01	messageArrived(topic: String, msg: MqttMessage): void	Receives lockr/drones/+/telemetry.

3.18.3.d DroneTelemetryService

Table 3.18.3.d: DroneTelemetryService description

No	Method	Description
01	ingest(droneCode: String, payload: JsonNode): void	Verifies the HMAC signature, stores the latest sample and forwards the position.

3.19 Drone Parcel Drop Incident

If a parcel falls during a flight the technician reports a drop incident with location, telemetry and photos. The drone is grounded, incident tickets are created, a return-to-launch is requested, a recovery technician is assigned, and the admin proposes free redelivery or compensation that the customer accepts or disputes.

3.19.1 Class Diagram

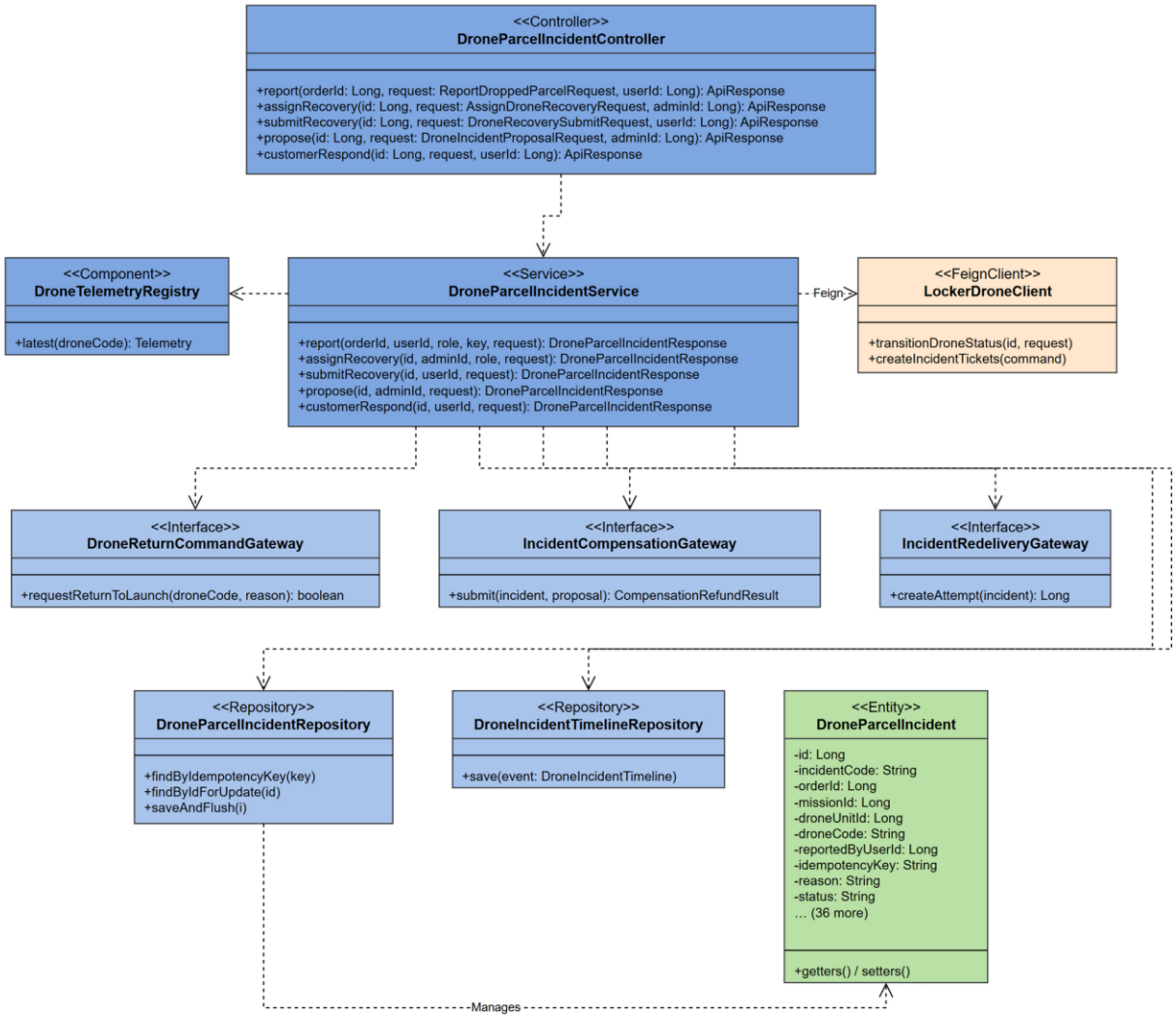


Figure 3.19.1: Class diagram - Drone Parcel Drop Incident

3.19.2 Report a Dropped Parcel

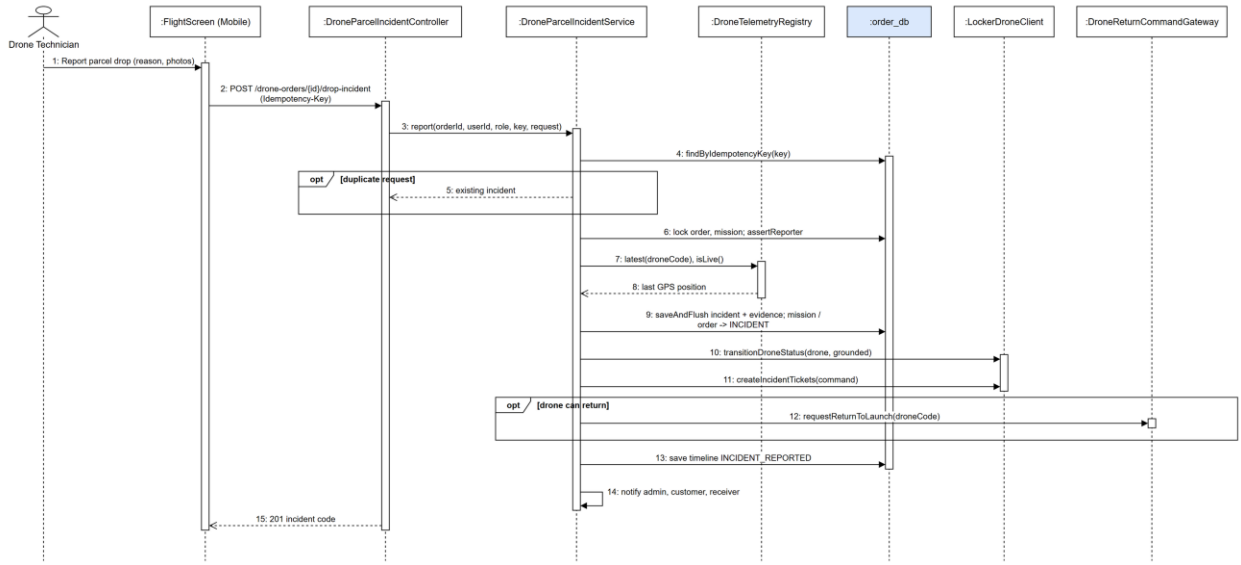


Figure 3.19.2: Sequence diagram - Report a Dropped Parcel

3.19.3 Recovery, Proposal and Customer Decision

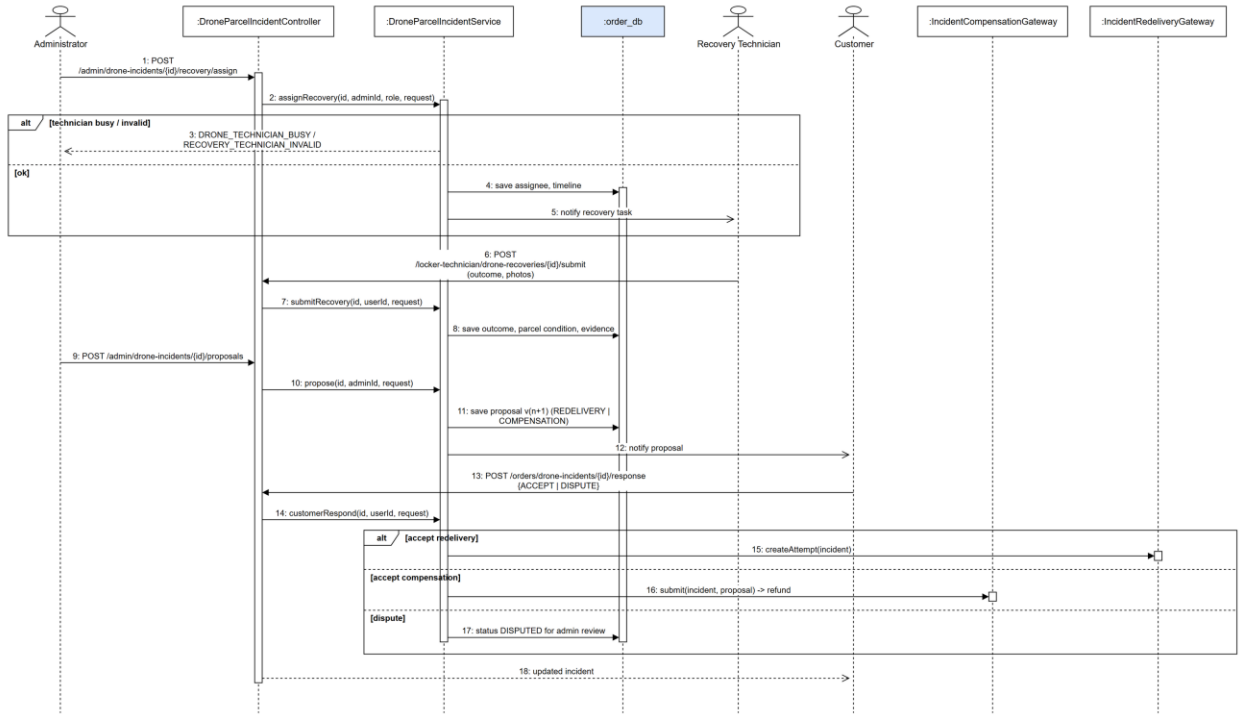


Figure 3.19.3: Sequence diagram - Recovery, Proposal and Customer Decision

3.19.4 Class Specification

3.19.4.a DroneParcelIncidentController

Table 3.19.4.a: DroneParcelIncidentController description

No	Method	Description
01	report(orderId: Long, request: ReportDroppedParcelRequest, userId: Long): ApiResponse	Technician reports a dropped parcel.
02	assignRecovery(id: Long, request: AssignDroneRecoveryRequest, adminId: Long): ApiResponse	Admin assigns a recovery technician.
03	submitRecovery(id: Long, request: DroneRecoverySubmitRequest, userId: Long): ApiResponse	Technician submits the recovery result.
04	propose(id: Long, request: DroneIncidentProposalRequest, adminId: Long): ApiResponse	Admin proposes redelivery / compensation.
05	customerRespond(id: Long, request, userId: Long): ApiResponse	Customer accepts or disputes the proposal.

3.19.4.b DroneParcelIncidentService

Table 3.19.4.b: DroneParcelIncidentService description

No	Method	Description
01	report(orderId, userId, role, key, request): DroneParcelIncidentResponse	Idempotent report: stores location / evidence, marks mission and order, grounds the drone, creates tickets and requests return-to-launch.
02	assignRecovery(id, adminId, role, request): DroneParcelIncidentResponse	Checks the technician is valid and not busy and assigns the recovery.
03	submitRecovery(id, userId, request): DroneParcelIncidentResponse	Saves outcome, parcel condition and evidence.
04	propose(id, adminId, request): DroneParcelIncidentResponse	Creates a versioned redelivery / compensation proposal according to the policy.
05	customerRespond(id, userId, request): DroneParcelIncidentResponse	Accept -> create redelivery or submit compensation refund; dispute -> admin review.

3.20 Locker Incident Ticket

A customer who cannot use a box reports it from the order. The box goes to PENDING_CHECK, a ticket is created and routed to the technician responsible for the locker, and the order is paused (no overtime). The technician claims the ticket within the SLA, adds notes and before / after photos, then resolves (box returns to service) or rejects it.

3.20.1 Class Diagram

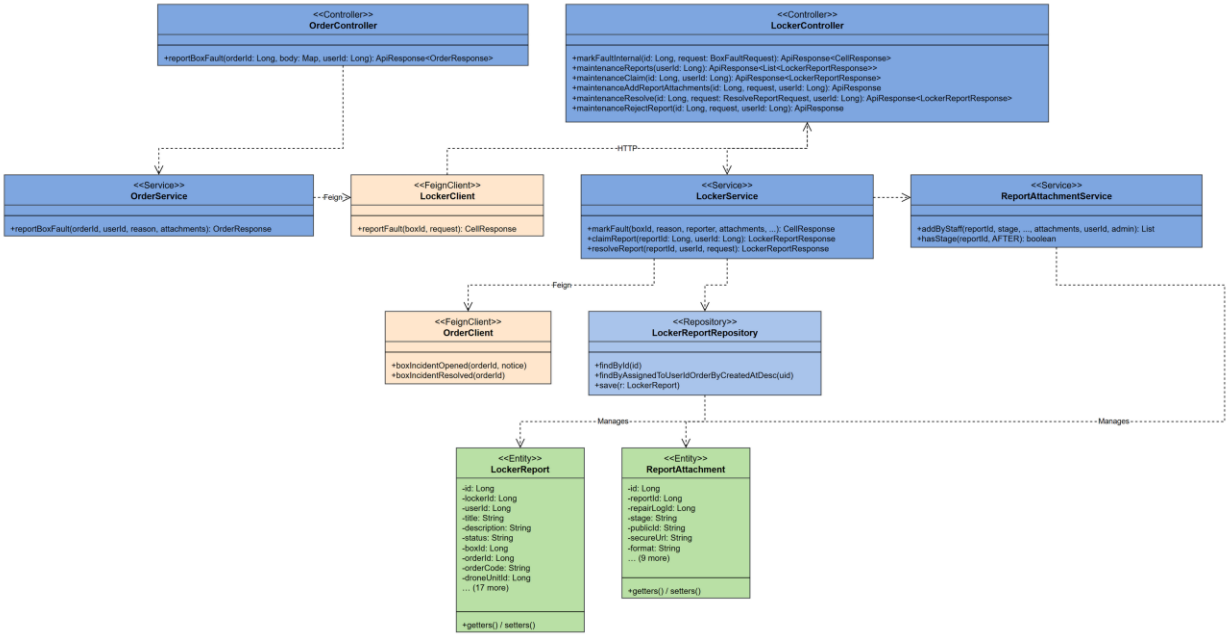


Figure 3.20.1: Class diagram - Locker Incident Ticket

3.20.2 Customer Reports a Box Fault

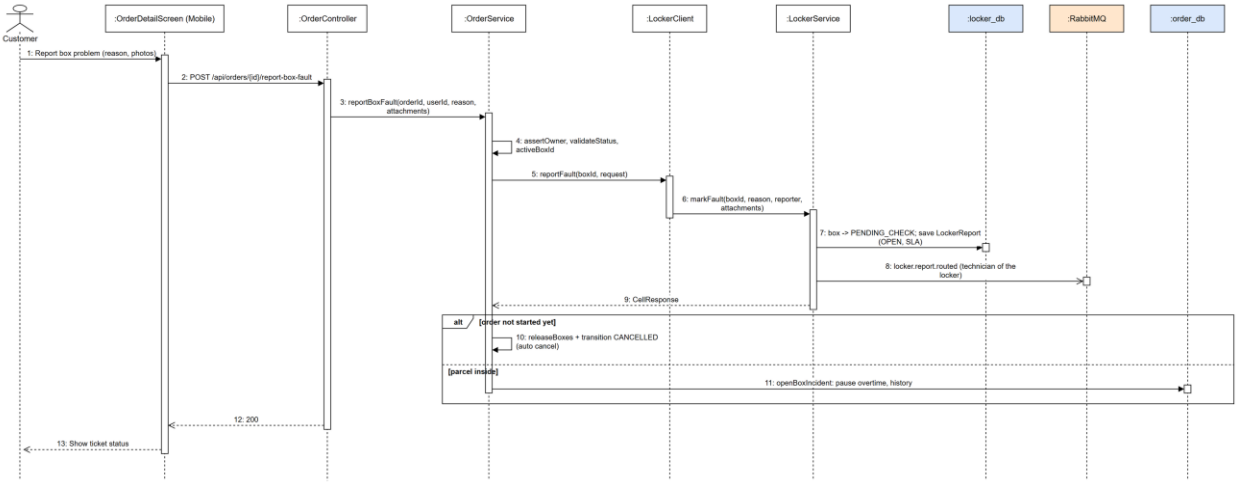


Figure 3.20.2: Sequence diagram - Customer Reports a Box Fault

3.20.3 Technician Claims and Resolves a Ticket

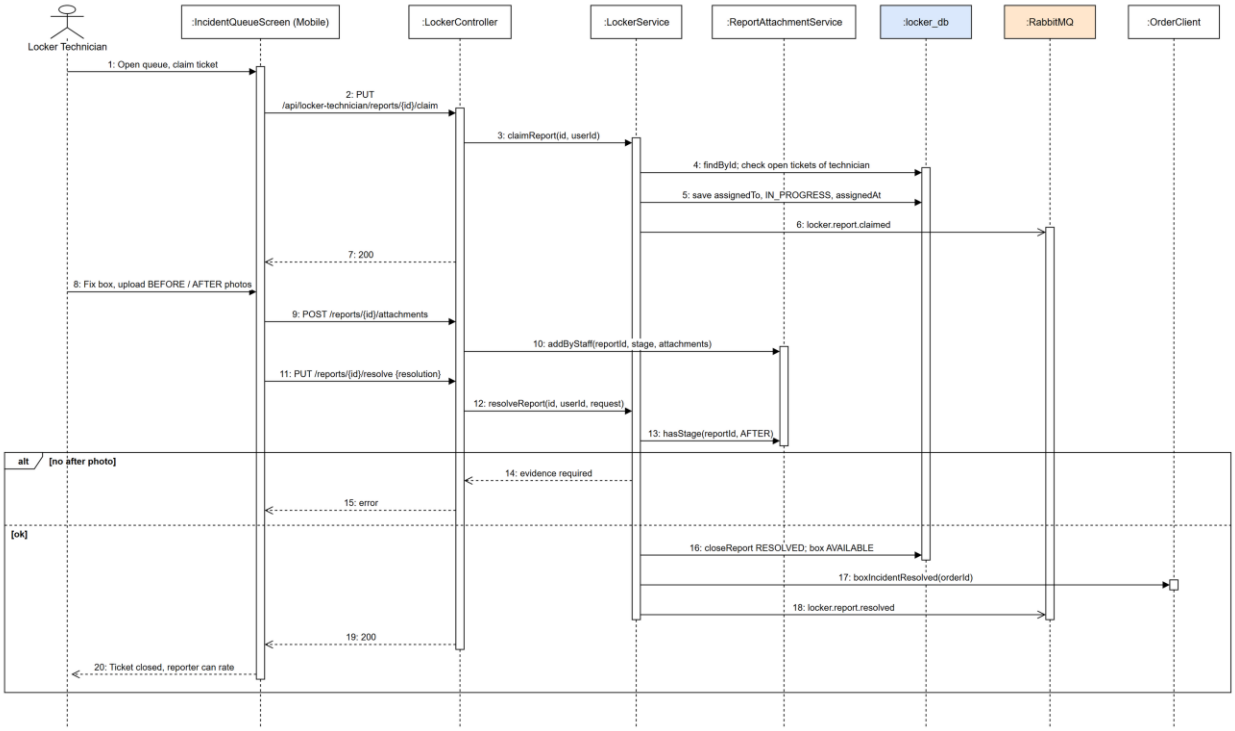


Figure 3.20.3: Sequence diagram - Technician Claims and Resolves a Ticket

3.20.4 Class Specification

3.20.4.a OrderController

Table 3.20.4.a: OrderController description

No	Method	Description
01	reportBoxFault(orderId: Long, body: Map, userId: Long): ApiResponse<OrderResponse>	Customer reports the box of an order.

3.20.4.b LockerController

Table 3.20.4.b: LockerController description

No	Method	Description
01	markFaultInternal(id: Long, request: BoxFaultRequest): ApiResponse<CellResponse>	Internal: opens a ticket for a box.
02	maintenanceReports(userId: Long): ApiResponse<List<LockerReportResponse>>	Technician ticket queue.
03	maintenanceClaim(id: Long, userId: Long): ApiResponse<LockerReportResponse>	Technician claims a ticket.
04	maintenanceAddReportAttachments(id: Long, request, userId: Long): ApiResponse	Adds before / during / after photos.

No	Method	Description
05	maintenanceResolve(id: Long, request: ResolveReportRequest, userId: Long): ApiResponse<LockerReportResponse>	Resolves the ticket and clears the fault.
06	maintenanceRejectReport(id: Long, request, userId: Long): ApiResponse	Rejects a wrong report with feedback.

3.20.4.c OrderService

Table 3.20.4.c: OrderService description

No	Method	Description
01	reportBoxFault(orderId, userId, reason, attachments): OrderResponse	Checks owner / status, calls locker-service and opens the box incident on the order (or auto-cancels).

3.20.4.d LockerService

Table 3.20.4.d: LockerService description

No	Method	Description
01	markFault(boxId, reason, reporter, attachments, ...): CellResponse	Sets the box PENDING_CHECK, creates the LockerReport, routes it to the locker technician and publishes locker.report.routed.
02	claimReport(reportId: Long, userId: Long): LockerReportResponse	Assigns the ticket (SLA) and publishes locker.report.claimed.
03	resolveReport(reportId, userId, request): LockerReportResponse	Requires the after photo, closes the ticket, clears the fault and notifies the order.

3.21 Periodic Inspection

Administrators plan periodic inspections of a locker or a drone (checklist, interval, time slot, technician). A scheduler reminds the technician when an inspection is due; the technician records Pass / Fail for each item with photos, and failed items create a ticket.

3.21.1 Class Diagram

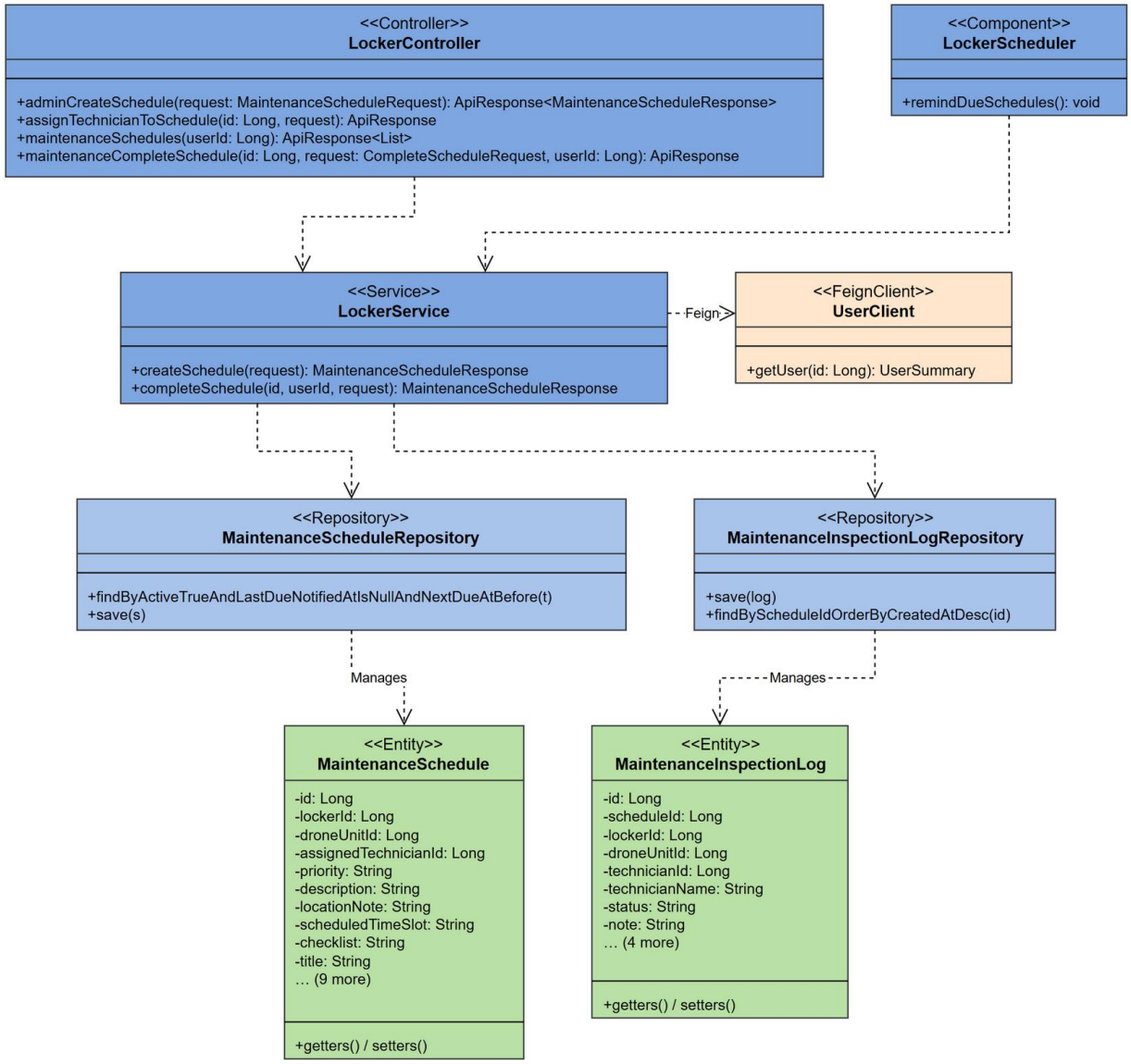


Figure 3.21.1: Class diagram - Periodic Inspection

3.21.2 Plan and Perform a Periodic Inspection

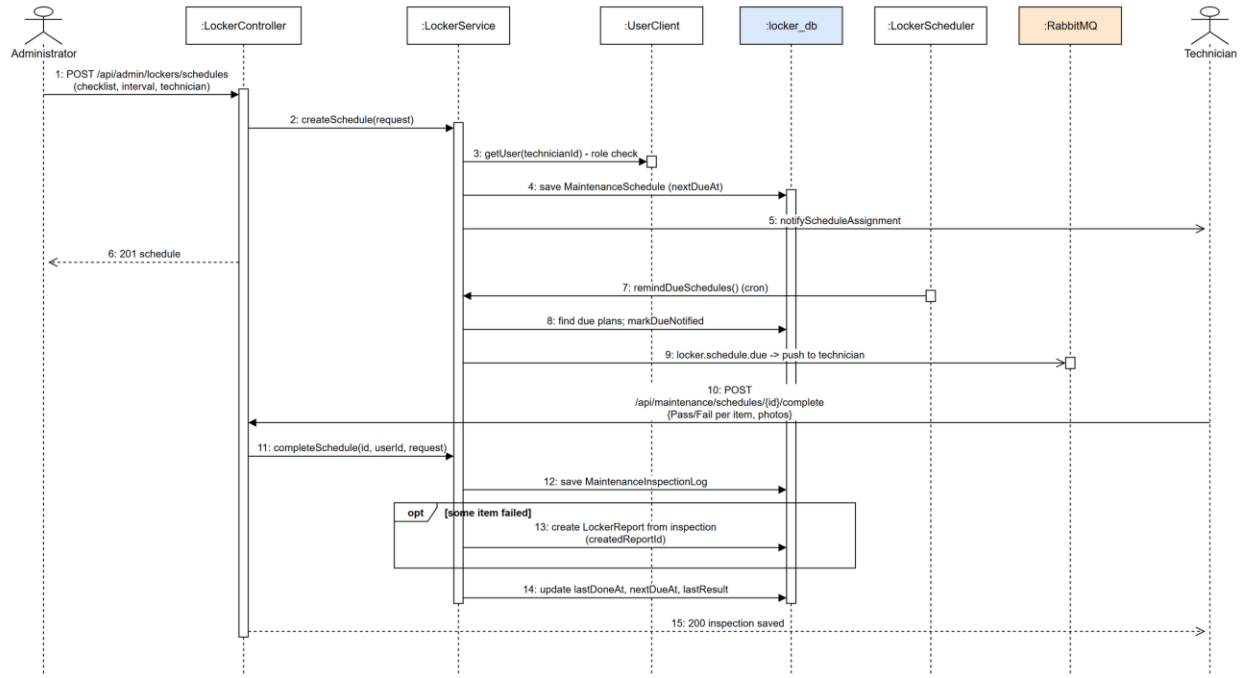


Figure 3.21.2: Sequence diagram - Plan and Perform a Periodic Inspection

3.21.3 Class Specification

3.21.3.a LockerController

Table 3.21.3.a: LockerController description

No	Method	Description
01	adminCreateSchedule(request: MaintenanceScheduleRequest): ApiResponse<MaintenanceScheduleResponse>	Admin creates an inspection plan.
02	assignTechnicianToSchedule(id: Long, request): ApiResponse	Admin assigns the technician.
03	maintenanceSchedules(userId: Long): ApiResponse<List>	Technician's due inspections.
04	maintenanceCompleteSchedule(id: Long, request: CompleteScheduleRequest, userId: Long): ApiResponse	Technician submits the inspection result.

3.21.3.b LockerService

Table 3.21.3.b: LockerService description

No	Method	Description
01	createSchedule(request): MaintenanceScheduleResponse	Validates the target and technician role, saves the plan and notifies the technician.
02	completeSchedule(id, userId, request): MaintenanceScheduleResponse	Saves the inspection log, moves next due date and creates a ticket for failed items.

3.21.3.c LockerScheduler

Table 3.21.3.c: LockerScheduler description

No	Method	Description
01	remindDueSchedules(): void	Scheduled: publishes locker.schedule.due for plans that became due.

3.22 Locker Controller Provisioning and Cabinet Control

A Raspberry Pi controller announces itself over MQTT (iot/{mac}/discovery). The administrator assigns it to a locker; iot-service sends the setup with the box layout and tracks progress. Technicians and administrators open boxes, move the sliding lid and read the panel status remotely; every action is written to cabinet_control_logs.

3.22.1 Class Diagram

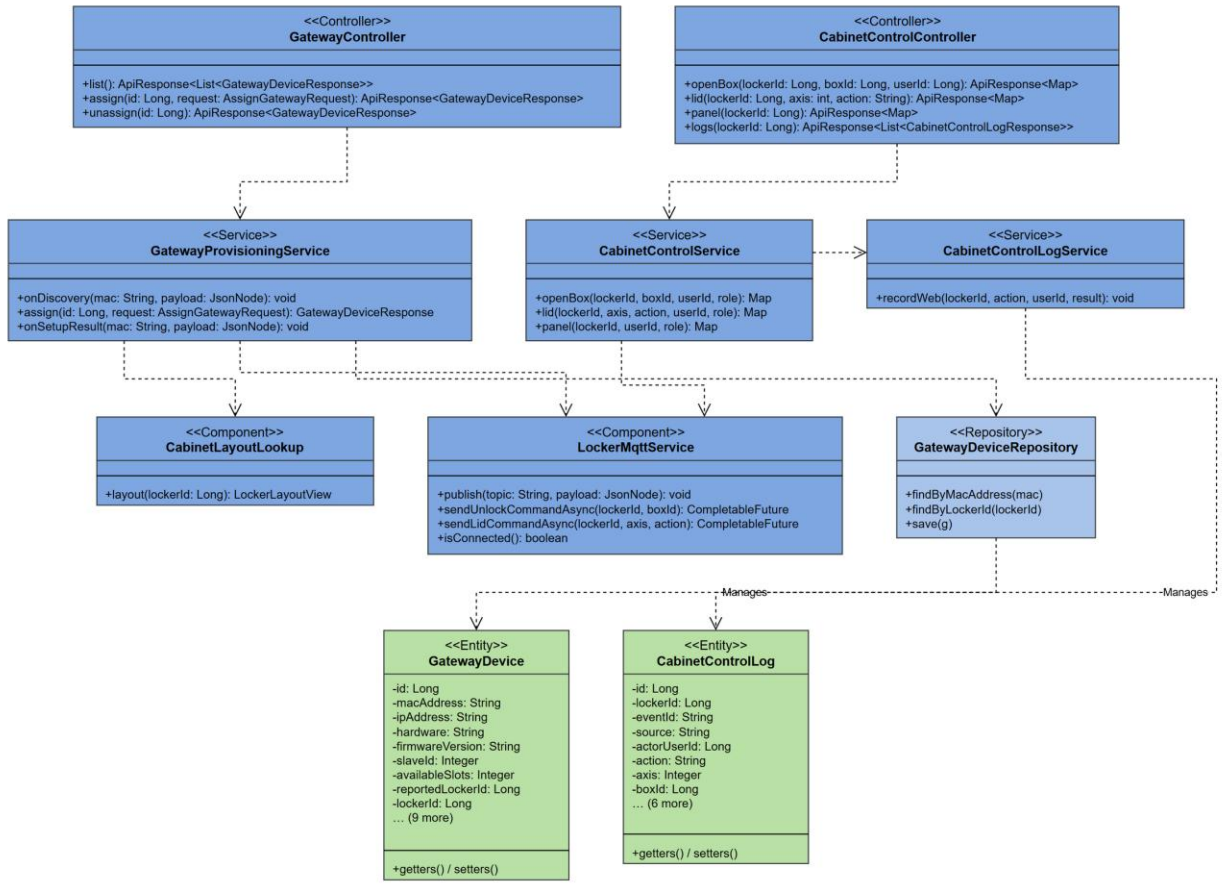


Figure 3.22.1: Class diagram - Locker Controller Provisioning and Cabinet Control

3.22.2 Assign a Controller to a Locker

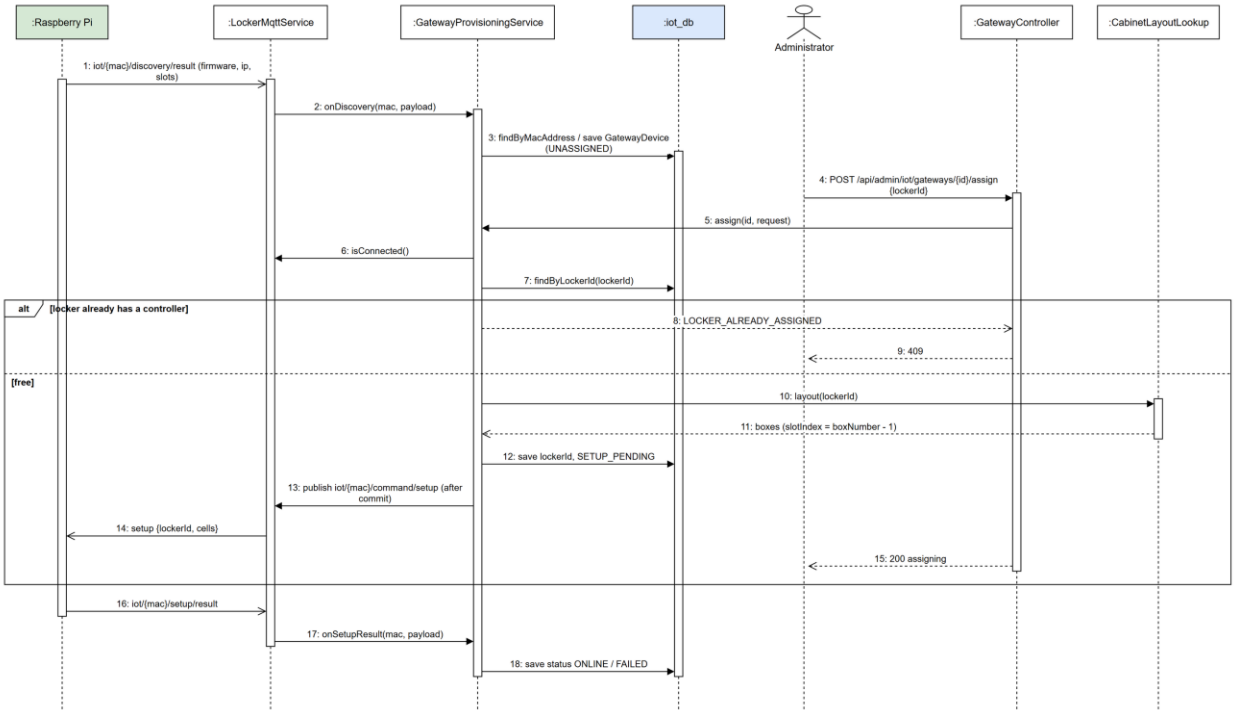


Figure 3.22.2: Sequence diagram - Assign a Controller to a Locker

3.22.3 Remote Open Box from the Admin Web

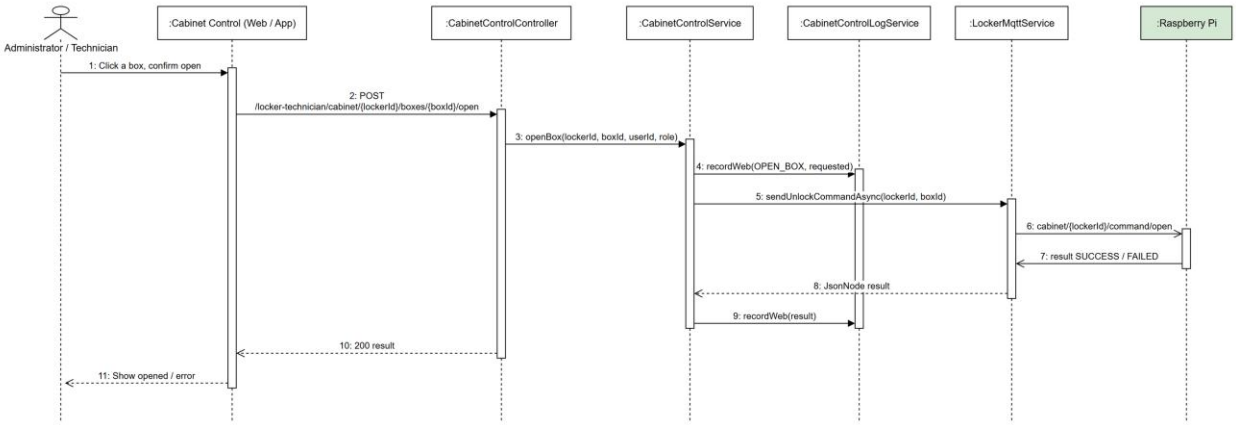


Figure 3.22.3: Sequence diagram - Remote Open Box from the Admin Web

3.22.4 Class Specification

3.22.4.a GatewayController

Table 3.22.4.a: GatewayController description

No	Method	Description
01	list(): ApiResponse<List<GatewayDeviceResponse>>	Controllers discovered / assigned.

No	Method	Description
02	assign(id: Long, request: AssignGatewayRequest): ApiResponse<GatewayDeviceResponse>	Assigns a controller to a locker.
03	unassign(id: Long): ApiResponse<GatewayDeviceResponse>	Clears the setup on the Pi.

3.22.4.b CabinetControlController

Table 3.22.4.b: CabinetControlController description

No	Method	Description
01	openBox(lockerId: Long, boxId: Long, userId: Long): ApiResponse<Map>	Remote box open by technician / admin.
02	lid(lockerId: Long, axis: int, action: String): ApiResponse<Map>	Moves the sliding lid of a drone box.
03	panel(lockerId: Long): ApiResponse<Map>	Live panel status from the Pi.
04	logs(lockerId: Long): ApiResponse<List<CabinetControlLogResponse>>	Control log of the cabinet.

3.22.4.c GatewayProvisioningService

Table 3.22.4.c: GatewayProvisioningService description

No	Method	Description
01	onDiscovery(mac: String, payload: JsonNode): void	Creates / updates the GatewayDevice from a discovery message.
02	assign(id: Long, request: AssignGatewayRequest): GatewayDeviceResponse	Requires the broker, checks the locker is free, validates cells and publishes iot/{mac}/command/setup after commit.
03	onSetupResult(mac: String, payload: JsonNode): void	Stores the setup result.

3.22.4.d CabinetControlService

Table 3.22.4.d: CabinetControlService description

No	Method	Description
01	openBox(lockerId, boxId, userId, role): Map	Logs the web action and sends the unlock command.
02	lid(lockerId, axis, action, userId, role): Map	Sends a lid command.
03	panel(lockerId, userId, role): Map	Queries the Pi panel.

3.23 Bluetooth Open Test

For testing offline opening, the administrator grants a customer account access to a locker. The app asks iot-service for a one-time Ed25519-signed token (valid 120 s) for one box and sends it to the Pi over Bluetooth; the Pi verifies it with the public key downloaded from /api/iot/ble/public-key.

3.23.1 Class Diagram

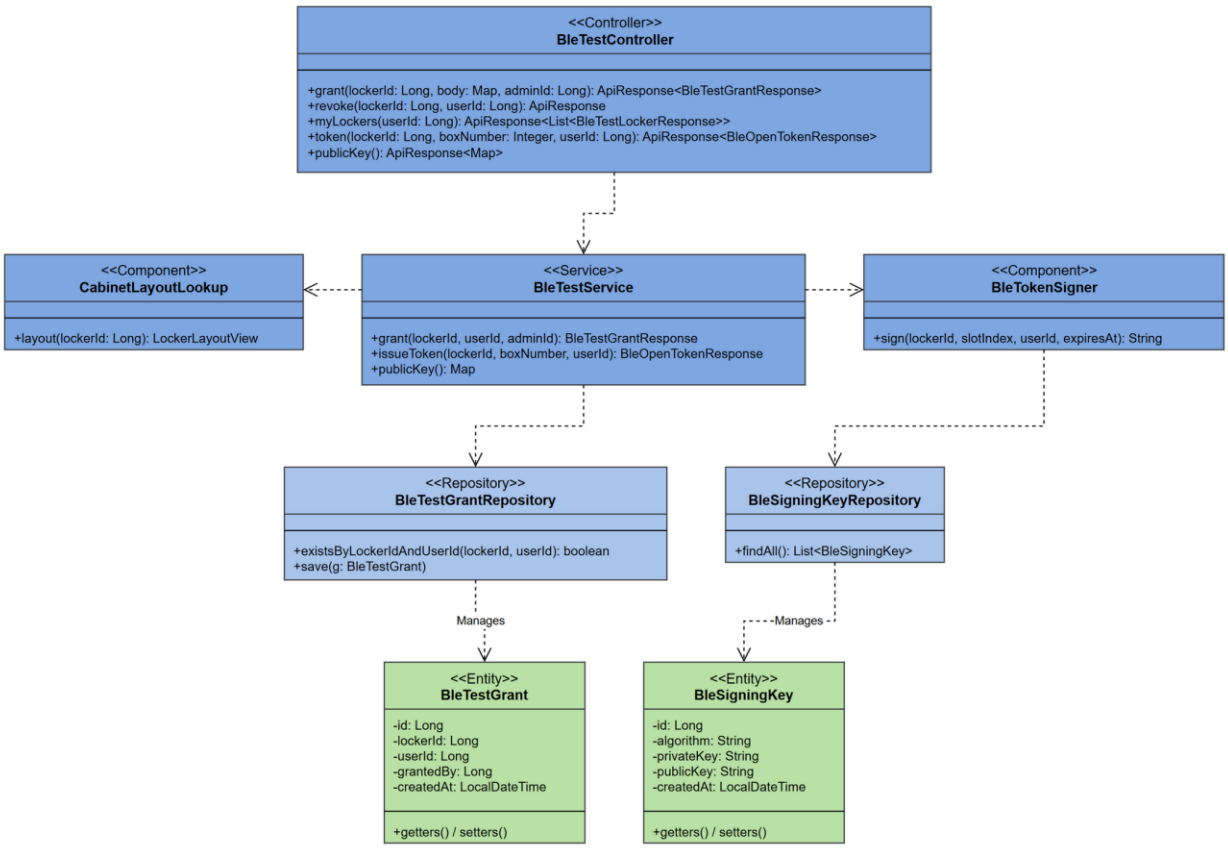


Figure 3.23.1: Class diagram - Bluetooth Open Test

3.23.2 Open a Box by Bluetooth

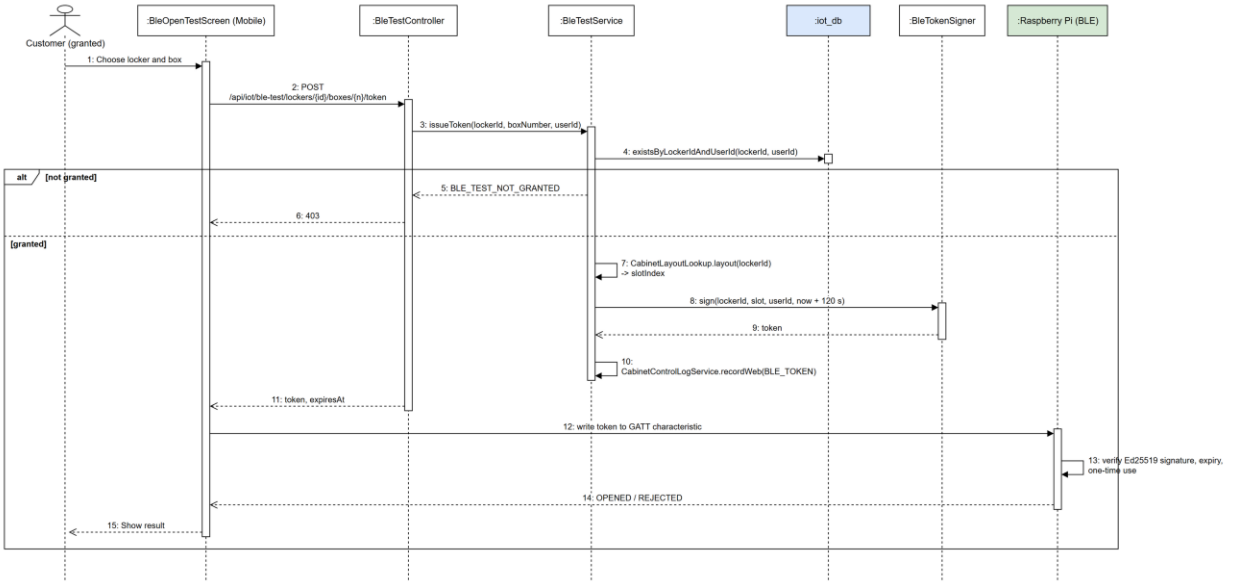


Figure 3.23.2: Sequence diagram - Open a Box by Bluetooth

3.23.3 Class Specification

3.23.3.a BleTestController

Table 3.23.3.a: BleTestController description

No	Method	Description
01	grant(lockerId: Long, body: Map, adminId: Long): ApiResponse<BleTestGrantResponse>	Admin grants BLE test access to a user.
02	revoke(lockerId: Long, userId: Long): ApiResponse	Admin revokes the grant.
03	myLockers(userId: Long): ApiResponse<List<BleTestLockerResponse>>	Lockers the user may test.
04	token(lockerId: Long, boxNumber: Integer, userId: Long): ApiResponse<BleOpenTokenResponse>	Issues a signed open token.
05	publicKey(): ApiResponse<Map>	Public key for the Pi (public route).

3.23.3.b BleTestService

Table 3.23.3.b: BleTestService description

No	Method	Description
01	grant(lockerId, userId, adminId): BleTestGrantResponse	Saves a grant for the locker and user.
02	issueToken(lockerId, boxNumber, userId): BleOpenTokenResponse	Checks the grant and box, signs a 120 s token and logs it.
03	publicKey(): Map	Returns the current verification key.

3.24 Notifications and Realtime Updates

Services publish domain events to the RabbitMQ topic exchange laundry.events or call notification-service directly. notification-service stores in-app notifications, sends FCM push to registered devices and pushes STOMP messages over /ws so that the app and admin web refresh orders, wallet, catalog and maintenance screens in real time.

3.24.1 Class Diagram

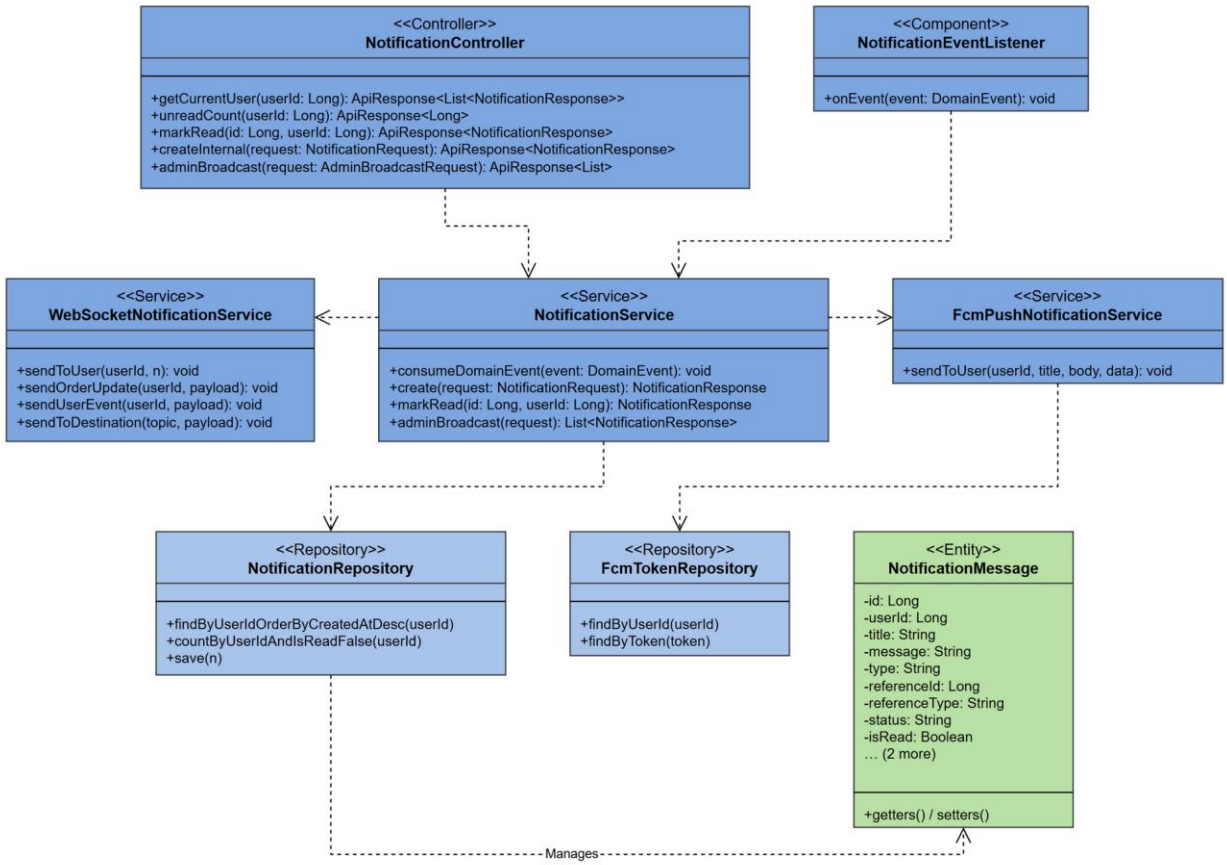


Figure 3.24.1: Class diagram - Notifications and Realtime Updates

3.24.2 Deliver a Domain Event to Users

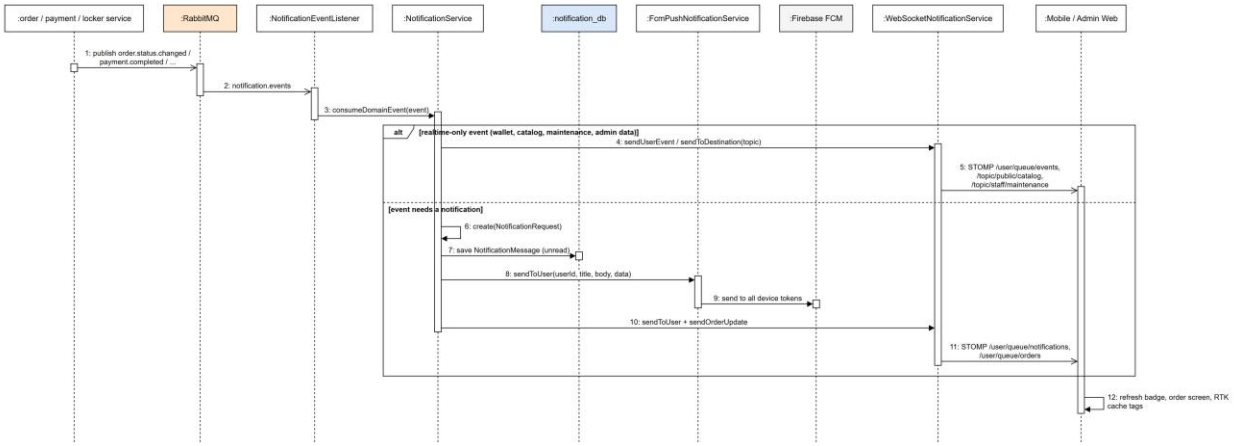


Figure 3.24.2: Sequence diagram - Deliver a Domain Event to Users

3.24.3 Class Specification

3.24.3.a NotificationController

Table 3.24.3.a: NotificationController description

No	Method	Description
01	getCurrentUser(userId: Long): ApiResponse<List<NotificationResponse>>	Notifications of the user.
02	unreadCount(userId: Long): ApiResponse<Long>	Unread badge.
03	markRead(id: Long, userId: Long): ApiResponse<NotificationResponse>	Marks one as read.
04	createInternal(request: NotificationRequest): ApiResponse<NotificationResponse>	Internal: create and deliver a notification.
05	adminBroadcast(request: AdminBroadcastRequest): ApiResponse<List>	Admin broadcast.

3.24.3.b NotificationEventListener

Table 3.24.3.b: NotificationEventListener description

No	Method	Description
01	onEvent(event: DomainEvent): void	Consumes notification.events from RabbitMQ.

3.24.3.c NotificationService

Table 3.24.3.c: NotificationService description

No	Method	Description
01	consumeDomainEvent(event: DomainEvent): void	Maps an event to realtime destinations and, when needed, to a stored notification.
02	create(request: NotificationRequest): NotificationResponse	Saves the notification, sends FCM push and STOMP to the user.
03	markRead(id: Long, userId: Long): NotificationResponse	Marks as read.
04	adminBroadcast(request): List<NotificationResponse>	Creates notifications for a group of users.

3.25 AI Assistant (RAG)

Users ask questions in Vietnamese about Lock.R. assistant-service normalises the question, embeds it with Voyage AI, searches the kb_chunks vector index (pgvector, cosine) limited to documents allowed for the user's roles, adds the user's own order context when relevant and asks Claude for an answer with citations; it refuses when nothing relevant is found. Administrators upload knowledge documents that are chunked and embedded in the background.

3.25.1 Class Diagram

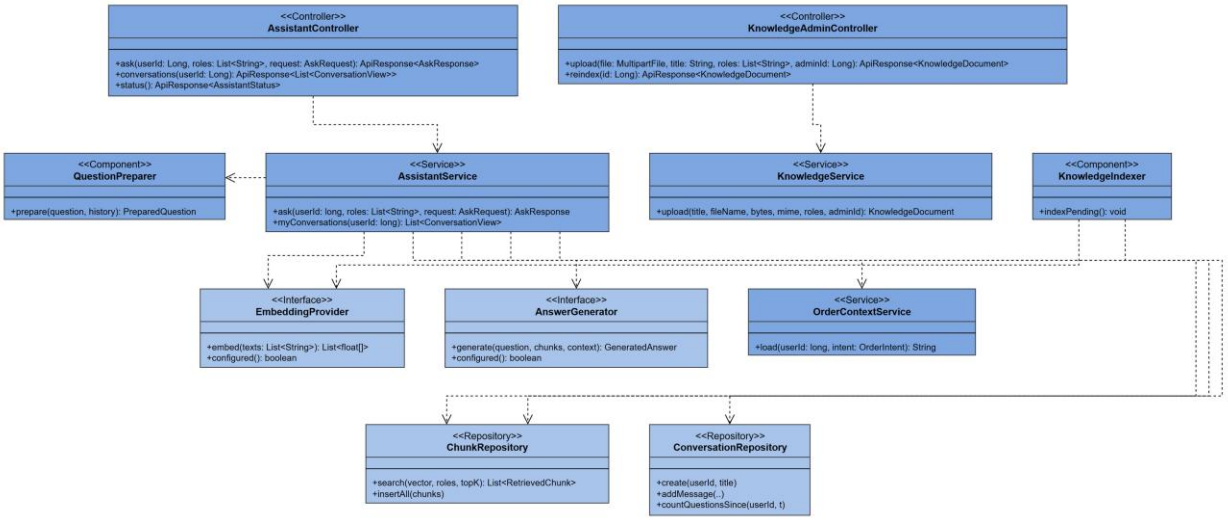


Figure 3.25.1: Class diagram - AI Assistant (RAG)

3.25.2 Ask the AI Assistant

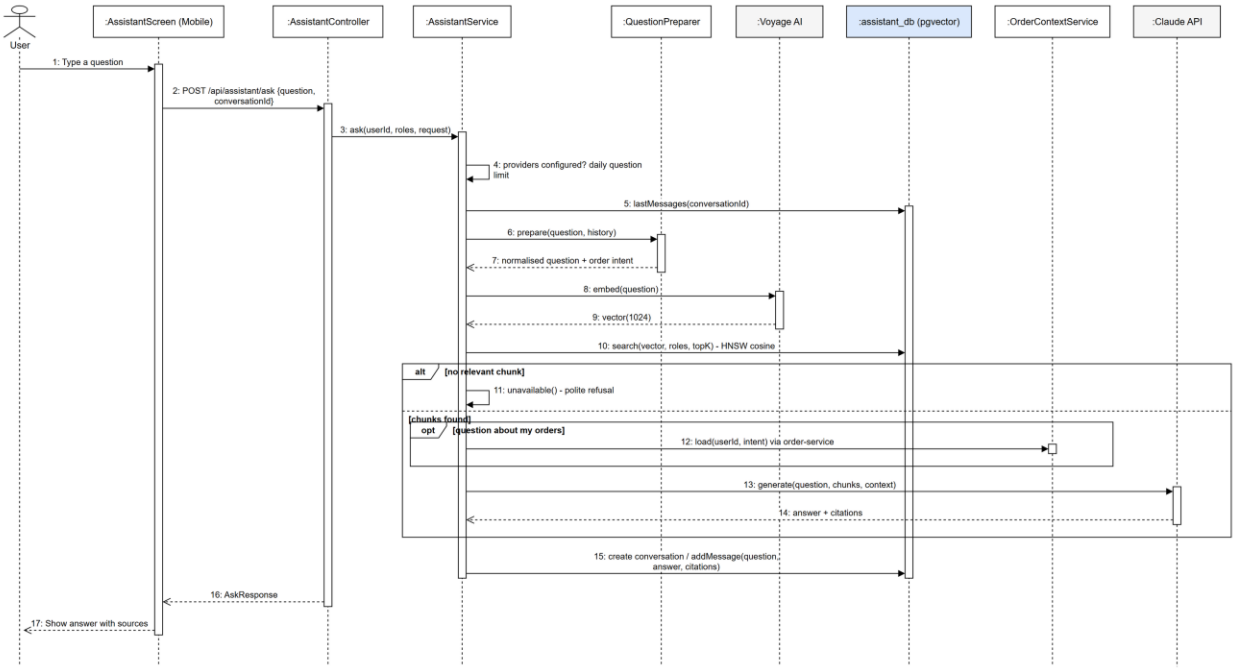


Figure 3.25.2: Sequence diagram - Ask the AI Assistant

3.25.3 Upload and Index a Knowledge Document

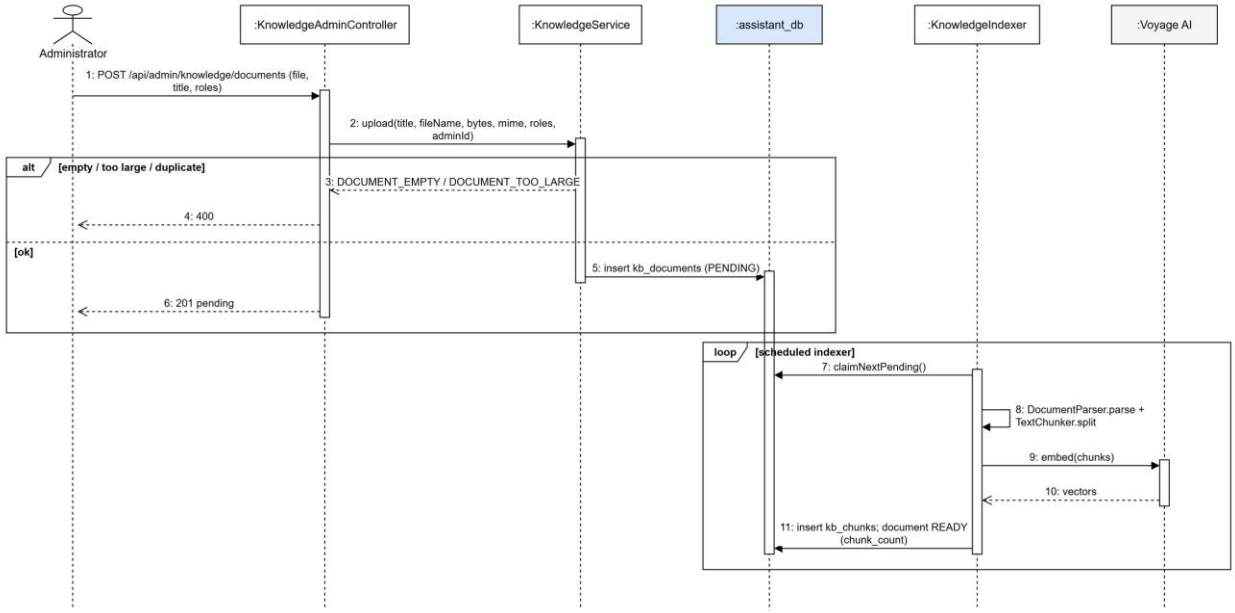


Figure 3.25.3: Sequence diagram - Upload and Index a Knowledge Document

3.25.4 Class Specification

3.25.4.a AssistantController

Table 3.25.4.a: AssistantController description

No	Method	Description
01	ask(userId: Long, roles: List<String>, request: AskRequest): ApiResponse<AskResponse>	Asks a question in a new or existing conversation.
02	conversations(userId: Long): ApiResponse<List<ConversationView>>	Conversations of the user.
03	status(): ApiResponse<AssistantStatus>	Whether the providers are configured.

3.25.4.b KnowledgeAdminController

Table 3.25.4.b: KnowledgeAdminController description

No	Method	Description
01	upload(file: MultipartFile, title: String, roles: List<String>, adminId: Long): ApiResponse<KnowledgeDocument>	Uploads a knowledge document.
02	reindex(id: Long): ApiResponse<KnowledgeDocument>	Re-indexes a document.

3.25.4.c AssistantService

Table 3.25.4.c: AssistantService description

No	Method	Description
01	ask(userId: long, roles: List<String>, request: AskRequest): AskResponse	Rate-limits, prepares the question, retrieves chunks, adds order context, generates the answer and stores both messages.
02	myConversations(userId: long): List<ConversationView>	Lists conversations.

3.25.4.d KnowledgeService

Table 3.25.4.d: KnowledgeService description

No	Method	Description
01	upload(title, fileName, bytes, mime, roles, adminId): KnowledgeDocument	Validates size / duplicates (checksum) and inserts a PENDING document.

3.25.4.e KnowledgeIndexer

Table 3.25.4.e: KnowledgeIndexer description

No	Method	Description
01	indexPending(): void	Scheduled: claims the next PENDING document, parses, chunks, embeds and stores chunks.

3.26 Business Settings

Business rules (prices, time limits, SLA, refund policy, payment methods, loyalty rate...) are defined in a SettingsCatalog of each service and can be changed by the administrator without redeploying. Values are validated, stored in the service's system_settings table with an audit trail, and public values are readable by the app at /api/settings/{scope}/public.

3.26.1 Class Diagram

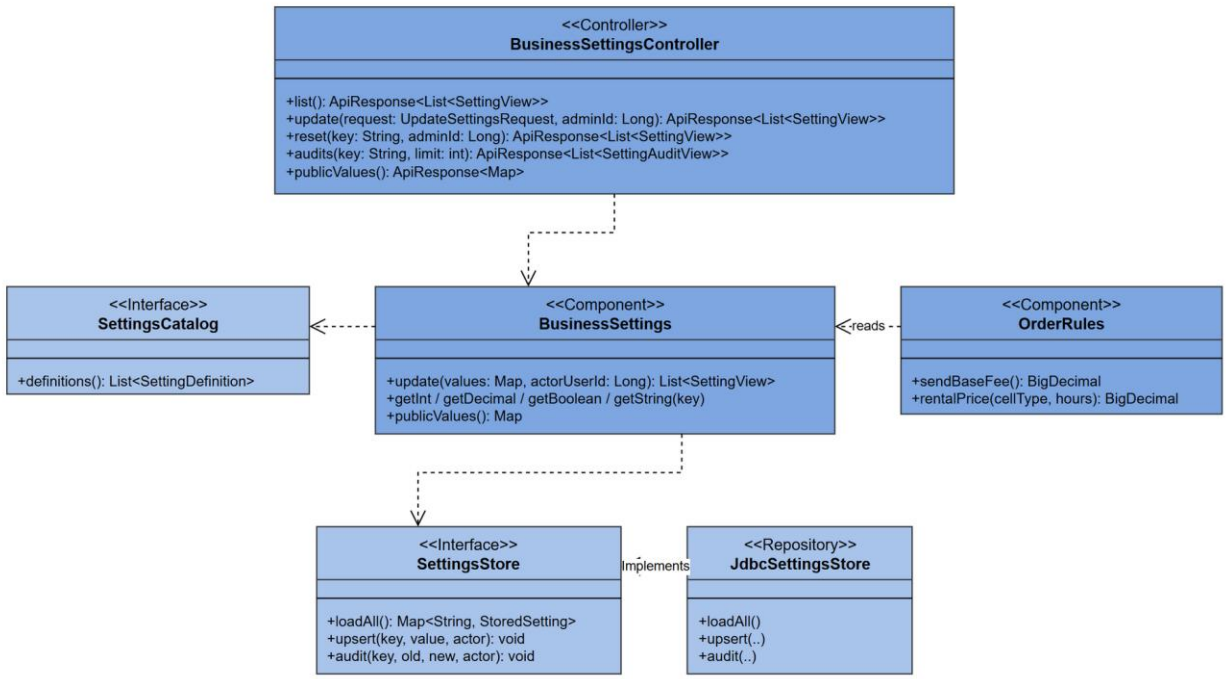


Figure 3.26.1: Class diagram - Business Settings

3.26.2 Administrator Changes a Business Rule

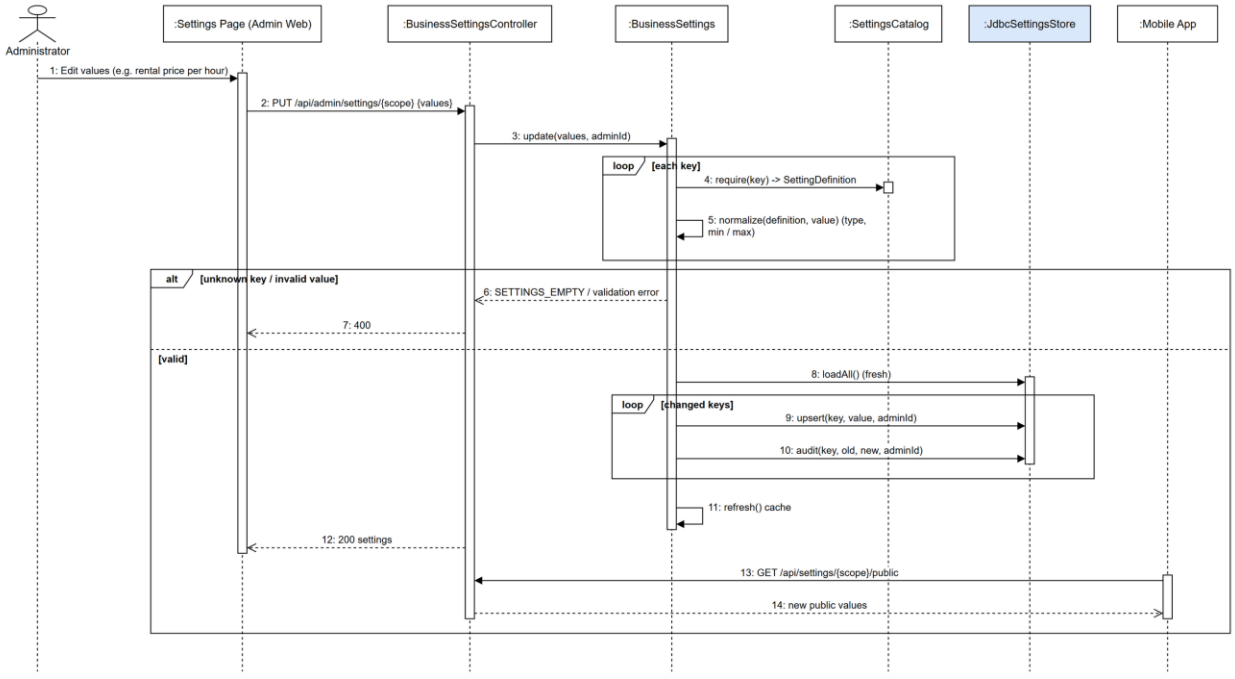


Figure 3.26.2: Sequence diagram - Administrator Changes a Business Rule

3.26.3 Class Specification

3.26.3.a BusinessSettingsController

Table 3.26.3.a: BusinessSettingsController description

No	Method	Description
01	list(): ApiResponse<List<SettingView>>	All settings of the service scope.
02	update(request: UpdateSettingsRequest, adminId: Long): ApiResponse<List<SettingView>>	Updates several settings.
03	reset(key: String, adminId: Long): ApiResponse<List<SettingView>>	Returns a setting to its default.
04	audits(key: String, limit: int): ApiResponse<List<SettingAuditView>>	Change history.
05	publicValues(): ApiResponse<Map>	Public values for clients.

3.26.3.b BusinessSettings

Table 3.26.3.b: BusinessSettings description

No	Method	Description
01	update(values: Map, actorUserId: Long): List<SettingView>	Validates every key against the catalog, normalises values, upserts changed ones, writes audits and refreshes the cache.
02	getInt / getDecimal / getBoolean / getString(key)	Typed reads used by the *Rules classes.
03	publicValues(): Map	Values flagged public.

3.27 Admin Dashboard and Revenue Reports

The admin dashboard shows revenue by day, service, payment method, locker, store and customer for a date range. order-service combines its orders with the money actually collected from payment-service and resolves locker, store and customer names.

3.27.1 Class Diagram

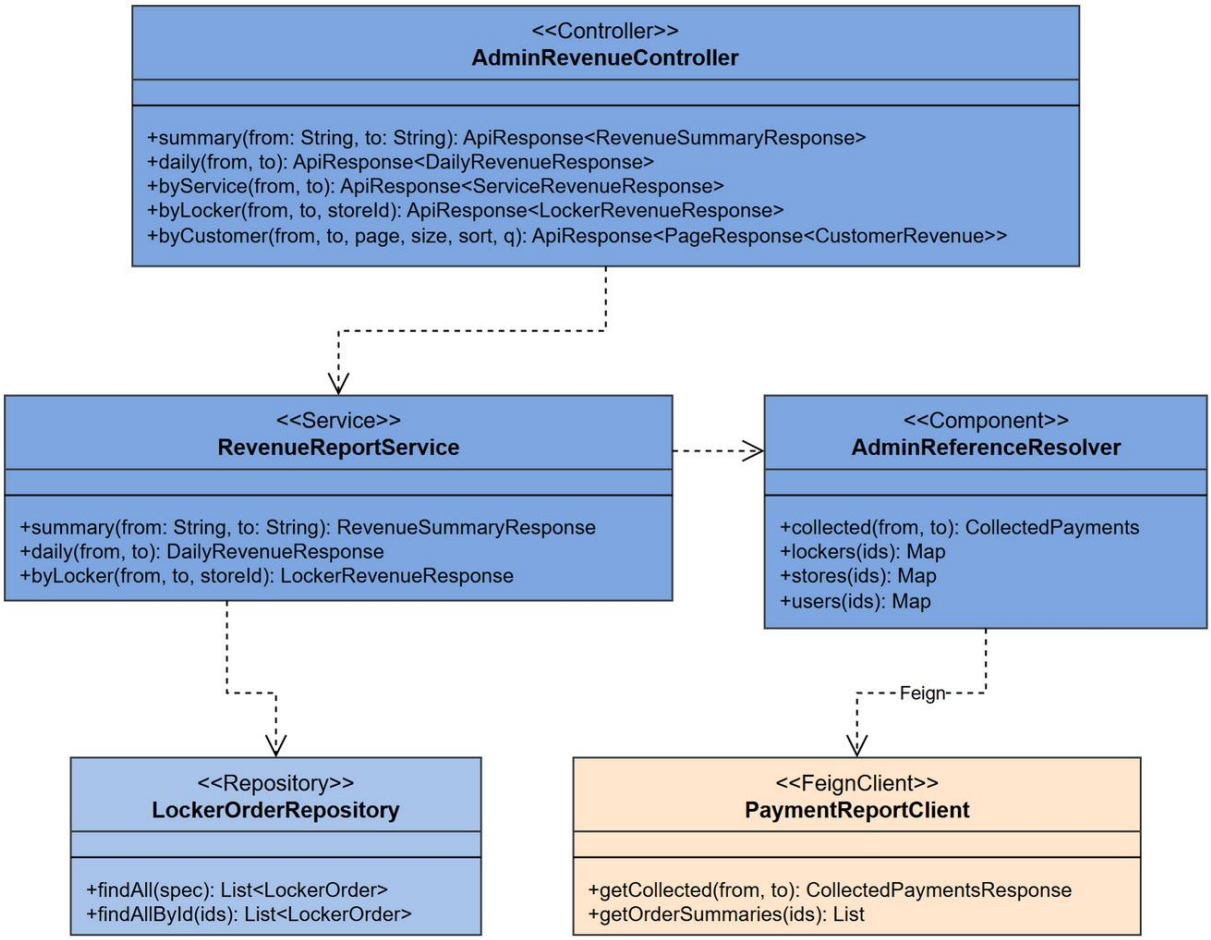


Figure 3.27.1: Class diagram - Admin Dashboard and Revenue Reports

3.27.2 View Revenue Summary

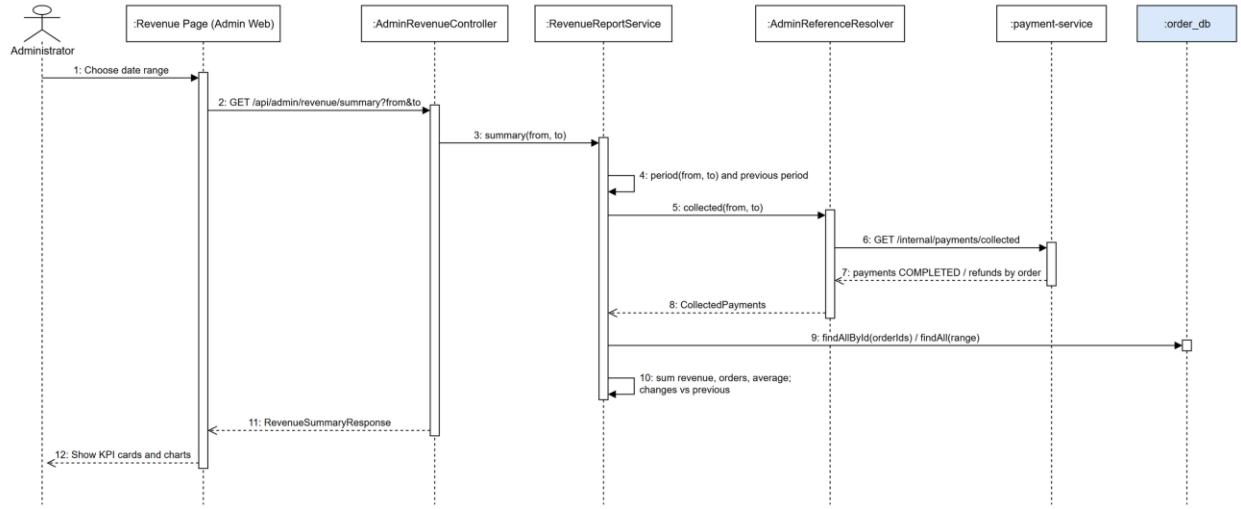


Figure 3.27.2: Sequence diagram - View Revenue Summary

3.27.3 Class Specification

3.27.3.a AdminRevenueController

Table 3.27.3.a: AdminRevenueController description

No	Method	Description
01	summary(from: String, to: String): ApiResponse<RevenueSummaryResponse>	Totals and change vs the previous period.
02	daily(from, to): ApiResponse<DailyRevenueResponse>	Revenue per day.
03	byService(from, to): ApiResponse<ServiceRevenueResponse>	Revenue per order type.
04	byLocker(from, to, storeId): ApiResponse<LockerRevenueResponse>	Revenue per locker.
05	byCustomer(from, to, page, size, sort, q): ApiResponse<PageResponse<CustomerRevenue>>	Top customers.

3.27.3.b RevenueReportService

Table 3.27.3.b: RevenueReportService description

No	Method	Description
01	summary(from: String, to: String): RevenueSummaryResponse	Loads orders and collected payments of the period and the previous one and computes totals and changes.
02	daily(from, to): DailyRevenueResponse	Groups by business day.
03	byLocker(from, to, storeId): LockerRevenueResponse	Groups by locker with names.

3.28 System Scheduled Jobs

Background jobs keep orders and boxes consistent: cancel unpaid orders, release boxes of completed orders, remind receivers before the pickup deadline, move overdue parcels to storage (EXPIRED), reconcile box states with locker-service and retry point refunds. Administrators can trigger them manually and see their status.

3.28.1 Class Diagram

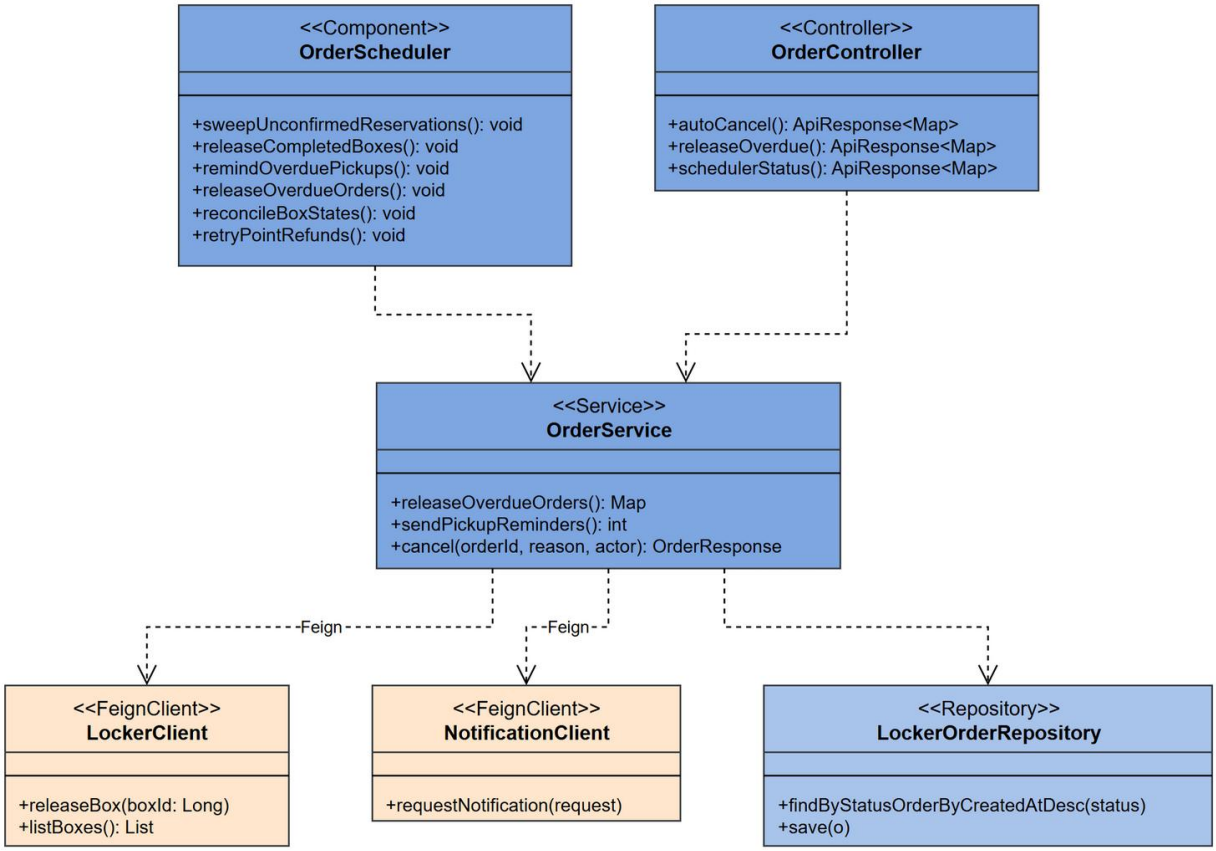


Figure 3.28.1: Class diagram - System Scheduled Jobs

3.28.2 Expire Overdue Orders

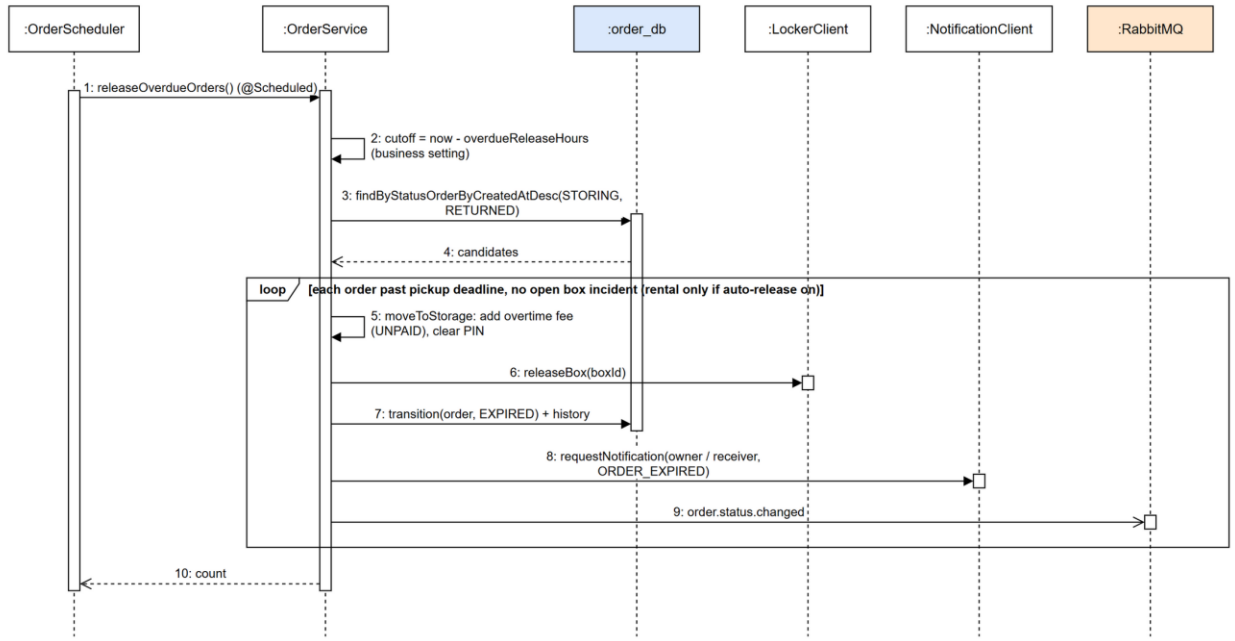


Figure 3.28.2: Sequence diagram - Expire Overdue Orders

3.28.3 Class Specification

3.28.3.a OrderScheduler

Table 3.28.3.a: OrderScheduler description

No	Method	Description
01	sweepUnconfirmedReservations(): void	Cancels orders not paid / dropped in time and releases boxes.
02	releaseCompletedBoxes(): void	Releases boxes of completed orders.
03	remindOverduePickups(): void	Sends pickup reminders.
04	releaseOverdueOrders(): void	Moves overdue parcels to storage (EXPIRED) and frees their boxes.
05	reconcileBoxStates(): void	Aligns box states with active orders.
06	retryPointRefunds(): void	Retries loyalty refunds that failed.

3.28.3.b OrderController

Table 3.28.3.b: OrderController description

No	Method	Description
01	autoCancel(): ApiResponse<Map>	Admin triggers the auto-cancel job.
02	releaseOverdue(): ApiResponse<Map>	Admin triggers overdue release.
03	schedulerStatus(): ApiResponse<Map>	Last run of each job.